AetherGain

Code Appendix

Generated 2026-10-17

45 source files included

Contents

AetherGain/App/AetherGainApp.swift
AetherGain/ContentView.swift
AetherGain/DesignSystem/AGSwipeToDelete.swift
AetherGain/DesignSystem/AGTheme.swift
AetherGain/DesignSystem/AGToast.swift
AetherGain/Models/Exercise.swift
AetherGain/Models/GameEngine.swift
AetherGain/Models/MealEntry.swift
AetherGain/Models/UserProfile.swift
AetherGain/Models/WorkoutSession.swift
AetherGain/Services/FoodClassifier.swift
AetherGain/Services/FoodClassifierRegions.swift
AetherGain/Services/FoodDatabase.swift
AetherGain/Services/NetworkMonitor.swift
AetherGain/Services/ProfileImageService.swift
AetherGain/Services/ProgressAnalytics.swift
AetherGain/Services/TargetsCalculator.swift
AetherGain/ViewModels/AuthViewModel.swift
AetherGain/ViewModels/HabitsViewModel.swift
AetherGain/ViewModels/LeaderboardViewModel.swift
AetherGain/ViewModels/MealViewModel.swift
AetherGain/ViewModels/MealViewModelRegions.swift
AetherGain/ViewModels/ProfileViewModel.swift
AetherGain/ViewModels/WorkoutViewModel.swift
AetherGain/Views/Auth/LoginView.swift
AetherGain/Views/ChartView.swift
AetherGain/Views/HabitsView.swift

AetherGain/Views/LeaderboardView.swift
AetherGain/Views/Main/HomeView.swift
AetherGain/Views/Main/MainTabView.swift
AetherGain/Views/Meal Logging/MealLogView.swift
AetherGain/Views/Onboarding/PrivacyNoticeView.swift
AetherGain/Views/Onboarding/TargetsOnboardingView.swift
AetherGain/Views/Profile/ProfileView.swift
AetherGain/Views/Progress/ProgressView.swift
AetherGain/Views/Workouts/LogWorkoutView.swift
AetherGain/Views/Workouts/WorkoutEditSheet.swift
AetherGain/Views/Workouts/WorkoutHistoryView.swift
AetherGainTests/AetherGainTests.swift
AetherGainUITests/AetherGainUITests.swift
AetherGainUITests/AetherGainUITestsLaunchTests.swift
MLTraining/check_dataset.py
MLTraining/prepare_dataset.py
MLTraining/train_model.swift
../analysis/generate_report.py

AetherGainApp.swift

AetherGain/App/AetherGainApp.swift

```swift


import SwiftUI
import FirebaseCore
import FirebaseFirestore


class AppDelegate: NSObject, UIApplicationDelegate {
    func application(_ application: UIApplication,
                     didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey : Any]? = nil) -> Bool {
        FirebaseApp.configure()



        let settings = Firestore.firestore().settings
        settings.cacheSettings = PersistentCacheSettings(sizeBytes: NSNumber(value: FirestoreCacheSizeUnlimited))
        Firestore.firestore().settings = settings


        NetworkMonitor.shared.start()

        return true
    }
}

@main
struct AetherGainApp: App {
    @UIApplicationDelegateAdaptor(AppDelegate.self) var delegate

    @StateObject private var authVM = AuthViewModel()

    var body: some Scene {
        WindowGroup {
            ZStack(alignment: .top) {
                ContentView()
                    .environmentObject(authVM)
                AGToastHost()
            }
        }
    }
```

ContentView.swift

AetherGain/ContentView.swift

```swift


import SwiftUI

struct ContentView: View {
    @EnvironmentObject var authVM: AuthViewModel
    @ObservedObject private var network = NetworkMonitor.shared
    @AppStorage("ag.privacy.accepted") private var privacyAccepted: Bool = false
    @AppStorage("ag.onboarding.targetsDone") private var targetsDone: Bool = false

    var body: some View {
        ZStack(alignment: .bottom) {
            Group {
                if !privacyAccepted {
                    PrivacyNoticeView(onDone: { privacyAccepted = true })
                } else if !authVM.isSignedIn {
                    LoginView()
                } else if !targetsDone {
                    TargetsOnboardingView(onDone: { targetsDone = true })
                } else {
                    MainTabView()
                        .onAppear { authVM.userDidInteract() }
                        .onTapGesture { authVM.userDidInteract() }
                }
            }

            if !network.isOnline && privacyAccepted {
                HStack(spacing: 8) {
                    Image(systemName: "wifi.slash")
                    Text("Offline ? changes will sync when you're back online.")
                        .lineLimit(2)
                }
                .font(AGFont.hud(11)).kerning(1)
                .foregroundColor(AGColor.ink)
                .padding(.horizontal, 14).padding(.vertical, 8)
                .background(Capsule().fill(AGColor.panelHi))
                .overlay(Capsule().stroke(AGColor.amber.opacity(0.4), lineWidth: 1))
                .padding(.bottom, 12)
                .transition(.move(edge: .bottom).combined(with: .opacity))
            }
        }
        .preferredColorScheme(.dark)
        .animation(.spring(response: 0.4, dampingFraction: 0.85), value: network.isOnline)
    }
}

#Preview {
```

```
        ContentView().environmentObject(AuthViewModel())
    }
```

AGSwipeToDelete.swift

AetherGain/DesignSystem/AGSwipeToDelete.swift

```swift


import SwiftUI

struct AGSwipeToDelete<Content: View>: View {
    let onDelete: () -> Void
    var actionWidth: CGFloat = 82
    var cornerRadius: CGFloat = 12
    let content: () -> Content

    @State private var offset: CGFloat = 0
    @State private var dragging: Bool = false

    var body: some View {
        ZStack(alignment: .trailing) {

            Button {
                withAnimation(.spring(response: 0.35, dampingFraction: 0.85)) {
                    offset = 0
                }
                AGHaptics.tap()
                onDelete()
            } label: {
                VStack(spacing: 4) {
                    Image(systemName: "trash.fill")
                        .font(.system(size: 16, weight: .bold))
                    Text("DELETE").kerning(1.2).font(AGFont.hud(9))
                }
                .foregroundColor(.white)
                .frame(width: actionWidth)
                .frame(maxHeight: .infinity)
                .background(AGColor.danger)
                .clipShape(RoundedRectangle(cornerRadius: cornerRadius, style: .continuous))
            }
            .buttonStyle(.plain)
            .opacity(offset < -8 ? 1 : 0)
            .allowsHitTesting(offset < -30)


            content()
                .background(AGColor.canvas)
                .offset(x: offset)
                .gesture(
```

```
                        DragGesture(minimumDistance: 12)
                            .onChanged { g in
                                dragging = true
                                let dx = g.translation.width

                                if dx <= 0 {
                                    offset = max(dx, -actionWidth - 24)
                                } else {

                                    offset = min(0, -actionWidth + dx)
                                }
                            }
                            .onEnded { g in
                                dragging = false
                                withAnimation(.spring(response: 0.35, dampingFraction: 0.85)) {
                                    offset = offset < -actionWidth * 0.5 ? -actionWidth : 0
                                }
                            }
                    )
            }
            .clipShape(RoundedRectangle(cornerRadius: cornerRadius, style: .continuous))


        }
    }
```

AGTheme.swift

AetherGain/DesignSystem/AGTheme.swift

```swift
import SwiftUI

enum AGColor {
    static let canvas   = Color(red: 0.037, green: 0.043, blue: 0.055)
    static let panel    = Color(red: 0.070, green: 0.078, blue: 0.098)
    static let panelHi  = Color(red: 0.100, green: 0.113, blue: 0.140)
    static let stroke   = Color.white.opacity(0.08)
    static let strokeHi = Color(red: 1.00, green: 0.55, blue: 0.18).opacity(0.35)

    static let ink       = Color(red: 0.965, green: 0.933, blue: 0.858)
    static let inkMuted  = Color(red: 0.60, green: 0.60, blue: 0.66)
    static let inkDim    = Color(red: 0.40, green: 0.40, blue: 0.46)

    static let amber      = Color(red: 1.00, green: 0.55, blue: 0.18)
    static let amberHi    = Color(red: 1.00, green: 0.72, blue: 0.42)
    static let amberLo    = Color(red: 0.55, green: 0.28, blue: 0.06)

    static let teal       = Color(red: 0.24, green: 0.86, blue: 0.82)
    static let magenta    = Color(red: 0.94, green: 0.36, blue: 0.85)
    static let sky        = Color(red: 0.42, green: 0.72, blue: 1.00)
    static let sage       = Color(red: 0.62, green: 0.88, blue: 0.66)

    static let success = teal
    static let warn    = Color(red: 1.00, green: 0.82, blue: 0.35)
    static let danger  = Color(red: 1.00, green: 0.32, blue: 0.32)

    static let coral      = amber
    static let peach      = amberHi
    static let mint       = teal
    static let lavender   = magenta
    static let sunshine   = warn
    static let rose       = magenta
    static let card       = panel
```

```
    }



    enum AGFont {
        static func display(_ size: CGFloat = 34) -> Font {
            .system(size: size, weight: .heavy, design: .rounded)
        }
        static func title(_ size: CGFloat = 22) -> Font {
            .system(size: size, weight: .bold, design: .rounded)
        }
        static func headline(_ size: CGFloat = 17) -> Font {
            .system(size: size, weight: .semibold, design: .rounded)
        }
        static func body(_ size: CGFloat = 15) -> Font {
            .system(size: size, weight: .regular, design: .default)
        }
        static func caption(_ size: CGFloat = 12) -> Font {
            .system(size: size, weight: .medium, design: .rounded)
        }
        static func mono(_ size: CGFloat = 14, weight: Font.Weight = .medium) -> Font {
            .system(size: size, weight: weight, design: .monospaced)
        }

        static func number(_ size: CGFloat = 28) -> Font {
            .system(size: size, weight: .heavy, design: .monospaced)
        }

        static func hud(_ size: CGFloat = 10) -> Font {
            .system(size: size, weight: .semibold, design: .monospaced)
        }
    }



    struct AGCardStyle: ViewModifier {
        var background: Color = AGColor.panel
        var corner: CGFloat = 14
        var padded: Bool = true
        var glow: Color? = nil

        func body(content: Content) -> some View {
            content
                .padding(padded ? 16 : 0)
                .background(
                    RoundedRectangle(cornerRadius: corner, style: .continuous)
                        .fill(background)
                )
                .overlay(
                    RoundedRectangle(cornerRadius: corner, style: .continuous)
                        .stroke(AGColor.stroke, lineWidth: 1)
                )
                .overlay(

                    CornerBrackets()
```

```
                        .stroke(AGColor.amber.opacity(0.35), lineWidth: 1)
                        .allowsHitTesting(false)
                )
                .shadow(color: (glow ?? .clear).opacity(0.35), radius: 20, x: 0, y: 0)
        }
    }

    private struct CornerBrackets: Shape {
        func path(in rect: CGRect) -> Path {
            var p = Path()
            let len: CGFloat = 12
            let inset: CGFloat = 4

            p.move(to: CGPoint(x: rect.minX + inset, y: rect.minY + inset + len))
            p.addLine(to: CGPoint(x: rect.minX + inset, y: rect.minY + inset))
            p.addLine(to: CGPoint(x: rect.minX + inset + len, y: rect.minY + inset))

            p.move(to: CGPoint(x: rect.maxX - inset - len, y: rect.minY + inset))
            p.addLine(to: CGPoint(x: rect.maxX - inset, y: rect.minY + inset))
            p.addLine(to: CGPoint(x: rect.maxX - inset, y: rect.minY + inset + len))

            p.move(to: CGPoint(x: rect.maxX - inset, y: rect.maxY - inset - len))
            p.addLine(to: CGPoint(x: rect.maxX - inset, y: rect.maxY - inset))
            p.addLine(to: CGPoint(x: rect.maxX - inset - len, y: rect.maxY - inset))

            p.move(to: CGPoint(x: rect.minX + inset + len, y: rect.maxY - inset))
            p.addLine(to: CGPoint(x: rect.minX + inset, y: rect.maxY - inset))
            p.addLine(to: CGPoint(x: rect.minX + inset, y: rect.maxY - inset - len))
            return p
        }
    }

    extension View {
        func agCard(background: Color = AGColor.panel, corner: CGFloat = 14,
                    padded: Bool = true, glow: Color? = nil) -> some View {
            modifier(AGCardStyle(background: background, corner: corner, padded: padded, glow: glow))
        }
    }



    struct AGPushButton: View {
        let title: String
        var icon: String? = nil
        var fill: Color = AGColor.amber
        var textColor: Color = .black
        var isEnabled: Bool = true
        var isLoading: Bool = false
        let action: () -> Void

        @State private var pressed = false

        var body: some View {
            Button(action: {
                guard isEnabled, !isLoading else { return }
```

```
                AGHaptics.tap()
                action()
            }) {
                ZStack {
                    RoundedRectangle(cornerRadius: 12, style: .continuous)
                        .fill(fill.opacity(isEnabled ? 0.55 : 0.22))
                        .offset(y: 4)
                    RoundedRectangle(cornerRadius: 12, style: .continuous)
                        .fill(fill.opacity(isEnabled ? 1 : 0.4))
                        .overlay(
                            RoundedRectangle(cornerRadius: 12, style: .continuous)
                                .stroke(Color.white.opacity(0.25), lineWidth: 1)
                        )
                        .offset(y: pressed ? 2 : 0)
                        .shadow(color: fill.opacity(isEnabled ? 0.55 : 0), radius: 14, x: 0, y: 0)
                        .overlay(
                            HStack(spacing: 8) {
                                if isLoading {
                                    ProgressView().tint(textColor)
                                } else {
                                    if let icon { Image(systemName: icon) }
                                    Text(title.uppercased())
                                        .kerning(1.4)
                                }
                            }
                            .font(AGFont.headline(14))
                            .foregroundColor(textColor)
                            .offset(y: pressed ? 2 : 0)
                        )
                }
                .frame(height: 50)
            }
            .buttonStyle(.plain)
            .disabled(!isEnabled || isLoading)
            .simultaneousGesture(
                DragGesture(minimumDistance: 0)
                    .onChanged { _ in withAnimation(.easeOut(duration: 0.06)) { pressed = true } }
                    .onEnded { _ in withAnimation(.easeOut(duration: 0.10)) { pressed = false } }
            )
        }
    }

    struct AGGhostButton: View {
        let title: String
        var icon: String? = nil
        var accent: Color = AGColor.teal
        let action: () -> Void

        var body: some View {
            Button(action: { AGHaptics.tap(); action() }) {
                HStack(spacing: 6) {
                    if let icon { Image(systemName: icon) }
                    Text(title.uppercased()).kerning(1.2)
                }
                .font(AGFont.hud(11))
```

```
                .foregroundColor(accent)
                .padding(.horizontal, 14).padding(.vertical, 9)
                .background(
                    RoundedRectangle(cornerRadius: 10, style: .continuous)
                        .fill(accent.opacity(0.10))
                )
                .overlay(
                    RoundedRectangle(cornerRadius: 10, style: .continuous)
                        .stroke(accent.opacity(0.35), lineWidth: 1)
                )
            }
            .buttonStyle(.plain)
        }
    }



    struct AGSectionHeader: View {
        let title: String
        var subtitle: String? = nil
        var trailing: AnyView? = nil

        var body: some View {
            HStack(alignment: .firstTextBaseline) {
                VStack(alignment: .leading, spacing: 2) {
                    HStack(spacing: 6) {
                        Rectangle()
                            .fill(AGColor.amber)
                            .frame(width: 3, height: 12)
                        Text(title.uppercased()).kerning(1.6)
                            .font(AGFont.hud(11))
                            .foregroundColor(AGColor.amber)
                    }
                    if let subtitle {
                        Text(subtitle).font(AGFont.body(12))
                            .foregroundColor(AGColor.inkMuted)
                    }
                }
                Spacer()
                trailing
            }
        }
    }



    struct AGStatPill: View {
        let icon: String
        let title: String
        let value: String
        var tint: Color = AGColor.amber

        var body: some View {
            HStack(spacing: 8) {
                Image(systemName: icon).font(.system(size: 12, weight: .bold))
```

```
                    .foregroundColor(tint)
                VStack(alignment: .leading, spacing: 0) {
                    Text(title.uppercased()).kerning(1.2)
                        .font(AGFont.hud(9)).foregroundColor(AGColor.inkMuted)
                    Text(value).font(AGFont.mono(13, weight: .semibold))
                        .foregroundColor(AGColor.ink)
                }
            }
            .padding(.horizontal, 10).padding(.vertical, 6)
            .background(
                RoundedRectangle(cornerRadius: 8).fill(tint.opacity(0.08))
            )
            .overlay(
                RoundedRectangle(cornerRadius: 8).stroke(tint.opacity(0.35), lineWidth: 1)
            )
        }
    }



    enum AGHaptics {
        static func tap() {
            let g = UIImpactFeedbackGenerator(style: .light)
            g.impactOccurred()
        }
        static func success() {
            let g = UINotificationFeedbackGenerator()
            g.notificationOccurred(.success)
        }
        static func error() {
            let g = UINotificationFeedbackGenerator()
            g.notificationOccurred(.error)
        }
    }



    struct AGProgressRing: View {
        var progress: Double
        var color: Color = AGColor.amber
        var lineWidth: CGFloat = 8
        var size: CGFloat = 84
        var label: String? = nil
        var sublabel: String? = nil

        var body: some View {
            ZStack {
                Circle()
                    .stroke(color.opacity(0.15), lineWidth: lineWidth)
                Circle()
                    .trim(from: 0, to: max(0.001, min(1, progress)))
                    .stroke(color, style: StrokeStyle(lineWidth: lineWidth, lineCap: .round))
                    .rotationEffect(.degrees(-90))
                    .shadow(color: color.opacity(0.5), radius: 6, x: 0, y: 0)
                    .animation(.spring(response: 0.6, dampingFraction: 0.8), value: progress)
```

```
                VStack(spacing: 2) {
                    if let label {
                        Text(label).font(AGFont.number(min(size * 0.24, 22)))
                            .foregroundColor(color)
                            .lineLimit(1)
                            .minimumScaleFactor(0.5)
                    }
                    if let sublabel {
                        Text(sublabel).font(AGFont.hud(9))
                            .foregroundColor(AGColor.inkMuted)
                            .lineLimit(1)
                            .truncationMode(.tail)
                            .multilineTextAlignment(.center)
                            .minimumScaleFactor(0.4)
                            .allowsTightening(true)
                            .padding(.horizontal, 2)
                    }
                }
                .frame(width: size - lineWidth - 4)
            }
            .frame(width: size, height: size)
        }
    }



    struct AGStreakBadge: View {
        let days: Int
        var body: some View {
            HStack(spacing: 6) {
                Image(systemName: "flame.fill").foregroundColor(AGColor.amber)
                Text("\(days)").font(AGFont.mono(14, weight: .heavy))
                    .foregroundColor(AGColor.ink)
                Text("DAY").font(AGFont.hud(9))
                    .foregroundColor(AGColor.inkMuted).kerning(1)
            }
            .padding(.horizontal, 10).padding(.vertical, 5)
            .background(
                RoundedRectangle(cornerRadius: 8).fill(AGColor.amber.opacity(0.12))
            )
            .overlay(
                RoundedRectangle(cornerRadius: 8).stroke(AGColor.amber.opacity(0.4), lineWidth: 1)
            )
        }
    }



    struct AGScreen<Content: View>: View {
        let content: () -> Content
        init(@ViewBuilder content: @escaping () -> Content) { self.content = content }
        var body: some View {
            ZStack {
                AGColor.canvas.ignoresSafeArea()
                AGGrid().opacity(0.4).ignoresSafeArea()
```

```
                content()
            }
        }
    }


    struct AGGrid: View {
        var spacing: CGFloat = 24
        var body: some View {
            GeometryReader { geo in
                Canvas { ctx, size in
                    let cols = Int(size.width / spacing) + 1
                    let rows = Int(size.height / spacing) + 1
                    for r in 0..<rows {
                        for c in 0..<cols {
                            let x = CGFloat(c) * spacing
                            let y = CGFloat(r) * spacing
                            let rect = CGRect(x: x, y: y, width: 1, height: 1)
                            ctx.fill(Path(ellipseIn: rect),
                                     with: .color(Color.white.opacity(0.06)))
                        }
                    }
                }
            }
        }
    }



    struct AGAvatar: View {
        let urlString: String?
        let initial: String
        var size: CGFloat = 44
        var tint: Color = AGColor.amber

        var body: some View {
            ZStack {
                Circle().fill(tint.opacity(0.15))
                    .overlay(Circle().stroke(tint.opacity(0.5), lineWidth: 1))
                if let s = urlString, let url = URL(string: s) {
                    AsyncImage(url: url) { phase in
                        switch phase {
                        case .empty:   ProgressView().tint(tint)
                        case .success(let img):
                            img.resizable().scaledToFill()
                        case .failure: fallback
                        @unknown default: fallback
                        }
                    }
                    .clipShape(Circle())
                } else {
                    fallback
                }
            }
            .frame(width: size, height: size)
```

```
        }

        private var fallback: some View {
            Text(initial).font(AGFont.display(size * 0.42))
                .foregroundColor(tint)
        }
    }



    struct AGHudChip: View {
        let text: String
        var tint: Color = AGColor.teal
        var body: some View {
            Text(text.uppercased()).kerning(1.4)
                .font(AGFont.hud(10))
                .foregroundColor(tint)
                .padding(.horizontal, 8).padding(.vertical, 4)
                .background(
                    RoundedRectangle(cornerRadius: 6).fill(tint.opacity(0.10))
                )
                .overlay(
                    RoundedRectangle(cornerRadius: 6).stroke(tint.opacity(0.35), lineWidth: 1)
                )
        }
    }
```

AGToast.swift

AetherGain/DesignSystem/AGToast.swift

```swift
import SwiftUI
import Combine

enum AGToastKind {
    case success, error, info

    var color: Color {
        switch self {
        case .success: return AGColor.success
        case .error:   return AGColor.danger
        case .info:    return AGColor.lavender
        }
    }
    var icon: String {
        switch self {
        case .success: return "checkmark.circle.fill"
        case .error:   return "exclamationmark.triangle.fill"
        case .info:    return "info.circle.fill"
        }
    }
}

struct AGToast: Equatable {
    var message: String
    var kind: AGToastKind
    var id = UUID()
}

@MainActor
final class AGToastCenter: ObservableObject {
    static let shared = AGToastCenter()
    @Published var current: AGToast?
    @Published var showConfetti: Bool = false

    func show(_ message: String, kind: AGToastKind = .success, celebrate: Bool = false) {
        current = AGToast(message: message, kind: kind)
        switch kind {
        case .success: AGHaptics.success()
        case .error:   AGHaptics.error()
        case .info:    AGHaptics.tap()
        }
        if celebrate {
            showConfetti = true
```

```
                Task { [weak self] in
                    try? await Task.sleep(nanoseconds: 1_600_000_000)
                    self?.showConfetti = false
                }
            }
            Task { [weak self, id = current?.id] in
                try? await Task.sleep(nanoseconds: 2_400_000_000)
                if self?.current?.id == id { self?.current = nil }
            }
        }
    }

    struct AGToastHost: View {
        @ObservedObject var center = AGToastCenter.shared

        var body: some View {
            ZStack(alignment: .top) {
                if let t = center.current {
                    HStack(spacing: 10) {
                        Image(systemName: t.kind.icon)
                            .foregroundColor(t.kind.color)
                        Text(t.message)
                            .font(AGFont.headline(14))
                            .foregroundColor(AGColor.ink)
                            .lineLimit(3)
                        Spacer(minLength: 0)
                    }
                    .padding(.horizontal, 14).padding(.vertical, 12)
                    .background(
                        RoundedRectangle(cornerRadius: 16, style: .continuous)
                            .fill(AGColor.card)
                            .shadow(color: Color.black.opacity(0.10), radius: 20, x: 0, y: 8)
                    )
                    .overlay(
                        RoundedRectangle(cornerRadius: 16, style: .continuous)
                            .stroke(t.kind.color.opacity(0.25), lineWidth: 1)
                    )
                    .padding(.horizontal, 16)
                    .padding(.top, 8)
                    .transition(.move(edge: .top).combined(with: .opacity))
                }
                if center.showConfetti {
                    AGConfettiView()
                        .allowsHitTesting(false)
                        .transition(.opacity)
                }
            }
            .animation(.spring(response: 0.45, dampingFraction: 0.75), value: center.current)
            .animation(.easeOut(duration: 0.3), value: center.showConfetti)
        }
    }



    extension View {
```

```
106
107
108
109         func agToastLayer() -> some View {
110             overlay(alignment: .top) { AGToastHost() }
111         }
112     }
113
114
115
116     struct AGConfettiView: View {
117         private let pieces: [ConfettiPiece] = (0..<40).map { _ in
118             ConfettiPiece(
119                 xStart: CGFloat.random(in: 0.05...0.95),
120                 hue: Double.random(in: 0...1),
121                 delay: Double.random(in: 0...0.6),
122                 duration: Double.random(in: 1.3...2.0),
123                 rotation: Double.random(in: 0...360)
124             )
125         }
126         var body: some View {
127             GeometryReader { geo in
128                 ZStack {
129                     ForEach(pieces) { piece in
130                         ConfettiParticle(piece: piece, size: geo.size)
131                     }
132                 }
133             }
134         }
135     }
136
137     private struct ConfettiPiece: Identifiable {
138         let id = UUID()
139         let xStart: CGFloat
140         let hue: Double
141         let delay: Double
142         let duration: Double
143         let rotation: Double
144     }
145
146     private struct ConfettiParticle: View {
147         let piece: ConfettiPiece
148         let size: CGSize
149         @State private var animate = false
150
151         var body: some View {
152             Rectangle()
153                 .fill(Color(hue: piece.hue, saturation: 0.75, brightness: 0.95))
154                 .frame(width: 9, height: 14)
155                 .rotationEffect(.degrees(animate ? piece.rotation + 360 : piece.rotation))
156                 .position(
157                     x: piece.xStart * size.width,
158                     y: animate ? size.height + 40 : -30
159                 )
160                 .onAppear {
```

```
                    withAnimation(.easeIn(duration: piece.duration).delay(piece.delay)) {
                        animate = true
                    }
                }
            }
        }
```

Exercise.swift

AetherGain/Models/Exercise.swift

```swift


import Foundation
import FirebaseFirestore

enum ExerciseCategory: String, Codable, CaseIterable, Identifiable {
    case strength
    case cardio
    case mobility

    var id: String { rawValue }

    var displayName: String {
        switch self {
        case .strength:
            return "Strength"
        case .cardio:
            return "Cardio"
        case .mobility:
            return "Mobility"
        }
    }
}

struct Exercise: Identifiable, Codable {




    var id: String = UUID().uuidString

    var type: String = ""
    var category: ExerciseCategory? = .strength
    var sets: Int = 3
    var reps: Int = 10
    var weightKG: Double = 0.0
    var distanceKM: Double? = nil
    var durationMinutes: Double? = nil
    var inclinePercent: Double? = nil
    var notes: String = ""

    init() {}

```

```
        init(
            type: String = "",
            category: ExerciseCategory = .strength,
            sets: Int = 3,
            reps: Int = 10,
            weightKG: Double = 0.0,
            distanceKM: Double? = nil,
            durationMinutes: Double? = nil,
            inclinePercent: Double? = nil,
            notes: String = ""
        ) {
            self.type = type
            self.category = category
            self.sets = sets
            self.reps = reps
            self.weightKG = weightKG
            self.distanceKM = distanceKM
            self.durationMinutes = durationMinutes
            self.inclinePercent = inclinePercent
            self.notes = notes
        }
    }

    struct ExerciseTemplate: Identifiable, Hashable {
        let name: String
        let category: ExerciseCategory

        var id: String { name }

        static let catalog: [ExerciseTemplate] = [
            ExerciseTemplate(name: "Bench Press", category: .strength),
            ExerciseTemplate(name: "Squat", category: .strength),
            ExerciseTemplate(name: "Deadlift", category: .strength),
            ExerciseTemplate(name: "Shoulder Press", category: .strength),
            ExerciseTemplate(name: "Pull Up", category: .strength),
            ExerciseTemplate(name: "Push Up", category: .strength),
            ExerciseTemplate(name: "Lat Pulldown", category: .strength),
            ExerciseTemplate(name: "Bicep Curl", category: .strength),
            ExerciseTemplate(name: "Tricep Pushdown", category: .strength),
            ExerciseTemplate(name: "Leg Press", category: .strength),
            ExerciseTemplate(name: "Running", category: .cardio),
            ExerciseTemplate(name: "Cycling", category: .cardio),
            ExerciseTemplate(name: "Swimming", category: .cardio),
            ExerciseTemplate(name: "Rowing", category: .cardio),
            ExerciseTemplate(name: "Treadmill Walk", category: .cardio),
            ExerciseTemplate(name: "Elliptical", category: .cardio),
            ExerciseTemplate(name: "Yoga", category: .mobility),
            ExerciseTemplate(name: "Stretching", category: .mobility),
            ExerciseTemplate(name: "Pilates", category: .mobility)
        ]

        static func template(named name: String) -> ExerciseTemplate? {
            catalog.first { $0.name.caseInsensitiveCompare(name) == .orderedSame }
        }

```

```
106         static func inferredCategory(for name: String) -> ExerciseCategory {
107             if let template = template(named: name) {
108                 return template.category
109             }
110 
111             let lowercasedName = name.lowercased()
112             let cardioKeywords = ["run", "jog", "cycle", "bike", "swim", "row", "walk", "treadmill", "elliptical", "cardio"]
113             let mobilityKeywords = ["yoga", "stretch", "mobility", "pilates", "warm up", "cool down"]
114 
115             if cardioKeywords.contains(where: lowercasedName.contains) {
116                 return .cardio
117             }
118 
119             if mobilityKeywords.contains(where: lowercasedName.contains) {
120                 return .mobility
121             }
122 
123             return .strength
124         }
125 
126         static func configuredExercise(named name: String) -> Exercise {
127             let trimmedName = name.trimmingCharacters(in: .whitespacesAndNewlines)
128             let category = inferredCategory(for: trimmedName)
129 
130             switch category {
131             case .strength:
132                 return Exercise(type: trimmedName, category: category, sets: 3, reps: 10, weightKG: 0)
133             case .cardio:
134                 return Exercise(type: trimmedName, category: category, sets: 1, reps: 1, distanceKM: 1, durationMinutes: 10, inclinePercent: 0)
135             case .mobility:
136                 return Exercise(type: trimmedName, category: category, sets: 1, reps: 1, durationMinutes: 10)
137             }
138         }
139     }
```

GameEngine.swift

AetherGain/Models/GameEngine.swift

```swift
import Foundation

struct Badge: Identifiable, Hashable {
    let id: String
    let title: String
    let requirement: String
}

enum GameEngine {
    static func workoutXP(for session: WorkoutSession) -> Int {
        guard session.isValidForXP else { return 0 }

        return session.exercises.reduce(50) { total, exercise in
            total + exerciseXP(for: exercise)
        }
    }

    static func exerciseXP(for exercise: Exercise) -> Int {
        switch exercise.category ?? .strength {
        case .strength:
            let volume = Double(exercise.sets * exercise.reps) * max(exercise.weightKG, 1)
            return max(5, Int(volume / 20))
        case .cardio:
            let durationScore = Int((exercise.durationMinutes ?? 0) * 2)
            let distanceScore = Int((exercise.distanceKM ?? 0) * 8)
            let inclineScore = Int((exercise.inclinePercent ?? 0) * 2)
            return max(10, durationScore + distanceScore + inclineScore)
        case .mobility:
            return max(5, Int((exercise.durationMinutes ?? 0) * 1.5))
        }
    }

    static func level(for xp: Int) -> Int {
        max(1, (xp / 500) + 1)
    }

    static func xpForNextLevel(from xp: Int) -> Int {
        let nextLevelStart = level(for: xp) * 500
        return max(0, nextLevelStart - xp)
    }

    static func badges(for xp: Int, streakDays: Int) -> [Badge] {
        var earned: [Badge] = []

        if xp >= 500 {
            earned.append(Badge(id: "level-2", title: "Level 2", requirement: "Earn 500 XP"))
        }
        if xp >= 2500 {
            earned.append(Badge(id: "level-6", title: "Consistency Builder", requirement: "Earn 2500 XP"))
        }
```

```
            if streakDays >= 7 {
                earned.append(Badge(id: "streak-7", title: "7 Day Streak", requirement: "Complete habits for 7 days"))
            }
            if streakDays >= 30 {
                earned.append(Badge(id: "streak-30", title: "30 Day Streak", requirement: "Complete habits for 30 days"))
            }

            return earned
        }
    }
```

MealEntry.swift

AetherGain/Models/MealEntry.swift

```swift


import Foundation
import FirebaseFirestore

enum MealType: String, Codable, CaseIterable, Identifiable {
    case breakfast = "Breakfast"
    case lunch     = "Lunch"
    case dinner    = "Dinner"
    case snack     = "Snack"

    var id: String { rawValue }
    var icon: String {
        switch self {
        case .breakfast: return "sun.max.fill"
        case .lunch:     return "fork.knife"
        case .dinner:    return "moon.stars.fill"
        case .snack:     return "sparkles"
        }
    }
    static func defaultFor(hour: Int) -> MealType {
        switch hour {
        case 4..<11:  return .breakfast
        case 11..<15: return .lunch
        case 15..<18: return .snack
        default:      return .dinner
        }
    }
}

struct MealEntry: Identifiable, Codable {
    @DocumentID var id: String? = UUID().uuidString

    var date: Date = Date()
    var mealType: MealType = .snack

    var foodName: String = ""


    var amount: Double = 100
    var unit: String = "g"


    var per100gCalories: Double = 0
    var per100gProtein: Double = 0
    var per100gCarbs: Double = 0
    var per100gFat: Double = 0
```

```
51
52
53          var grams: Double = 100
54
55
56          var calories: Double = 0
57          var protein: Double = 0
58          var carbs: Double = 0
59          var fat: Double = 0
60
61          var notes: String = ""
62          var photoURL: String? = nil
63
64
65
66
67          var macroSource: String = "Manual"
68
69          init() {}
70      }
```

UserProfile.swift

AetherGain/Models/UserProfile.swift

```swift


import Foundation
import FirebaseFirestore

enum Sex: String, Codable, CaseIterable, Identifiable {
    case male   = "Male"
    case female = "Female"
    case other  = "Prefer not to say"
    var id: String { rawValue }
}

enum FitnessGoal: String, Codable, CaseIterable, Identifiable {
    case cut      = "Cut"
    case maintain = "Maintain"
    case bulk     = "Bulk"
    var id: String { rawValue }
    var icon: String {
        switch self {
        case .cut:      return "arrow.down.circle.fill"
        case .maintain: return "equal.circle.fill"
        case .bulk:     return "arrow.up.circle.fill"
        }
    }
}

enum ProteinPreference: String, Codable, CaseIterable, Identifiable {
    case low    = "Low"
    case medium = "Medium"
    case high   = "High"
    var id: String { rawValue }

    var gramsPerKg: Double {
        switch self {
        case .low: return 1.6
        case .medium: return 2.0
        case .high: return 2.5
        }
    }
}

struct UserProfile: Identifiable, Codable {
    @DocumentID var id: String? = UUID().uuidString
```

```
51
52
53          var name: String = ""
54          var age: Int = 18
55          var sex: Sex = .other
56          var goals: String = ""
57          var preferences: String = ""
58          var experienceLevel: String = "Beginner"
59
60
61          var heightCM: Double = 170.0
62          var weightKG: Double = 70.0
63          var bodyFatPercent: Double? = nil
64
65
66          var avgStepsPerDay: Int = 6000
67          var workoutsPerWeek: Int = 3
68
69
70          var fitnessGoal: FitnessGoal = .maintain
71          var goalRateKgPerWeek: Double = 0.5
72          var proteinPreference: ProteinPreference = .medium
73
74
75          var bmr: Int = 0
76          var tdee: Int = 0
77          var calorieTarget: Int = 0
78          var proteinTargetG: Int = 0
79          var carbsTargetG: Int = 0
80          var fatTargetG: Int = 0
81          var hasOnboardedTargets: Bool = false
82
83
84          var dailyWaterML: Double = 0.0
85          var waterMLTarget: Double = 2500
86          var sleepHours: Double = 0.0
87          var sleepHoursTarget: Double = 8
88          var customHabitName: String = "Meditation"
89          var customHabitCompletedToday: Bool = false
90          var streakDays: Int = 0
91          var lastHabitDate: Date? = nil
92
93          var remindersEnabled: Bool = false
94          var reminderHour: Int = 20
95          var reminderMinute: Int = 0
96
97
98          var xp: Int = 0
99          var sharesToLeaderboard: Bool? = false
100
101
102         var profileImageURL: String? = nil
103
104         init() {}
105     }
```

WorkoutSession.swift

AetherGain/Models/WorkoutSession.swift

```swift
import Foundation
import FirebaseFirestore

struct WorkoutSession: Identifiable, Codable {
    @DocumentID var id: String? = UUID().uuidString
    var date: Date = Date()
    var exercises: [Exercise] = []
    var totalXP: Int = 0
    var notes: String = ""


    var isValidForXP: Bool {
        !exercises.isEmpty && exercises.allSatisfy { exercise in
            guard !exercise.type.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty else {
                return false
            }

            switch exercise.category ?? .strength {
            case .strength:
                return exercise.sets > 0 && exercise.reps > 0
            case .cardio:
                return (exercise.durationMinutes ?? 0) > 0 || (exercise.distanceKM ?? 0) > 0
            case .mobility:
                return (exercise.durationMinutes ?? 0) > 0
            }
        }
    }

    init() {}
}
```

FoodClassifier.swift

AetherGain/Services/FoodClassifier.swift

```
import Foundation
import CoreML
import Vision
import UIKit
import ImageIO

struct ClassifiedFood {
    let label: String
    let confidence: Float
}

enum FoodClassifierError: Error, LocalizedError {
    case invalidImage
    case classificationFailed(String)

    var errorDescription: String? {
        switch self {
        case .invalidImage: return "Couldn't read this image."
        case .classificationFailed(let msg): return "Vision failure: \(msg)"
        }
    }
}

enum FoodClassifier {




    private static let bundledModel: VNCoreMLModel? = loadBundledModel()
```

```
    private static func loadBundledModel() -> VNCoreMLModel? {
        let bundle = Bundle.main


        let candidates: [URL?] = [
            bundle.url(forResource: "AetherGainFoodModel", withExtension: "mlmodelc"),
            bundle.url(forResource: "AetherGainFoodModel", withExtension: "mlmodel")
        ]



        let configurations: [MLComputeUnits] = {
            #if targetEnvironment(simulator)
            return [.cpuOnly]
            #else
            return [.all, .cpuOnly]
            #endif
        }()

        for case let url? in candidates {
            for units in configurations {
                do {
                    let config = MLModelConfiguration()
                    config.computeUnits = units
                    let mlModel = try MLModel(contentsOf: url, configuration: config)
                    let vn = try VNCoreMLModel(for: mlModel)
                    print("[FoodClassifier] Bundled model loaded from \(url.lastPathComponent) (computeUnits: \(units))")
                    return vn
                } catch {
                    print("[FoodClassifier] Failed to load bundled model at \(url) with \(units): \(error)")
                }
            }
        }
        print("[FoodClassifier] No bundled FoodClassifier model ? using Vision default classifier.")
        return nil
    }

    static var isBundledModelAvailable: Bool { bundledModel != nil }






    static func classify(image: UIImage,
                         completion: @escaping (Result<[ClassifiedFood], Error>) -> Void) {
        guard let cgImage = image.cgImage else {
            completion(.failure(FoodClassifierError.invalidImage))
            return
        }



        let orientation = cgOrientation(from: image.imageOrientation)

        DispatchQueue.global(qos: .userInitiated).async {
```

```
                if let model = bundledModel {
                    runCoreMLModel(model, on: cgImage, orientation: orientation, completion: completion)
                } else {
                    runVisionDefault(on: cgImage, orientation: orientation, completion: completion)
                }
            }
        }

        private static func cgOrientation(from ui: UIImage.Orientation) -> CGImagePropertyOrientation {
            switch ui {
            case .up:            return .up
            case .down:          return .down
            case .left:          return .left
            case .right:         return .right
            case .upMirrored:    return .upMirrored
            case .downMirrored:  return .downMirrored
            case .leftMirrored:  return .leftMirrored
            case .rightMirrored: return .rightMirrored
            @unknown default:    return .up
            }
        }



        private static func runCoreMLModel(_ model: VNCoreMLModel,
                                           on cgImage: CGImage,
                                           orientation: CGImagePropertyOrientation,
                                           completion: @escaping (Result<[ClassifiedFood], Error>) -> Void) {
            let request = VNCoreMLRequest(model: model) { request, error in
                if let error {



                    print("[FoodClassifier] bundled request failed, using Vision default: \(error)")
                    runVisionDefault(on: cgImage, orientation: orientation, completion: completion)
                    return
                }
                guard let results = request.results as? [VNClassificationObservation] else {
                    completion(.failure(FoodClassifierError.classificationFailed("No observations.")))
                    return
                }
                let top = results
                    .sorted { $0.confidence > $1.confidence }
                    .prefix(15)
                    .map { ClassifiedFood(label: normalize($0.identifier),
                                          confidence: $0.confidence) }
                print("[FoodClassifier] bundled model returned: "
                      + top.prefix(3).map { "\($0.label) \(String(format: "%.2f", $0.confidence))" }
                            .joined(separator: ", "))
                completion(.success(Array(top)))
            }

            request.imageCropAndScaleOption = .centerCrop
            do {
                try VNImageRequestHandler(cgImage: cgImage, orientation: orientation, options: [:]).perform([request])
```

```
            } catch {
                print("[FoodClassifier] perform() threw, using Vision default: \(error)")
                runVisionDefault(on: cgImage, orientation: orientation, completion: completion)
            }
        }

        private static func runVisionDefault(on cgImage: CGImage,
                                             orientation: CGImagePropertyOrientation,
                                             completion: @escaping (Result<[ClassifiedFood], Error>) -> Void) {
            let request = VNClassifyImageRequest()
            do {
                try VNImageRequestHandler(cgImage: cgImage, orientation: orientation, options: [:]).perform([request])
                let all = request.results ?? []



                let strict = all.filter { $0.hasMinimumRecall(0.5, forPrecision: 0.7) }
                let medium = all.filter { $0.hasMinimumRecall(0.5, forPrecision: 0.5) }




                var ordered: [VNClassificationObservation] = []
                var seen = Set<String>()
                for tier in [strict, medium, all] {
                    for obs in tier.sorted(by: { $0.confidence > $1.confidence })
                    where !seen.contains(obs.identifier) {
                        ordered.append(obs)
                        seen.insert(obs.identifier)
                    }
                }

                let top = ordered
                    .prefix(15)
                    .map { ClassifiedFood(label: normalize($0.identifier),
                                          confidence: $0.confidence) }
                print("[FoodClassifier] Vision default: \(all.count) observations, "
                      + "\(strict.count) high precision, \(medium.count) medium. Returning: "
                      + top.map { "\($0.label) \(String(format: "%.2f", $0.confidence))" }
                            .joined(separator: ", "))
                completion(.success(Array(top)))
            } catch {
                print("[FoodClassifier] Vision default ALSO failed: \(error)")
                completion(.failure(error))
            }
        }




        private static func normalize(_ raw: String) -> String {
            raw
                .replacingOccurrences(of: "_", with: " ")
                .replacingOccurrences(of: "-", with: " ")
```

```
                .lowercased()
                .trimmingCharacters(in: .whitespaces)
        }
    }
```

FoodClassifierRegions.swift

AetherGain/Services/FoodClassifierRegions.swift

```swift
import Foundation
import CoreGraphics
import UIKit
import Vision

struct RegionCandidate: Identifiable {
    let id = UUID()
    let label: String
    let confidence: Float

    let boundingBox: CGRect
    var matchedFood: String?
}

extension FoodClassifier {

    static let regionConfidenceFloor: Float = 0.25

    static func classifyRegions(image: UIImage,
                                maxRegions: Int = 4,
                                completion: @escaping (Result<[RegionCandidate], Error>) -> Void) {
```

```
            guard let cgImage = image.cgImage else {
                completion(.failure(FoodClassifierError.invalidImage))
                return
            }

            DispatchQueue.global(qos: .userInitiated).async {
                let regions = salientRegions(in: cgImage, limit: maxRegions)

                guard !regions.isEmpty else {

                    classify(image: image) { result in
                        completion(result.map { classified in
                            classified.prefix(1).map {
                                RegionCandidate(label: $0.label,
                                                confidence: $0.confidence,
                                                boundingBox: CGRect(x: 0, y: 0, width: 1, height: 1),
                                                matchedFood: FoodDatabase.bestMatch(
                                                    fromLabels: [($0.label, $0.confidence)])?.name)
                            }
                        })
                    }
                    return
                }

                var found: [RegionCandidate] = []
                let lock = NSLock()
                let group = DispatchGroup()

                for box in regions {
                    guard let crop = cgImage.cropping(to: pixelRect(for: box, in: cgImage)) else { continue }
                    group.enter()
                    classify(image: UIImage(cgImage: crop)) { result in
                        defer { group.leave() }
                        guard case .success(let classified) = result,
                              let best = classified.first,
                              best.confidence >= regionConfidenceFloor else { return }

                        let matched = FoodDatabase.bestMatch(
                            fromLabels: [(best.label, best.confidence)])?.name
                        let candidate = RegionCandidate(label: best.label,
                                                        confidence: best.confidence,
                                                        boundingBox: topLeftRect(for: box),
                                                        matchedFood: matched)
                        lock.lock()
                        found.append(candidate)
                        lock.unlock()
                    }
                }

                group.notify(queue: .global(qos: .userInitiated)) {


                    var bestByFood: [String: RegionCandidate] = [:]
                    for candidate in found {
                        let key = candidate.matchedFood ?? candidate.label
```

```
                    if let existing = bestByFood[key], existing.confidence >= candidate.confidence {
                        continue
                    }
                    bestByFood[key] = candidate
                }
                completion(.success(bestByFood.values.sorted { $0.confidence > $1.confidence }))
            }
        }
    }




    private static func salientRegions(in cgImage: CGImage, limit: Int) -> [CGRect] {
        let request = VNGenerateObjectnessBasedSaliencyImageRequest()
        do {
            try VNImageRequestHandler(cgImage: cgImage, options: [:]).perform([request])
        } catch {
            return []
        }
        guard let observation = request.results?.first as? VNSaliencyImageObservation,
              let objects = observation.salientObjects else { return [] }

        return objects


            .filter { $0.boundingBox.width * $0.boundingBox.height >= 0.04 }
            .sorted { $0.confidence > $1.confidence }
            .prefix(limit)
            .map { $0.boundingBox }
    }




    private static func pixelRect(for box: CGRect, in cgImage: CGImage) -> CGRect {
        let width = CGFloat(cgImage.width)
        let height = CGFloat(cgImage.height)
        let padX = box.width * 0.08
        let padY = box.height * 0.08

        let x = (box.origin.x - padX) * width

        let y = (1 - box.origin.y - box.height - padY) * height
        let w = (box.width + padX * 2) * width
        let h = (box.height + padY * 2) * height

        return CGRect(x: max(0, x),
                      y: max(0, y),
                      width: min(w, width - max(0, x)),
                      height: min(h, height - max(0, y)))
    }
```

```
        private static func topLeftRect(for box: CGRect) -> CGRect {
            CGRect(x: box.origin.x,
                   y: 1 - box.origin.y - box.height,
                   width: box.width,
                   height: box.height)
        }
    }
```

FoodDatabase.swift

AetherGain/Services/FoodDatabase.swift

```swift


import Foundation

struct FoodMacros {
    let name: String
    let calories: Double
    let protein: Double
    let carbs: Double
    let fat: Double
    let defaultServingG: Double
    let aliases: [String]
}

enum FoodDatabase {

    static let all: [FoodMacros] = [

        FoodMacros(name: "Idli",     calories: 133, protein: 4.0,  carbs: 25.0, fat: 1.5,  defaultServingG: 60,
                   aliases: ["idli", "steamed cake", "rice cake"]),
        FoodMacros(name: "Dosa",     calories: 168, protein: 4.5,  carbs: 26.0, fat: 5.5,  defaultServingG: 80,
                   aliases: ["dosa", "crepe", "pancake"]),
        FoodMacros(name: "Vada",     calories: 335, protein: 12.0, carbs: 30.0, fat: 20.0, defaultServingG: 40,
                   aliases: ["vada", "medu vada", "fritter", "doughnut", "donut", "savory doughnut"]),
        FoodMacros(name: "Chapati",  calories: 297, protein: 9.5,  carbs: 55.0, fat: 4.5,  defaultServingG: 40,
                   aliases: ["chapati", "roti", "flatbread", "tortilla", "naan"]),
        FoodMacros(name: "Rice",     calories: 130, protein: 2.7,  carbs: 28.0, fat: 0.3,  defaultServingG: 150,
                   aliases: ["rice", "white rice", "steamed rice", "brown rice"]),
        FoodMacros(name: "Fried rice", calories: 174, protein: 4.5, carbs: 22.0, fat: 7.5, defaultServingG: 200,
                   aliases: ["fried rice", "pilaf"]),
        FoodMacros(name: "Biryani",  calories: 200, protein: 8.0,  carbs: 27.0, fat: 7.5,  defaultServingG: 250,
                   aliases: ["biryani", "pulao"]),
        FoodMacros(name: "Dal",      calories: 116, protein: 9.0,  carbs: 20.0, fat: 0.4,  defaultServingG: 200,
                   aliases: ["dal", "daal", "lentil", "lentils", "soup"]),
        FoodMacros(name: "Paneer",   calories: 265, protein: 18.0, carbs: 1.2,  fat: 20.8, defaultServingG: 80,
                   aliases: ["paneer", "cottage cheese"]),
        FoodMacros(name: "Samosa",   calories: 262, protein: 5.0,  carbs: 24.0, fat: 17.0, defaultServingG: 60,
                   aliases: ["samosa", "pastry"]),
        FoodMacros(name: "Poha",     calories: 180, protein: 4.0,  carbs: 32.0, fat: 4.0,  defaultServingG: 150,
                   aliases: ["poha", "flattened rice"]),
        FoodMacros(name: "Upma",     calories: 192, protein: 5.0,  carbs: 32.0, fat: 5.0,  defaultServingG: 150,
                   aliases: ["upma"]),
        FoodMacros(name: "Paratha",  calories: 297, protein: 7.5,  carbs: 40.0, fat: 12.0, defaultServingG: 80,
                   aliases: ["paratha", "aloo paratha"]),
```

```
            FoodMacros(name: "Chole",    calories: 164, protein: 8.9,  carbs: 27.0, fat: 2.6,  defaultServingG: 200,
                       aliases: ["chole", "chickpea curry", "chana masala"]),
            FoodMacros(name: "Rajma",    calories: 140, protein: 8.7,  carbs: 22.8, fat: 0.5,  defaultServingG: 200,
                       aliases: ["rajma", "kidney bean curry"]),


            FoodMacros(name: "Pizza",    calories: 266, protein: 11.0, carbs: 33.0, fat: 10.0, defaultServingG: 100,
                       aliases: ["pizza", "margherita"]),
            FoodMacros(name: "Burger",   calories: 295, protein: 17.0, carbs: 30.0, fat: 12.0, defaultServingG: 220,
                       aliases: ["burger", "hamburger", "cheeseburger"]),
            FoodMacros(name: "Sandwich", calories: 250, protein: 12.0, carbs: 33.0, fat: 8.0,  defaultServingG: 180,
                       aliases: ["sandwich", "sub"]),
            FoodMacros(name: "Pasta",    calories: 158, protein: 6.0,  carbs: 31.0, fat: 1.3,  defaultServingG: 200,
                       aliases: ["pasta", "spaghetti", "macaroni", "penne", "lasagna"]),
            FoodMacros(name: "Bread",    calories: 265, protein: 9.0,  carbs: 49.0, fat: 3.0,  defaultServingG: 30,
                       aliases: ["bread", "toast", "bun", "loaf", "white bread", "wheat bread"]),
            FoodMacros(name: "Noodles",  calories: 138, protein: 4.5,  carbs: 25.0, fat: 2.0,  defaultServingG: 200,
                       aliases: ["noodles", "ramen", "udon", "chow mein"]),
            FoodMacros(name: "Salad",    calories: 100, protein: 3.0,  carbs: 8.0,  fat: 5.0,  defaultServingG: 150,
                       aliases: ["salad", "lettuce", "greens", "caesar salad"]),
            FoodMacros(name: "Sushi",    calories: 143, protein: 5.8,  carbs: 30.0, fat: 0.7,  defaultServingG: 200,
                       aliases: ["sushi", "roll", "maki", "nigiri"]),
            FoodMacros(name: "Taco",     calories: 226, protein: 9.0,  carbs: 20.0, fat: 12.0, defaultServingG: 80,
                       aliases: ["taco", "burrito", "quesadilla"]),


            FoodMacros(name: "Chicken",  calories: 165, protein: 31.0, carbs: 0.0,  fat: 3.6,  defaultServingG: 100,
                       aliases: ["chicken", "poultry", "grilled chicken", "chicken breast"]),
            FoodMacros(name: "Fish",     calories: 206, protein: 22.0, carbs: 0.0,  fat: 12.0, defaultServingG: 100,
                       aliases: ["fish", "seafood", "salmon", "tuna", "cod", "tilapia"]),
            FoodMacros(name: "Egg",      calories: 155, protein: 13.0, carbs: 1.1,  fat: 11.0, defaultServingG: 50,
                       aliases: ["egg", "omelette", "omelet", "boiled egg", "scrambled eggs"]),
            FoodMacros(name: "Beef",     calories: 250, protein: 26.0, carbs: 0.0,  fat: 15.0, defaultServingG: 100,
                       aliases: ["beef", "steak", "ground beef"]),
            FoodMacros(name: "Pork",     calories: 242, protein: 27.0, carbs: 0.0,  fat: 14.0, defaultServingG: 100,
                       aliases: ["pork", "bacon", "ham"]),
            FoodMacros(name: "Tofu",     calories: 76,  protein: 8.0,  carbs: 1.9,  fat: 4.8,  defaultServingG: 100,
                       aliases: ["tofu", "bean curd"]),
            FoodMacros(name: "Shrimp",   calories: 99,  protein: 24.0, carbs: 0.2,  fat: 0.3,  defaultServingG: 100,
                       aliases: ["shrimp", "prawn"]),


            FoodMacros(name: "Oats",     calories: 389, protein: 16.9, carbs: 66.3, fat: 6.9,  defaultServingG: 40,
                       aliases: ["oats", "oatmeal", "porridge"]),
            FoodMacros(name: "Quinoa",   calories: 120, protein: 4.4,  carbs: 21.3, fat: 1.9,  defaultServingG: 150,
                       aliases: ["quinoa"]),
            FoodMacros(name: "Potato",   calories: 77,  protein: 2.0,  carbs: 17.0, fat: 0.1,  defaultServingG: 150,
                       aliases: ["potato", "aloo", "mashed potato"]),
            FoodMacros(name: "Sweet potato", calories: 86, protein: 1.6, carbs: 20.1, fat: 0.1, defaultServingG: 150,
                       aliases: ["sweet potato", "yam"]),
            FoodMacros(name: "French fries", calories: 312, protein: 3.4, carbs: 41.0, fat: 15.0, defaultServingG: 100,
                       aliases: ["fries", "french fries", "chips"]),


            FoodMacros(name: "Milk",     calories: 61,  protein: 3.2,  carbs: 4.8,  fat: 3.3,  defaultServingG: 250,
```

```
                aliases: ["milk", "whole milk", "skimmed milk"]),
        FoodMacros(name: "Yogurt",   calories: 61,  protein: 3.5,  carbs: 4.7,  fat: 3.3,  defaultServingG: 150,
                aliases: ["yogurt", "yoghurt", "curd", "greek yogurt"]),
        FoodMacros(name: "Cheese",   calories: 402, protein: 25.0, carbs: 1.3,  fat: 33.0, defaultServingG: 30,
                aliases: ["cheese", "cheddar", "mozzarella", "parmesan"]),
        FoodMacros(name: "Butter",   calories: 717, protein: 0.9,  carbs: 0.1,  fat: 81.0, defaultServingG: 10,
                aliases: ["butter", "ghee"]),
        FoodMacros(name: "Nuts",     calories: 607, protein: 20.0, carbs: 21.0, fat: 54.0, defaultServingG: 30,
                aliases: ["nuts", "almond", "cashew", "peanut", "walnut", "pistachio"]),
        FoodMacros(name: "Peanut butter", calories: 588, protein: 25.0, carbs: 20.0, fat: 50.0, defaultServingG: 32,
                aliases: ["peanut butter", "pb"]),
        FoodMacros(name: "Chocolate", calories: 546, protein: 4.9, carbs: 60.0, fat: 31.0, defaultServingG: 30,
                aliases: ["chocolate", "dark chocolate", "milk chocolate"]),
        FoodMacros(name: "Ice cream", calories: 207, protein: 3.5, carbs: 24.0, fat: 11.0, defaultServingG: 100,
                aliases: ["ice cream", "gelato"]),
        FoodMacros(name: "Cookie",   calories: 502, protein: 5.7, carbs: 64.0,  fat: 25.0, defaultServingG: 25,
                aliases: ["cookie", "biscuit"]),
        FoodMacros(name: "Cake",     calories: 350, protein: 4.9, carbs: 50.0,  fat: 15.0, defaultServingG: 90,
                aliases: ["cake", "cupcake"]),
        FoodMacros(name: "Croissant", calories: 406, protein: 8.2, carbs: 45.8, fat: 21.0, defaultServingG: 60,
                aliases: ["croissant"]),
        FoodMacros(name: "Donut",    calories: 452, protein: 4.9, carbs: 51.0, fat: 25.0,  defaultServingG: 60,
                aliases: ["glazed donut", "krispy kreme"]),


        FoodMacros(name: "Banana",   calories: 89,  protein: 1.1,  carbs: 22.8, fat: 0.3,  defaultServingG: 120,
                aliases: ["banana"]),
        FoodMacros(name: "Apple",    calories: 52,  protein: 0.3,  carbs: 14.0, fat: 0.2,  defaultServingG: 180,
                aliases: ["apple"]),
        FoodMacros(name: "Orange",   calories: 47,  protein: 0.9,  carbs: 12.0, fat: 0.1,  defaultServingG: 150,
                aliases: ["orange", "tangerine"]),
        FoodMacros(name: "Grapes",   calories: 69,  protein: 0.7,  carbs: 18.0, fat: 0.2,  defaultServingG: 100,
                aliases: ["grapes"]),
        FoodMacros(name: "Mango",    calories: 60,  protein: 0.8,  carbs: 15.0, fat: 0.4,  defaultServingG: 150,
                aliases: ["mango"]),
        FoodMacros(name: "Strawberry", calories: 33, protein: 0.7, carbs: 7.7, fat: 0.3,   defaultServingG: 100,
                aliases: ["strawberry", "berries", "berry"]),
        FoodMacros(name: "Watermelon", calories: 30, protein: 0.6, carbs: 7.6, fat: 0.2,   defaultServingG: 150,
                aliases: ["watermelon"]),
        FoodMacros(name: "Avocado",  calories: 160, protein: 2.0,  carbs: 8.5,  fat: 15.0, defaultServingG: 100,
                aliases: ["avocado", "guacamole"]),
        FoodMacros(name: "Broccoli", calories: 34,  protein: 2.8,  carbs: 6.6,  fat: 0.4,  defaultServingG: 100,
                aliases: ["broccoli"]),
        FoodMacros(name: "Spinach",  calories: 23,  protein: 2.9,  carbs: 3.6,  fat: 0.4,  defaultServingG: 80,
                aliases: ["spinach", "palak"]),
        FoodMacros(name: "Carrot",   calories: 41,  protein: 0.9,  carbs: 10.0, fat: 0.2,  defaultServingG: 80,
                aliases: ["carrot"]),
        FoodMacros(name: "Tomato",   calories: 18,  protein: 0.9,  carbs: 3.9,  fat: 0.2,  defaultServingG: 100,
                aliases: ["tomato"]),


        FoodMacros(name: "Coke",     calories: 42,  protein: 0.0,  carbs: 10.6, fat: 0.0,  defaultServingG: 330,
                aliases: ["coke", "cola", "pepsi", "soda", "soft drink"]),
        FoodMacros(name: "Orange juice", calories: 45, protein: 0.7, carbs: 10.4, fat: 0.2, defaultServingG: 250,
                aliases: ["orange juice", "oj"]),
```

```
            FoodMacros(name: "Coffee",   calories: 2,   protein: 0.3,  carbs: 0.0,  fat: 0.0,  defaultServingG: 240,
                       aliases: ["coffee", "black coffee", "espresso"]),
            FoodMacros(name: "Latte",    calories: 42,  protein: 2.2,  carbs: 3.4,  fat: 2.3,  defaultServingG: 350,
                       aliases: ["latte", "cappuccino", "flat white"]),
            FoodMacros(name: "Beer",     calories: 43,  protein: 0.5,  carbs: 3.6,  fat: 0.0,  defaultServingG: 350,
                       aliases: ["beer", "lager"]),
            FoodMacros(name: "Wine",     calories: 83,  protein: 0.1,  carbs: 2.7,  fat: 0.0,  defaultServingG: 150,
                       aliases: ["wine", "red wine", "white wine"]),
            FoodMacros(name: "Protein shake", calories: 100, protein: 20.0, carbs: 5.0, fat: 1.0, defaultServingG: 300,
                       aliases: ["protein shake", "whey", "protein powder"]),
            FoodMacros(name: "Tea",      calories: 1,   protein: 0.0,  carbs: 0.3,  fat: 0.0,  defaultServingG: 240,
                       aliases: ["tea", "chai", "green tea", "black tea"]),
            FoodMacros(name: "Smoothie", calories: 60,  protein: 1.0,  carbs: 14.0, fat: 0.3,  defaultServingG: 300,
                       aliases: ["smoothie", "fruit smoothie"]),


            FoodMacros(name: "Turkey",   calories: 189, protein: 29.0, carbs: 0.0,  fat: 7.4,  defaultServingG: 100,
                       aliases: ["turkey", "turkey breast"]),
            FoodMacros(name: "Lamb",     calories: 294, protein: 25.0, carbs: 0.0,  fat: 21.0, defaultServingG: 100,
                       aliases: ["lamb", "mutton"]),
            FoodMacros(name: "Duck",     calories: 337, protein: 19.0, carbs: 0.0,  fat: 28.0, defaultServingG: 100,
                       aliases: ["duck"]),
            FoodMacros(name: "Sardines", calories: 208, protein: 25.0, carbs: 0.0,  fat: 11.0, defaultServingG: 60,
                       aliases: ["sardines", "sardine"]),
            FoodMacros(name: "Crab",     calories: 87,  protein: 18.0, carbs: 0.0,  fat: 1.1,  defaultServingG: 100,
                       aliases: ["crab", "crab meat"]),
            FoodMacros(name: "Lobster",  calories: 89,  protein: 19.0, carbs: 0.0,  fat: 0.9,  defaultServingG: 100,
                       aliases: ["lobster"]),
            FoodMacros(name: "Tempeh",   calories: 192, protein: 20.3, carbs: 7.6,  fat: 10.8, defaultServingG: 100,
                       aliases: ["tempeh"]),
            FoodMacros(name: "Seitan",   calories: 143, protein: 25.0, carbs: 14.0, fat: 2.0,  defaultServingG: 100,
                       aliases: ["seitan", "wheat meat"]),
            FoodMacros(name: "Chickpeas", calories: 164, protein: 8.9, carbs: 27.0, fat: 2.6,  defaultServingG: 100,
                       aliases: ["chickpeas", "garbanzo"]),
            FoodMacros(name: "Black beans", calories: 132, protein: 8.9, carbs: 24.0, fat: 0.5, defaultServingG: 100,
                       aliases: ["black beans", "beans"]),
            FoodMacros(name: "Cottage cheese", calories: 98, protein: 11.0, carbs: 3.4, fat: 4.3, defaultServingG: 100,
                       aliases: ["cottage cheese"]),


            FoodMacros(name: "Bagel",    calories: 257, protein: 10.0, carbs: 51.0, fat: 1.5,  defaultServingG: 100,
                       aliases: ["bagel"]),
            FoodMacros(name: "Muffin",   calories: 424, protein: 5.0,  carbs: 55.0, fat: 20.0, defaultServingG: 80,
                       aliases: ["muffin", "blueberry muffin"]),
            FoodMacros(name: "Pancake",  calories: 227, protein: 6.4,  carbs: 28.0, fat: 9.7,  defaultServingG: 80,
                       aliases: ["pancakes", "hotcake"]),
            FoodMacros(name: "Waffle",   calories: 291, protein: 8.0,  carbs: 33.0, fat: 14.1, defaultServingG: 75,
                       aliases: ["waffle", "waffles"]),
            FoodMacros(name: "Cereal",   calories: 379, protein: 7.5,  carbs: 84.0, fat: 0.9,  defaultServingG: 40,
                       aliases: ["cereal", "corn flakes"]),
            FoodMacros(name: "Granola",  calories: 471, protein: 10.0, carbs: 64.0, fat: 20.0, defaultServingG: 40,
                       aliases: ["granola", "muesli"]),
            FoodMacros(name: "Couscous", calories: 112, protein: 3.8,  carbs: 23.0, fat: 0.2,  defaultServingG: 150,
                       aliases: ["couscous"]),
            FoodMacros(name: "Barley",   calories: 123, protein: 2.3,  carbs: 28.0, fat: 0.4,  defaultServingG: 150,
```

```
                aliases: ["barley", "pearl barley"]),
            FoodMacros(name: "Corn",     calories: 86,  protein: 3.3,  carbs: 19.0, fat: 1.4,  defaultServingG: 100,
                aliases: ["corn", "sweetcorn", "maize"]),
            FoodMacros(name: "Popcorn",  calories: 387, protein: 13.0, carbs: 78.0, fat: 4.5,  defaultServingG: 30,
                aliases: ["popcorn"]),


            FoodMacros(name: "Pineapple", calories: 50, protein: 0.5, carbs: 13.0, fat: 0.1,  defaultServingG: 150,
                aliases: ["pineapple"]),
            FoodMacros(name: "Papaya",   calories: 43,  protein: 0.5,  carbs: 11.0, fat: 0.3,  defaultServingG: 150,
                aliases: ["papaya"]),
            FoodMacros(name: "Pear",     calories: 57,  protein: 0.4,  carbs: 15.0, fat: 0.1,  defaultServingG: 150,
                aliases: ["pear"]),
            FoodMacros(name: "Peach",    calories: 39,  protein: 0.9,  carbs: 9.5,  fat: 0.3,  defaultServingG: 150,
                aliases: ["peach"]),
            FoodMacros(name: "Blueberry", calories: 57, protein: 0.7, carbs: 14.0, fat: 0.3,  defaultServingG: 100,
                aliases: ["blueberry", "blueberries"]),
            FoodMacros(name: "Raspberry", calories: 52, protein: 1.2, carbs: 12.0, fat: 0.7,  defaultServingG: 100,
                aliases: ["raspberry", "raspberries"]),
            FoodMacros(name: "Kiwi",     calories: 61,  protein: 1.1,  carbs: 15.0, fat: 0.5,  defaultServingG: 100,
                aliases: ["kiwi", "kiwifruit"]),
            FoodMacros(name: "Cherry",   calories: 50,  protein: 1.0,  carbs: 12.0, fat: 0.3,  defaultServingG: 100,
                aliases: ["cherry", "cherries"]),
            FoodMacros(name: "Pomegranate", calories: 83, protein: 1.7, carbs: 19.0, fat: 1.2, defaultServingG: 100,
                aliases: ["pomegranate", "anaar"]),
            FoodMacros(name: "Coconut",  calories: 354, protein: 3.3,  carbs: 15.0, fat: 33.0, defaultServingG: 30,
                aliases: ["coconut"]),
            FoodMacros(name: "Dates",    calories: 277, protein: 1.8,  carbs: 75.0, fat: 0.2,  defaultServingG: 30,
                aliases: ["dates", "date"]),
            FoodMacros(name: "Raisins",  calories: 299, protein: 3.1,  carbs: 79.0, fat: 0.5,  defaultServingG: 30,
                aliases: ["raisins"]),


            FoodMacros(name: "Cucumber", calories: 16,  protein: 0.7,  carbs: 3.6,  fat: 0.1,  defaultServingG: 100,
                aliases: ["cucumber"]),
            FoodMacros(name: "Onion",    calories: 40,  protein: 1.1,  carbs: 9.3,  fat: 0.1,  defaultServingG: 80,
                aliases: ["onion"]),
            FoodMacros(name: "Bell pepper", calories: 31, protein: 1.0, carbs: 6.0, fat: 0.3,  defaultServingG: 100,
                aliases: ["bell pepper", "capsicum"]),
            FoodMacros(name: "Cauliflower", calories: 25, protein: 1.9, carbs: 5.0, fat: 0.3,  defaultServingG: 100,
                aliases: ["cauliflower", "gobi"]),
            FoodMacros(name: "Zucchini", calories: 17,  protein: 1.2,  carbs: 3.1,  fat: 0.3,  defaultServingG: 100,
                aliases: ["zucchini", "courgette"]),
            FoodMacros(name: "Eggplant", calories: 25,  protein: 1.0,  carbs: 5.9,  fat: 0.2,  defaultServingG: 100,
                aliases: ["eggplant", "brinjal", "aubergine"]),
            FoodMacros(name: "Mushroom", calories: 22,  protein: 3.1,  carbs: 3.3,  fat: 0.3,  defaultServingG: 80,
                aliases: ["mushroom", "button mushroom"]),
            FoodMacros(name: "Cabbage",  calories: 25,  protein: 1.3,  carbs: 5.8,  fat: 0.1,  defaultServingG: 100,
                aliases: ["cabbage"]),
            FoodMacros(name: "Peas",     calories: 81,  protein: 5.4,  carbs: 14.0, fat: 0.4,  defaultServingG: 80,
                aliases: ["peas", "green peas"]),
            FoodMacros(name: "Beetroot", calories: 43,  protein: 1.6,  carbs: 9.6,  fat: 0.2,  defaultServingG: 100,
                aliases: ["beetroot", "beet"]),

```

```
            FoodMacros(name: "Ketchup",  calories: 112, protein: 1.7,  carbs: 26.0, fat: 0.4,  defaultServingG: 17,
                       aliases: ["ketchup", "tomato sauce"]),
            FoodMacros(name: "Mayonnaise", calories: 680, protein: 1.0, carbs: 0.6, fat: 75.0, defaultServingG: 15,
                       aliases: ["mayonnaise", "mayo"]),
            FoodMacros(name: "Soy sauce", calories: 53, protein: 8.0, carbs: 4.9, fat: 0.6,   defaultServingG: 15,
                       aliases: ["soy sauce", "soya sauce"]),
            FoodMacros(name: "Hummus",   calories: 166, protein: 8.0,  carbs: 14.0, fat: 10.0, defaultServingG: 30,
                       aliases: ["hummus"]),
            FoodMacros(name: "Olive oil", calories: 884, protein: 0.0, carbs: 0.0, fat: 100.0, defaultServingG: 14,
                       aliases: ["olive oil"]),
            FoodMacros(name: "Honey",    calories: 304, protein: 0.3,  carbs: 82.0, fat: 0.0,  defaultServingG: 21,
                       aliases: ["honey"]),
            FoodMacros(name: "Sugar",    calories: 387, protein: 0.0,  carbs: 100.0, fat: 0.0, defaultServingG: 4,
                       aliases: ["sugar", "white sugar"]),
            FoodMacros(name: "Jam",      calories: 250, protein: 0.5,  carbs: 66.0, fat: 0.1,  defaultServingG: 20,
                       aliases: ["jam", "jelly", "preserve"]),


            FoodMacros(name: "Naan",     calories: 310, protein: 9.0,  carbs: 55.0, fat: 6.0,  defaultServingG: 90,
                       aliases: ["naan"]),
            FoodMacros(name: "Pav",      calories: 274, protein: 8.5,  carbs: 51.0, fat: 3.6,  defaultServingG: 40,
                       aliases: ["pav", "pao"]),
            FoodMacros(name: "Pakora",   calories: 315, protein: 6.0,  carbs: 30.0, fat: 20.0, defaultServingG: 60,
                       aliases: ["pakora", "bhajiya", "bhaji"]),
            FoodMacros(name: "Butter chicken", calories: 155, protein: 12.0, carbs: 4.0, fat: 10.0, defaultServingG: 200,
                       aliases: ["butter chicken", "murgh makhani"]),
            FoodMacros(name: "Tikka masala", calories: 148, protein: 10.0, carbs: 6.0, fat: 9.0, defaultServingG: 200,
                       aliases: ["tikka masala", "chicken tikka"]),
            FoodMacros(name: "Palak paneer", calories: 180, protein: 8.0, carbs: 6.0, fat: 14.0, defaultServingG: 200,
                       aliases: ["palak paneer", "saag paneer"]),
            FoodMacros(name: "Gulab jamun", calories: 322, protein: 4.6, carbs: 50.0, fat: 12.0, defaultServingG: 40,
                       aliases: ["gulab jamun"]),
            FoodMacros(name: "Jalebi",   calories: 356, protein: 3.5,  carbs: 55.0, fat: 15.0, defaultServingG: 50,
                       aliases: ["jalebi"]),
            FoodMacros(name: "Kheer",    calories: 143, protein: 4.5,  carbs: 22.0, fat: 4.5,  defaultServingG: 150,
                       aliases: ["kheer", "rice pudding", "payasam"]),
            FoodMacros(name: "Lassi",    calories: 91,  protein: 3.4,  carbs: 12.0, fat: 3.0,  defaultServingG: 250,
                       aliases: ["lassi"]),


            FoodMacros(name: "Chips",    calories: 536, protein: 7.0,  carbs: 50.0, fat: 34.0, defaultServingG: 30,
                       aliases: ["chips", "crisps", "potato chips"]),
            FoodMacros(name: "Pretzels", calories: 380, protein: 10.0, carbs: 80.0, fat: 3.0,  defaultServingG: 30,
                       aliases: ["pretzels", "pretzel"]),
            FoodMacros(name: "Pop tart", calories: 400, protein: 4.0,  carbs: 74.0, fat: 10.0, defaultServingG: 52,
                       aliases: ["pop tart", "poptart"]),
            FoodMacros(name: "Brownie",  calories: 466, protein: 6.0,  carbs: 57.0, fat: 24.0, defaultServingG: 60,
                       aliases: ["brownie", "chocolate brownie"]),
            FoodMacros(name: "Cheesecake", calories: 321, protein: 5.5, carbs: 26.0, fat: 22.0, defaultServingG: 100,
                       aliases: ["cheesecake"]),
            FoodMacros(name: "Muesli bar", calories: 380, protein: 6.0, carbs: 65.0, fat: 12.0, defaultServingG: 30,
                       aliases: ["muesli bar", "granola bar", "protein bar"]),
            FoodMacros(name: "Milkshake", calories: 112, protein: 3.9, carbs: 18.0, fat: 3.0,  defaultServingG: 300,
                       aliases: ["milkshake", "shake"]),
            FoodMacros(name: "Gum",      calories: 360, protein: 0.0,  carbs: 90.0, fat: 0.0,  defaultServingG: 3,
```

```
                    aliases: ["gum", "chewing gum"]),


            FoodMacros(name: "Energy drink", calories: 45, protein: 0.0, carbs: 11.0, fat: 0.0, defaultServingG: 250,
                    aliases: ["energy drink", "red bull", "monster"]),
            FoodMacros(name: "Sports drink", calories: 24, protein: 0.0, carbs: 6.0, fat: 0.0, defaultServingG: 250,
                    aliases: ["sports drink", "gatorade", "powerade"]),
            FoodMacros(name: "Kombucha", calories: 30,  protein: 0.0,  carbs: 7.0,  fat: 0.0,  defaultServingG: 250,
                    aliases: ["kombucha"]),
            FoodMacros(name: "Almond milk", calories: 17, protein: 0.6, carbs: 0.3, fat: 1.5,  defaultServingG: 240,
                    aliases: ["almond milk"]),
            FoodMacros(name: "Oat milk", calories: 47,  protein: 1.0,  carbs: 7.0,  fat: 1.5,  defaultServingG: 240,
                    aliases: ["oat milk"]),
        ]

        static func byName(_ query: String) -> FoodMacros? {
            let q = query.lowercased().trimmingCharacters(in: .whitespaces)
            if q.isEmpty { return nil }
            if let exact = all.first(where: { $0.name.lowercased() == q }) { return exact }
            if let alias = all.first(where: { $0.aliases.contains(where: { $0 == q }) }) { return alias }
            if let sub = all.first(where: {
                $0.name.lowercased().contains(q) || q.contains($0.name.lowercased()) ||
                $0.aliases.contains(where: { q.contains($0) || $0.contains(q) })
            }) { return sub }
            return nil
        }



        private static func containsPhrase(_ haystack: String, _ needle: String) -> Bool {
            guard !needle.isEmpty else { return false }
            return (" " + haystack + " ").contains(" " + needle + " ")
        }




        static let minimumMatchScore: Float = 0.08

        static func bestMatch(fromLabels labels: [(label: String, confidence: Float)]) -> FoodMacros? {
            var scores: [String: Float] = [:]

            for (rawLabel, conf) in labels {
                let lower = rawLabel
                    .lowercased()
                    .replacingOccurrences(of: "_", with: " ")
                    .replacingOccurrences(of: "-", with: " ")
                    .trimmingCharacters(in: .whitespaces)
                if lower.isEmpty { continue }

                let tokens = Set(lower.split(separator: " ").map(String.init))

                for food in all {
                    let foodNameLower = food.name.lowercased()

```

```
381
382                     if lower == foodNameLower || food.aliases.contains(lower) {
383                         scores[food.name, default: 0] += conf * 3
384                         continue
385                     }
386
387
388                     if containsPhrase(lower, foodNameLower) {
389                         scores[food.name, default: 0] += conf * 2
390                         continue
391                     }
392
393
394
395                     var aliasHit: Float = 0
396                     for alias in food.aliases {
397                         if containsPhrase(lower, alias) {
398                             aliasHit = max(aliasHit, conf)
399                         } else if tokens.contains(alias) {
400                             aliasHit = max(aliasHit, conf * 0.8)
401                         }
402                     }
403                     if aliasHit > 0 { scores[food.name, default: 0] += aliasHit }
404                 }
405             }
406
407             guard let best = scores.max(by: { $0.value < $1.value }),
408                   best.value >= minimumMatchScore else { return nil }
409             return all.first { $0.name == best.key }
410         }
411     }
```

NetworkMonitor.swift

AetherGain/Services/NetworkMonitor.swift

```swift
import Foundation
import Combine
import Network
import FirebaseFirestore
import SwiftUI
import UIKit

@MainActor
final class NetworkMonitor: ObservableObject {
    static let shared = NetworkMonitor()

    @Published private(set) var isOnline: Bool = true
    private let monitor = NWPathMonitor()
    private let queue = DispatchQueue(label: "aethergain.network.monitor")
    private var started = false
    private var pollTimer: Timer?

    func start() {
        guard !started else { return }
        started = true
        monitor.pathUpdateHandler = { [weak self] path in
            let online = path.status == .satisfied
            Task { @MainActor in
                self?.apply(online: online)
            }
        }
        monitor.start(queue: queue)



        pollTimer = Timer.scheduledTimer(withTimeInterval: 3.0, repeats: true) { [weak self] _ in
            Task { @MainActor in self?.reconcile() }
        }
        NotificationCenter.default.addObserver(
            forName: UIApplication.didBecomeActiveNotification,
            object: nil, queue: .main) { [weak self] _ in
                Task { @MainActor in self?.reconcile() }
            }
    }
```

```
        private func reconcile() {
            let online = monitor.currentPath.status == .satisfied
            guard online != isOnline else { return }
            print("[NetworkMonitor] reconcile corrected a missed transition")
            apply(online: online)
        }

        private func apply(online: Bool) {
            let wasOnline = isOnline
            isOnline = online
            print("[NetworkMonitor] path update: \(wasOnline ? "online" : "offline") -> \(online ? "online" : "offline")")

            if !wasOnline && online {

                Firestore.firestore().enableNetwork { _ in }
                AGToastCenter.shared.show("Back online ? syncing your data.", kind: .success)
            } else if wasOnline && !online {
                AGToastCenter.shared.show("You're offline. Changes will sync when reconnected.", kind: .info)
            }
        }
    }
```

ProfileImageService.swift

AetherGain/Services/ProfileImageService.swift

```swift
import Foundation
import UIKit
import FirebaseAuth
import FirebaseStorage

enum ProfileImageService {


    static func upload(image: UIImage) async throws -> URL {
        guard let uid = Auth.auth().currentUser?.uid else {
            throw NSError(domain: "ProfileImage", code: 401,
                          userInfo: [NSLocalizedDescriptionKey: "Not signed in."])
        }
        let resized = image.resized(maxDimension: 600)
        guard let data = resized.jpegData(compressionQuality: 0.75) else {
            throw NSError(domain: "ProfileImage", code: 500,
                          userInfo: [NSLocalizedDescriptionKey: "Couldn't encode image."])
        }
        let ref = Storage.storage().reference().child("profile_images/\(uid).jpg")
        let metadata = StorageMetadata()
        metadata.contentType = "image/jpeg"
        _ = try await ref.putDataAsync(data, metadata: metadata)
        let url = try await ref.downloadURL()
        return url
    }
}

private extension UIImage {
    func resized(maxDimension: CGFloat) -> UIImage {
        let long = max(size.width, size.height)
        guard long > maxDimension else { return self }
        let scale = maxDimension / long
        let newSize = CGSize(width: size.width * scale, height: size.height * scale)
        let renderer = UIGraphicsImageRenderer(size: newSize)
        return renderer.image { _ in
            self.draw(in: CGRect(origin: .zero, size: newSize))
        }
    }
}
```

ProgressAnalytics.swift

AetherGain/Services/ProgressAnalytics.swift

```swift
import Foundation

struct DailyStat: Identifiable {
    let id = UUID()
    let day: Date
    let dayLabel: String
    let xp: Int
    let workouts: Int
    let calories: Double
    let protein: Double
    let carbs: Double
    let fat: Double
    let waterML: Double
    let sleepHours: Double
    let habitDone: Bool
}

enum ProgressAnalytics {

    static func weeklyStats(workouts: [WorkoutSession],
                            meals: [MealEntry],
                            profile: UserProfile,
                            reference: Date = Date()) -> [DailyStat] {
        let cal = Calendar.current
        let today = cal.startOfDay(for: reference)
        let days: [Date] = (0..<7).reversed().map { offset in
            cal.date(byAdding: .day, value: -offset, to: today) ?? today
        }

        let df = DateFormatter()
        df.dateFormat = "EEE"

        return days.map { day in
            let dayEnd = cal.date(byAdding: .day, value: 1, to: day) ?? day

            let dayWorkouts = workouts.filter { $0.date >= day && $0.date < dayEnd }
            let dayXP = dayWorkouts.reduce(0) { $0 + $1.totalXP }

            let dayMeals = meals.filter { $0.date >= day && $0.date < dayEnd }
            let cals = dayMeals.reduce(0) { $0 + $1.calories }
            let pro  = dayMeals.reduce(0) { $0 + $1.protein }
            let carb = dayMeals.reduce(0) { $0 + $1.carbs }
```

```
                let fat  = dayMeals.reduce(0) { $0 + $1.fat }

                let isToday = cal.isDate(day, inSameDayAs: today)
                let waterML = isToday ? profile.dailyWaterML : 0
                let sleep   = isToday ? profile.sleepHours   : 0
                let habit   = isToday ? profile.customHabitCompletedToday : false

                return DailyStat(day: day,
                                 dayLabel: df.string(from: day),
                                 xp: dayXP,
                                 workouts: dayWorkouts.count,
                                 calories: cals,
                                 protein: pro,
                                 carbs: carb,
                                 fat: fat,
                                 waterML: waterML,
                                 sleepHours: sleep,
                                 habitDone: habit)
            }
        }



        struct HeatmapCell: Identifiable {
            var id: Date { day }
            let day: Date
            let intensity: Double
        }

        static func heatmap(workouts: [WorkoutSession],
                            weeks: Int = 12,
                            reference: Date = Date()) -> [HeatmapCell] {
            let cal = Calendar.current
            let today = cal.startOfDay(for: reference)
            let totalDays = weeks * 7
            let start = cal.date(byAdding: .day, value: -(totalDays - 1), to: today) ?? today


            var xpByDay: [Date: Int] = [:]
            for w in workouts {
                let d = cal.startOfDay(for: w.date)
                xpByDay[d, default: 0] += w.totalXP
            }

            let maxXP = max(1, xpByDay.values.max() ?? 1)

            var cells: [HeatmapCell] = []
            for offset in 0..<totalDays {
                let day = cal.date(byAdding: .day, value: offset, to: start) ?? start
                let xp = xpByDay[cal.startOfDay(for: day)] ?? 0
                let intensity = min(1, Double(xp) / Double(maxXP))
                cells.append(HeatmapCell(day: day, intensity: intensity))
            }
            return cells
        }
```

```
106
107
108
109         static func csv(from stats: [DailyStat]) -> String {
110             var lines = ["date,day,xp,workouts,calories,protein_g,carbs_g,fat_g,water_ml,sleep_hours,habit_done"]
111             let df = DateFormatter()
112             df.dateFormat = "yyyy-MM-dd"
113             for s in stats {
114                 let row = [
115                     df.string(from: s.day),
116                     s.dayLabel,
117                     String(s.xp),
118                     String(s.workouts),
119                     String(format: "%.1f", s.calories),
120                     String(format: "%.1f", s.protein),
121                     String(format: "%.1f", s.carbs),
122                     String(format: "%.1f", s.fat),
123                     String(format: "%.0f", s.waterML),
124                     String(format: "%.1f", s.sleepHours),
125                     s.habitDone ? "1" : "0"
126                 ].joined(separator: ",")
127                 lines.append(row)
128             }
129             return lines.joined(separator: "\n")
130         }
131
132
133
134
135         static func writeCSVToTemp(_ csv: String, name: String = "aethergain_weekly.csv") -> URL? {
136             let fm = FileManager.default
137             let base: URL
138             if let docs = fm.urls(for: .documentDirectory, in: .userDomainMask).first {
139                 base = docs
140             } else {
141                 base = fm.temporaryDirectory
142             }
143
144             let timestamp = ISO8601DateFormatter().string(from: Date())
145                 .replacingOccurrences(of: ":", with: "-")
146             let final = "aethergain_weekly_\(timestamp).csv"
147             let url = base.appendingPathComponent(final)
148             do {
149                 try csv.write(to: url, atomically: true, encoding: .utf8)
150                 return url
151             } catch {
152                 print("CSV write failed: \(error)")
153                 return nil
154             }
155         }
156     }
```

TargetsCalculator.swift

AetherGain/Services/TargetsCalculator.swift

```swift
import Foundation

struct DailyTargets: Equatable {
    var bmr: Int
    var activityMultiplier: Double
    var tdee: Int
    var calories: Int
    var proteinG: Int
    var carbsG: Int
    var fatG: Int


    var summary: String {
        "BMR \(bmr) kcal · TDEE \(tdee) kcal · target \(calories) kcal"
    }
}

enum TargetsCalculator {



    static func compute(for profile: UserProfile) -> DailyTargets {
        let bmr = basalMetabolicRate(profile)
        let multiplier = activityMultiplier(profile)
        let tdee = bmr * multiplier

        let goalDeltaKcal = goalDailyKcalDelta(profile)
        let calories = max(1200, tdee + goalDeltaKcal)


        let protein = profile.weightKG * profile.proteinPreference.gramsPerKg
        let fat     = (calories * 0.25) / 9.0
        var carbs   = (calories - protein * 4 - fat * 9) / 4.0
        if carbs < 0 { carbs = 0 }

        return DailyTargets(
            bmr: Int(bmr.rounded()),
            activityMultiplier: multiplier,
```

```
                tdee: Int(tdee.rounded()),
                calories: Int(calories.rounded()),
                proteinG: Int(protein.rounded()),
                carbsG: Int(carbs.rounded()),
                fatG: Int(fat.rounded())
            )
        }


        static func apply(_ targets: DailyTargets, to profile: inout UserProfile) {
            profile.bmr = targets.bmr
            profile.tdee = targets.tdee
            profile.calorieTarget = targets.calories
            profile.proteinTargetG = targets.proteinG
            profile.carbsTargetG = targets.carbsG
            profile.fatTargetG = targets.fatG
            profile.hasOnboardedTargets = true
        }




        static func basalMetabolicRate(_ p: UserProfile) -> Double {

            if let bfPercent = p.bodyFatPercent, bfPercent > 0, bfPercent < 60 {
                let lbmKg = p.weightKG * (1 - bfPercent / 100.0)
                return 370 + 21.6 * lbmKg
            }


            let base = 10 * p.weightKG + 6.25 * p.heightCM - 5 * Double(p.age)
            switch p.sex {
            case .male:   return base + 5
            case .female: return base - 161
            case .other:  return base - 78
            }
        }




        static func activityMultiplier(_ p: UserProfile) -> Double {

            let workoutBase: Double = {
                switch p.workoutsPerWeek {
                case ..<1: return 1.20
                case 1...2: return 1.375
                case 3...4: return 1.55
                case 5...6: return 1.725
                default:    return 1.90
                }
            }()


            let steps = Double(p.avgStepsPerDay)
```

```
            let stepBonus: Double = {
                switch steps {
                case ..<3000:   return 0.0
                case 3000..<6000:  return 0.05
                case 6000..<9000:  return 0.10
                case 9000..<12000: return 0.15
                default:            return 0.20
                }
            }()

            return min(2.05, workoutBase + stepBonus)
        }



        static func goalDailyKcalDelta(_ p: UserProfile) -> Double {
            let daily = (p.goalRateKgPerWeek * 7700.0) / 7.0
            switch p.fitnessGoal {
            case .maintain: return 0
            case .cut:      return -daily
            case .bulk:     return +daily
            }
        }
    }
```

AuthViewModel.swift

AetherGain/ViewModels/AuthViewModel.swift

```swift


import Foundation

import FirebaseAuth
import Combine
import SwiftUI



class AuthViewModel: ObservableObject {
    @Published var isSignedIn = false
    @Published var showTimeoutAlert = false

    private var inactivityTimer: Timer?
    private let inactivityTimeout: TimeInterval = 30*60


    init() {
        isSignedIn = Auth.auth().currentUser != nil

        Auth.auth().addStateDidChangeListener { [weak self] auth, user in
            guard let self = self else { return }

            let wasSignedIn = self.isSignedIn
            self.isSignedIn = user != nil

            if self.isSignedIn {
                self.startInactivityTimer()
            }

            else if wasSignedIn {
                self.stopInactivityTimer()
            }
        }
    }



    func signUp(email: String, password: String, completion: @escaping (Error?) -> Void) {
        Auth.auth().createUser(withEmail: email, password: password) { _, error in
            if let error = error {
                completion(error)
```

```
                    return
                }
                self.isSignedIn = true
                self.startInactivityTimer()
                completion(nil)
            }
        }


        func signIn(email: String, password: String, completion: @escaping (Error?) -> Void) {
            Auth.auth().signIn(withEmail: email, password: password) { _, error in
                if let error = error {
                    completion(error)
                    return
                }
                self.isSignedIn = true
                self.startInactivityTimer()
                completion(nil)
            }
        }


        func sendPasswordReset(email: String, completion: @escaping (Error?) -> Void) {
            let trimmed = email.trimmingCharacters(in: .whitespaces)
            guard !trimmed.isEmpty else {
                let err = NSError(domain: "AGAuth", code: 400,
                                  userInfo: [NSLocalizedDescriptionKey: "Enter your email first."])
                completion(err); return
            }
            Auth.auth().sendPasswordReset(withEmail: trimmed) { error in
                completion(error)
            }
        }


        func signOut() {
            do {
                try Auth.auth().signOut()
            } catch {
                print("Sign out error: \(error.localizedDescription)")
            }
            isSignedIn = false
            stopInactivityTimer()
        }




        private func startInactivityTimer() {
            stopInactivityTimer()

            inactivityTimer = Timer.scheduledTimer(withTimeInterval: inactivityTimeout, repeats: false) { [weak self] _ in
                guard let self else { return }

```

```
                self.signOut()
                DispatchQueue.main.async { self.showTimeoutAlert = true }
            }
        }


        private func stopInactivityTimer() {
            inactivityTimer?.invalidate()
            inactivityTimer = nil
        }


        func userDidInteract() {
            if isSignedIn {
                startInactivityTimer()
            }
        }
    }
```

HabitsViewModel.swift

AetherGain/ViewModels/HabitsViewModel.swift

```swift
import Foundation
import FirebaseFirestore
import UserNotifications
import Combine
import FirebaseAuth


class HabitsViewModel: ObservableObject {
    @Published var profile: UserProfile = UserProfile()
    @Published var isLoading = false
    @Published var errorMessage: String?

    private var db = Firestore.firestore()
    private var listener: ListenerRegistration?

    private var userID: String? { Auth.auth().currentUser?.uid }

    init() {
        loadProfile()
    }

    func loadProfile() {
        guard let uid = userID else { return }
        let ref = db.collection("users").document(uid)

        listener = ref.addSnapshotListener { [weak self] snapshot, error in
            if error != nil {
                self?.errorMessage = "Couldn't load your habits. Check your connection and reopen this screen."
                return
            }
            if let loaded = try? snapshot?.data(as: UserProfile.self) {
                self?.profile = loaded
            }
        }
    }

    func saveHabits() {
        guard let uid = userID else {
            DispatchQueue.main.async {
                self.errorMessage = "You're signed out. Sign in again to continue."
                self.isLoading = false
            }
            return
```

```
            }


            applyStreakChange()

            isLoading = true

            let ref = db.collection("users").document(uid)
            do {
                try ref.setData(from: profile, merge: true) { [weak self] error in
                    DispatchQueue.main.async {
                        guard let self = self else { return }
                        if error != nil {
                            self.errorMessage = "Couldn't save your habits. Check your connection and tap Save again."
                        } else {
                            self.errorMessage = nil
                        }
                        self.isLoading = false
                    }
                }
            } catch {
                DispatchQueue.main.async {
                    self.errorMessage = "Your habits couldn't be prepared for saving. Check the custom habit name."
                    self.isLoading = false
                }
            }
        }




        private func applyStreakChange() {
            let allDoneToday = waterHitToday && sleepHitToday && profile.customHabitCompletedToday
            guard allDoneToday else { return }

            let calendar = Calendar.current
            let today = calendar.startOfDay(for: Date())

            if let last = profile.lastHabitDate {
                let lastDay = calendar.startOfDay(for: last)
                let daysSince = calendar.dateComponents([.day], from: lastDay, to: today).day ?? 0
                if daysSince == 0 { return }
                if daysSince == 1 {
                    profile.streakDays += 1
                } else {
                    profile.streakDays = 1
                }
            } else {
                profile.streakDays = 1
            }
            profile.lastHabitDate = today

            let bonusXP = 20 + (profile.streakDays * 5)
```

```
            profile.xp += bonusXP
        }

        private var waterHitToday: Bool { profile.dailyWaterML >= profile.waterMLTarget * 0.8 }
        private var sleepHitToday: Bool { profile.sleepHours >= profile.sleepHoursTarget * 0.85 }


        func toggleReminders(enabled: Bool) {
            profile.remindersEnabled = enabled
            saveHabits()

            if enabled {
                requestNotificationPermission()
                scheduleDailyReminder()
            } else {
                UNUserNotificationCenter.current().removeAllPendingNotificationRequests()
            }
        }

        private func requestNotificationPermission() {
            UNUserNotificationCenter.current().requestAuthorization(options: [.alert, .sound, .badge]) { granted, error in
                if let error {
                    print("Notification permission error: \(error)")
                }
            }
        }

        func scheduleDailyReminder() {
            UNUserNotificationCenter.current().removeAllPendingNotificationRequests()

            let content = UNMutableNotificationContent()
            content.title = "Daily habit check-in"
            content.body  = "Log \(Int(profile.waterMLTarget)) ml water, \(Int(profile.sleepHoursTarget))h sleep, and \(profile.customHabitName)."
            content.sound = .default

            var dateComponents = DateComponents()
            dateComponents.hour   = profile.reminderHour
            dateComponents.minute = profile.reminderMinute

            let trigger = UNCalendarNotificationTrigger(dateMatching: dateComponents, repeats: true)
            let request = UNNotificationRequest(identifier: "dailyHabitReminder", content: content, trigger: trigger)
            UNUserNotificationCenter.current().add(request) { error in
                if let error { print("Notification scheduling error: \(error)") }
            }
        }



    }
```

LeaderboardViewModel.swift

AetherGain/ViewModels/LeaderboardViewModel.swift

```
import Foundation
import FirebaseFirestore
import FirebaseAuth
import Combine

struct LeaderboardUser: Identifiable, Codable {
    @DocumentID var id: String?
    var name: String = ""
    var xp: Int = 0
    var streakDays: Int? = nil
    var sharesToLeaderboard: Bool? = false
    var profileImageURL: String? = nil

    var level: Int { GameEngine.level(for: xp) }
    var badges: [Badge] {
        GameEngine.badges(for: xp, streakDays: streakDays ?? 0)
    }
}

class LeaderboardViewModel: ObservableObject {

    @Published var users: [LeaderboardUser] = []
    @Published var errorMessage: String?
    @Published var rulesHint: String?

    private let db = Firestore.firestore()
    private var listener: ListenerRegistration?









    private static let missingIndexCode = 9

    func loadLeaderboard() {
        attach(orderedByServer: true)
    }

    private func attach(orderedByServer: Bool) {
        listener?.remove()

        var query: Query = db.collection("users")
            .whereField("sharesToLeaderboard", isEqualTo: true)
        if orderedByServer {
            query = query.order(by: "xp", descending: true).limit(to: 25)
```

```
            } else {
                
                
                
                query = query.limit(to: 50)
            }

            listener = query.addSnapshotListener { [weak self] snapshot, error in
                guard let self else { return }

                if let error {
                    

                    let ns = error as NSError
                    print("[Leaderboard] ERROR domain=\(ns.domain) code=\(ns.code): \(ns.localizedDescription)")

                    if orderedByServer && ns.code == Self.missingIndexCode {
                        print("[Leaderboard] no composite index ? re-attaching without server-side ordering")
                        DispatchQueue.main.async { self.attach(orderedByServer: false) }
                        return
                    }

                    DispatchQueue.main.async {
                        self.users = []
                        self.errorMessage = "The leaderboard couldn't load. Check your connection and open it again."
                        self.rulesHint = nil
                    }
                    return
                }

                guard let snapshot else { return }
                let documents = snapshot.documents



                print("[Leaderboard] snapshot fromCache=\(snapshot.metadata.isFromCache) "
                      + "ordered=\(orderedByServer) docs=\(documents.count)")

                var decoded: [LeaderboardUser] = documents.compactMap {
                    do {
                        return try $0.data(as: LeaderboardUser.self)
                    } catch {
                        
                        

                        print("[Leaderboard]   decode failed for \($0.documentID): \(error)")
                        return nil
                    }
                }

                if !orderedByServer {
                    decoded.sort { $0.xp > $1.xp }
                    decoded = Array(decoded.prefix(25))
                }

```

```
                let myUID = Auth.auth().currentUser?.uid
                let currentUserIsOptedIn = decoded.contains(where: { $0.id == myUID })

                DispatchQueue.main.async {
                    self.users = decoded
                    self.errorMessage = nil




                    if currentUserIsOptedIn && decoded.count <= 1 {
                        self.rulesHint = "Only see yourself? Either nobody else has opted in, or your Firestore rules block reading other users' documents."
                    } else {
                        self.rulesHint = nil
                    }
                }
            }
        }

        deinit { listener?.remove() }
    }
```

MealViewModel.swift

AetherGain/ViewModels/MealViewModel.swift

```
import Foundation
import FirebaseAuth
import FirebaseFirestore
import Combine
import Vision
import UIKit

struct FoodCandidate: Identifiable, Hashable {
    let id = UUID()
    let rawLabel: String
    let confidence: Float
    let matched: String?
}

class MealViewModel: ObservableObject {
    @Published var meals: [MealEntry] = []
    @Published var currentMeal: MealEntry = MealEntry()
    @Published var isLoading = false
    @Published var isAnalyzingPhoto = false
    @Published var errorMessage: String?
    @Published var candidates: [FoodCandidate] = []


    @Published var regionCandidates: [RegionCandidate] = []
    @Published var referenceHint: String? = nil

```

```
51
52          @Published var editingMealID: String? = nil
53
54          private var db = Firestore.firestore()
55          private var listener: ListenerRegistration?
56          private var authStateHandle: AuthStateDidChangeListenerHandle?
57          private var userID: String? { Auth.auth().currentUser?.uid }
58
59          init() {
60              resetCurrentMeal()
61
62
63
64              authStateHandle = Auth.auth().addStateDidChangeListener { [weak self] _, user in
65                  guard let self else { return }
66                  if user?.uid != nil {
67                      self.loadMeals()
68                  } else {
69                      self.meals = []
70                      self.listener?.remove()
71                      self.listener = nil
72                  }
73              }
74
75              loadMeals()
76          }
77
78          deinit {
79              listener?.remove()
80              if let handle = authStateHandle {
81                  Auth.auth().removeStateDidChangeListener(handle)
82              }
83          }
84
85
86          func editMeal(_ meal: MealEntry) {
87              currentMeal = meal
88              editingMealID = meal.id
89              candidates = []
90              referenceHint = nil
91              errorMessage = nil
92          }
93
94
95
96          func loadMeals() {
97              guard let uid = userID else { return }
98              listener?.remove()
99              listener = db.collection("users").document(uid).collection("meals")
100                 .order(by: "date", descending: true)
101                 .addSnapshotListener { [weak self] snapshot, error in
102                     guard let self else { return }
103                     if error != nil {
104                         DispatchQueue.main.async { self.errorMessage = "Couldn't load your meals. Check your connection and reopen this screen." }
105                         return
```

```
                }
                let meals = snapshot?.documents.compactMap { try? $0.data(as: MealEntry.self) } ?? []
                DispatchQueue.main.async { self.meals = meals }
            }
    }

    func saveMeal() {
        guard let uid = userID else {
            DispatchQueue.main.async { self.errorMessage = "You're signed out. Sign in again to continue." }
            return
        }
        guard !currentMeal.foodName.trimmingCharacters(in: .whitespaces).isEmpty else {
            DispatchQueue.main.async { self.errorMessage = "Enter a food name or use a photo." }
            return
        }
        if currentMeal.calories == 0 { recalculateFromCurrent() }
        guard currentMeal.calories > 0 else {
            DispatchQueue.main.async {
                self.errorMessage = "No macros calculated yet. Tap Calculate."
            }
            return
        }

        DispatchQueue.main.async { self.isLoading = true }

        let collection = db.collection("users").document(uid).collection("meals")
        let ref: DocumentReference
        if let existingID = editingMealID {
            ref = collection.document(existingID)
        } else {
            ref = collection.document()
        }
        do {
            try ref.setData(from: currentMeal, merge: true) { [weak self] error in
                guard let self else { return }
                DispatchQueue.main.async {
                    if error != nil {
                        self.errorMessage = "Couldn't save this meal. Check your connection and tap Log again."
                    } else {
                        self.errorMessage = nil
                        self.resetCurrentMeal()
                    }
                    self.isLoading = false
                }
            }
        } catch {
            DispatchQueue.main.async {
                self.errorMessage = "This meal couldn't be prepared for saving. Check the food name and amount."
                self.isLoading = false
            }
        }
    }

    func deleteMeal(_ meal: MealEntry) {
        guard let uid = userID, let mealID = meal.id else { return }
```

```
            db.collection("users").document(uid).collection("meals").document(mealID).delete()
        }

        func resetCurrentMeal() {
            var fresh = MealEntry()
            fresh.mealType = MealType.defaultFor(hour: Calendar.current.component(.hour, from: Date()))
            fresh.unit = "g"
            fresh.amount = 100
            currentMeal = fresh
            candidates = []
            referenceHint = nil
            editingMealID = nil
        }









        func analyzeMealPhoto(_ image: UIImage) {
            guard image.cgImage != nil else {
                errorMessage = "Couldn't read this image (format not supported)."
                return
            }
            DispatchQueue.main.async {
                self.isAnalyzingPhoto = true
                self.errorMessage = nil
                self.candidates = []
            }



            FoodClassifier.classify(image: image) { [weak self] result in
                guard let self else { return }
                switch result {
                case .failure(_):
                    DispatchQueue.main.async {
                        self.isAnalyzingPhoto = false
                        self.errorMessage = "That photo couldn't be analysed. Take another one in better light, or enter the food by hand."
                    }
                case .success(let classified):
                    let labelsAndConfs = classified.map { ($0.label, $0.confidence) }
                    let candidates: [FoodCandidate] = classified.map { c in
                        let matched = FoodDatabase.bestMatch(fromLabels: [(c.label, c.confidence)])?.name
                        return FoodCandidate(rawLabel: c.label,
                                             confidence: c.confidence,
                                             matched: matched)
                    }
                    let bestFood = FoodDatabase.bestMatch(fromLabels: labelsAndConfs)

                    DispatchQueue.main.async {
```

```
                        self.candidates = candidates
                        self.isAnalyzingPhoto = false
                        let usingBundled = FoodClassifier.isBundledModelAvailable
                        if let food = bestFood {
                            self.applyFood(food)
                        } else if let top = classified.first {

                            self.currentMeal.foodName = top.label.capitalized
                            let src = usingBundled ? "FoodClassifier" : "Vision default"
                            self.errorMessage = "\(src) returned \(classified.count) labels ? none matched the local DB. Tap a chip, or edit the name and
        Calculate (OpenFoodFacts will try to resolve it)."
                        } else {
                            self.errorMessage = "The classifier returned no labels. Enter a food name manually."
                        }
                    }
                }
            }
        }


        func clearPhotoAnalysis() {
            candidates = []
            referenceHint = nil
            errorMessage = nil
        }

        func selectCandidate(_ candidate: FoodCandidate) {
            if let name = candidate.matched, let food = FoodDatabase.byName(name) {
                applyFood(food)
            } else {
                currentMeal.foodName = candidate.rawLabel.capitalized
                calculateNutritionForCurrentMeal()
            }
        }



        func calculateNutritionForCurrentMeal() {
            let name = currentMeal.foodName.trimmingCharacters(in: .whitespaces)
            guard !name.isEmpty else {
                errorMessage = "Enter a food name first."
                return
            }



            if name.contains(",") {
                calculateBowlMacros(from: name)
                return
            }
            if let match = FoodDatabase.byName(name) {
                applyFood(match)
                return
            }
            fetchFromOpenFoodFacts(query: name)
```

```
        }




        private struct BowlIngredient {
            var amount: Double
            var unit: String
            var name: String
        }




        private func calculateBowlMacros(from input: String) {
            let tokens = input
                .split(separator: ",")
                .map { $0.trimmingCharacters(in: .whitespaces) }
                .filter { !$0.isEmpty }

            var totalCal = 0.0, totalPro = 0.0, totalCarb = 0.0, totalFat = 0.0
            var totalGrams = 0.0
            var summaries: [String] = []
            var unknown: [String] = []

            for token in tokens {
                let ing = parseIngredient(token)
                guard let food = FoodDatabase.byName(ing.name) else {
                    unknown.append(ing.name)
                    continue
                }
                let grams = gramsFor(amount: ing.amount, unit: ing.unit, food: food)
                let factor = grams / 100.0
                let cal = food.calories * factor
                let pro = food.protein * factor
                let carb = food.carbs * factor
                let fat = food.fat * factor
                totalCal += cal
                totalPro += pro
                totalCarb += carb
                totalFat += fat
                totalGrams += grams
                summaries.append(String(format: "%@ (%.0f g ? %.0f kcal)", food.name, grams, cal))
            }

            guard !summaries.isEmpty else {
                errorMessage = "Couldn't recognise any items in \"\(input)\". Try simpler names (e.g. \"100g oats, 250ml milk\")."
                return
            }




            let per100Factor = max(1, totalGrams) / 100.0
            currentMeal.per100gCalories = totalCal / per100Factor
```

```
            currentMeal.per100gProtein  = totalPro / per100Factor
            currentMeal.per100gCarbs    = totalCarb / per100Factor
            currentMeal.per100gFat      = totalFat / per100Factor
            currentMeal.grams = totalGrams
            currentMeal.calories = totalCal.rounded()
            currentMeal.protein  = (totalPro * 10).rounded() / 10
            currentMeal.carbs    = (totalCarb * 10).rounded() / 10
            currentMeal.fat      = (totalFat * 10).rounded() / 10
            currentMeal.amount = totalGrams
            currentMeal.unit = "g"
            currentMeal.macroSource = "Local DB"

            var hint = "SRC: BOWL · " + summaries.joined(separator: " + ")
            if !unknown.isEmpty {
                hint += " · SKIPPED: " + unknown.joined(separator: ", ")
            }
            referenceHint = hint
            errorMessage = unknown.isEmpty ? nil :
                "Ingredients not recognised: \(unknown.joined(separator: ", ")). Rename or omit them."
        }


        private func parseIngredient(_ raw: String) -> BowlIngredient {


            let text = raw.lowercased().trimmingCharacters(in: .whitespaces)


            if let match = text.range(of: #"^(\d+(?:\.\d+)?)([a-z]+)\s+(.+)$"#,
                                      options: .regularExpression) {
                let full = String(text[match])


                var num = ""
                var unit = ""
                var idx = full.startIndex
                while idx < full.endIndex, full[idx].isNumber || full[idx] == "." {
                    num.append(full[idx]); idx = full.index(after: idx)
                }
                while idx < full.endIndex, full[idx].isLetter {
                    unit.append(full[idx]); idx = full.index(after: idx)
                }
                let rest = full[idx...].trimmingCharacters(in: .whitespaces)
                let amount = Double(num) ?? 1
                return BowlIngredient(amount: amount, unit: unit, name: rest)
            }


            if let match = text.range(of: #"^(\d+(?:\.\d+)?)\s+(\S+)\s+(.+)$"#,
                                      options: .regularExpression) {
                let parts = text[match].split(separator: " ", maxSplits: 2,
                                              omittingEmptySubsequences: true)
                if parts.count == 3 {
                    return BowlIngredient(
                        amount: Double(parts[0]) ?? 1,
```

```
                        unit: String(parts[1]),
                        name: String(parts[2])
                    )
                }
            }

            if let match = text.range(of: #"^(\d+(?:\.\d+)?)\s+(.+)$"#,
                                      options: .regularExpression) {
                let parts = text[match].split(separator: " ", maxSplits: 1,
                                              omittingEmptySubsequences: true)
                if parts.count == 2 {
                    return BowlIngredient(
                        amount: Double(parts[0]) ?? 1,
                        unit: "piece",
                        name: String(parts[1])
                    )
                }
            }

            return BowlIngredient(amount: 1, unit: "serving", name: text)
        }



        private func gramsFor(amount: Double, unit: String, food: FoodMacros) -> Double {
            let u = unit.lowercased()
            if ["g", "gm", "gram", "grams"].contains(u) { return amount }
            if u == "kg" { return amount * 1000 }
            if ["ml", "milliliter", "milliliters"].contains(u) { return amount }
            if u == "l" { return amount * 1000 }
            if u == "cup" || u == "cups" { return amount * 240 }
            if u == "tbsp" || u == "tablespoon" { return amount * 15 }
            if u == "tsp" || u == "teaspoon" { return amount * 5 }
            if u == "oz" || u == "ounce" { return amount * 28.35 }
            if u == "small"  { return amount * food.defaultServingG * 0.75 }
            if u == "large"  { return amount * food.defaultServingG * 1.25 }
            if u == "medium" { return amount * food.defaultServingG }

            return amount * food.defaultServingG
        }



        func applyFood(_ food: FoodMacros) {
            currentMeal.foodName = food.name
            currentMeal.per100gCalories = food.calories
            currentMeal.per100gProtein  = food.protein
            currentMeal.per100gCarbs    = food.carbs
            currentMeal.per100gFat      = food.fat
            currentMeal.macroSource = "Local DB"
            if currentMeal.unit == "g" && currentMeal.amount == 100 {
                currentMeal.amount = food.defaultServingG
```

```
            }
            recalculateFromCurrent()
            referenceHint = String(
                format: "SRC: LOCAL DB · 100 g of %@ ? %.0f kcal · P %.1fg · C %.1fg · F %.1fg",
                food.name, food.calories, food.protein, food.carbs, food.fat
            )
            errorMessage = nil
        }


        func recalculateFromCurrent() {
            let grams = gramsForCurrentAmount()
            currentMeal.grams = grams
            let factor = grams / 100.0
            currentMeal.calories = (currentMeal.per100gCalories * factor).rounded()
            currentMeal.protein  = (currentMeal.per100gProtein  * factor * 10).rounded() / 10
            currentMeal.carbs    = (currentMeal.per100gCarbs    * factor * 10).rounded() / 10
            currentMeal.fat      = (currentMeal.per100gFat      * factor * 10).rounded() / 10
        }

        private func gramsForCurrentAmount() -> Double {
            let unit = currentMeal.unit.lowercased()
            let amount = currentMeal.amount

            if ["g", "gm", "gram", "grams"].contains(unit) { return amount }
            if unit == "kg" { return amount * 1000 }
            if ["ml", "milliliter", "milliliters"].contains(unit) { return amount }
            if unit == "l" { return amount * 1000 }
            if unit == "cup" || unit == "cups" { return amount * 240 }
            if unit == "tbsp" || unit == "tablespoon" { return amount * 15 }
            if unit == "tsp" || unit == "teaspoon" { return amount * 5 }
            if unit == "oz" || unit == "ounce" { return amount * 28.35 }

            let food = FoodDatabase.byName(currentMeal.foodName)
            let baseServing = food?.defaultServingG ?? 100
            if unit == "small"  { return amount * baseServing * 0.75 }
            if unit == "large"  { return amount * baseServing * 1.25 }
            if unit == "medium" { return amount * baseServing }
            if unit == "piece" || unit == "pieces" || unit == "serving" || unit == "servings" || unit == "item" {
                return amount * baseServing
            }
            return amount * baseServing
        }



        private func fetchFromOpenFoodFacts(query: String) {
            DispatchQueue.main.async { self.isLoading = true }
            let encoded = query.addingPercentEncoding(withAllowedCharacters: .urlQueryAllowed) ?? ""
            guard let url = URL(string:
                "https://world.openfoodfacts.org/cgi/search.pl?search_terms=\(encoded)&search_simple=1&action=process&json=1")
            else { return }

            URLSession.shared.dataTask(with: url) { [weak self] data, _, error in
                guard let self else { return }
```

```
                DispatchQueue.main.async { self.isLoading = false }
                if error != nil {
                    DispatchQueue.main.async {
                        self.errorMessage = "Couldn't reach the food database. Check your connection, or enter the macros by hand."
                    }
                    return
                }
                guard let data,
                      let json = (try? JSONSerialization.jsonObject(with: data)) as? [String: Any],
                      let products = json["products"] as? [[String: Any]] else {
                    DispatchQueue.main.async {
                        self.errorMessage = "No nutrition match found for \"\(query)\"."
                    }
                    return
                }



                var cals = 0.0, pro = 0.0, carbs = 0.0, fat = 0.0
                var matchedProduct: String? = nil
                for p in products.prefix(15) {
                    guard let n = p["nutriments"] as? [String: Any] else { continue }
                    let c = Self.doubleValue(from: n["energy-kcal_100g"])
                    if c > 0 {
                        cals = c
                        pro = Self.doubleValue(from: n["proteins_100g"])
                        carbs = Self.doubleValue(from: n["carbohydrates_100g"])
                        fat = Self.doubleValue(from: n["fat_100g"])
                        matchedProduct = (p["product_name"] as? String)
                        break
                    }
                }

                guard cals > 0 else {
                    DispatchQueue.main.async {
                        self.errorMessage = "No reliable macros found for \"\(query)\". Try a simpler name or enter manually."
                    }
                    return
                }

                DispatchQueue.main.async {
                    self.currentMeal.per100gCalories = cals
                    self.currentMeal.per100gProtein  = pro
                    self.currentMeal.per100gCarbs    = carbs
                    self.currentMeal.per100gFat      = fat
                    self.currentMeal.macroSource = "OpenFoodFacts"
                    self.recalculateFromCurrent()
                    let displayName = matchedProduct ?? self.currentMeal.foodName
                    self.referenceHint = String(
                        format: "SRC: OPENFOODFACTS · %@ · 100 g ? %.0f kcal · P %.1fg · C %.1fg · F %.1fg",
                        displayName, cals, pro, carbs, fat
                    )
                    self.errorMessage = nil
                }
```

```
            }.resume()
        }

        private static func doubleValue(from any: Any?) -> Double {
            if let d = any as? Double { return d }
            if let i = any as? Int { return Double(i) }
            if let s = any as? String { return Double(s) ?? 0 }
            return 0
        }
    }
```

MealViewModelRegions.swift

AetherGain/ViewModels/MealViewModelRegions.swift

```swift
import Foundation
import UIKit

extension MealViewModel {



    func analyzeMealPhotoByRegion(_ image: UIImage) {
        DispatchQueue.main.async {
            self.isAnalyzingPhoto = true
            self.errorMessage = nil
            self.regionCandidates = []
        }

        FoodClassifier.classifyRegions(image: image) { [weak self] result in
            guard let self else { return }
            DispatchQueue.main.async {
                self.isAnalyzingPhoto = false
                switch result {
                case .failure(_):
                    self.errorMessage = "That photo couldn't be analysed. Take another one in better light, or enter the food by hand."
                case .success(let regions):
                    let recognised = regions.filter { $0.matchedFood != nil }
                    self.regionCandidates = regions
                    if recognised.isEmpty {
                        self.errorMessage = "No food on this plate matched the local database. "
                            + "Enter the items by name, or use bowl mode for a mixed dish."
                    } else if recognised.count == 1 {

                        if let name = recognised[0].matchedFood,
                           let food = FoodDatabase.byName(name) {
                            self.applyFood(food)
                        }
                    }

                }
            }
```

```
            }
        }


        func selectRegionCandidate(_ candidate: RegionCandidate) {
            if let name = candidate.matchedFood, let food = FoodDatabase.byName(name) {
                applyFood(food)
            } else {
                currentMeal.foodName = candidate.label.capitalized
                calculateNutritionForCurrentMeal()
            }
        }
    }
```

ProfileViewModel.swift

AetherGain/ViewModels/ProfileViewModel.swift

```swift
import Foundation
import FirebaseAuth
import FirebaseFirestore
import Combine



class ProfileViewModel: ObservableObject {

    @Published var profile: UserProfile = UserProfile()


    @Published var isLoading = false
    @Published var errorMessage: String?

    private var db = Firestore.firestore()
    private var listener: ListenerRegistration?
    private var authStateHandle: AuthStateDidChangeListenerHandle?


    private var userID: String? {
        Auth.auth().currentUser?.uid
    }

    init() {
        authStateHandle = Auth.auth().addStateDidChangeListener { [weak self] _, user in

            if let uid = user?.uid {
                self?.loadProfile(for: uid)
            } else {

                self?.profile = UserProfile()
                self?.removeListener()
            }
        }
    }


```

```
        func loadProfile(for uid: String) {
            isLoading = true

            let docRef = db.collection("users").document(uid)

            listener = docRef.addSnapshotListener { [weak self] snapshot, error in
                guard let self = self else { return }

                if error != nil {
                    self.errorMessage = "Couldn't load your profile. Check your connection and reopen this screen."
                    self.isLoading = false
                    return
                }



                guard let snapshot else { self.isLoading = false; return }

                if snapshot.exists {
                    do {
                        self.profile = try snapshot.data(as: UserProfile.self)
                    } catch {
                        self.errorMessage = "Your profile could not be read. Nothing was changed."
                        print("[Profile] decode failed, document left untouched: \(error)")
                    }
                } else {
                    self.profile = UserProfile()
                    self.saveProfile()
                }

                self.isLoading = false
            }
        }


        func saveProfile() {
            guard let uid = userID else {
                errorMessage = "You're signed out. Sign in again to continue."
                return
            }

            isLoading = true
            stopSavingIndicatorIfServerIsSlow()

            let docRef = db.collection("users").document(uid)

            do {


                try docRef.setData(from: profile, merge: true) { [weak self] error in
                    DispatchQueue.main.async {
                        if error != nil {
                            self?.errorMessage = "Couldn't save your profile. Check your connection and tap Save again."
                        } else {
                            self?.errorMessage = nil
```

```
                        }
                        self?.isLoading = false
                    }
                }
            } catch {
                errorMessage = "Your profile couldn't be prepared for saving. Check the text fields for unusual characters."
                isLoading = false
            }
        }


        private func removeListener() {
            listener?.remove()
            listener = nil
            if let handle = authStateHandle {
                Auth.auth().removeStateDidChangeListener(handle)
                authStateHandle = nil
            }
        }

        private func stopSavingIndicatorIfServerIsSlow() {
            DispatchQueue.main.asyncAfter(deadline: .now() + 1.5) { [weak self] in
                guard let self, self.isLoading else { return }
                self.isLoading = false
            }
        }

        deinit {
            removeListener()
        }
    }
```

WorkoutViewModel.swift

AetherGain/ViewModels/WorkoutViewModel.swift

```swift


import Foundation
import FirebaseAuth
import FirebaseFirestore
import Combine

class WorkoutViewModel: ObservableObject {

    @Published var currentSession: WorkoutSession = WorkoutSession()
    @Published var history: [WorkoutSession] = []
    @Published var isLoading = false
    @Published var errorMessage: String?

    private var db = Firestore.firestore()
    private var listener: ListenerRegistration?

    private var userID: String? {
        Auth.auth().currentUser?.uid
    }

    init() {
        Auth.auth().addStateDidChangeListener { [weak self] _, user in
            if let uid = user?.uid {
                self?.loadHistory(for: uid)
            } else {
                self?.history = []
                self?.removeListener()
            }
        }
    }


    func addExercise() {
        currentSession.exercises.append(ExerciseTemplate.configuredExercise(named: "Bench Press"))
    }

    func addCustomExercise(named name: String) {
        let trimmedName = name.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmedName.isEmpty else {
            errorMessage = "Enter an exercise name"
            return
        }

        currentSession.exercises.append(ExerciseTemplate.configuredExercise(named: trimmedName))
```

```
            errorMessage = nil
        }

        func applyTemplate(_ template: ExerciseTemplate, at index: Int) {
            guard currentSession.exercises.indices.contains(index) else { return }
            currentSession.exercises[index] = ExerciseTemplate.configuredExercise(named: template.name)
        }


        func removeExercise(at index: Int) {
            currentSession.exercises.remove(at: index)
        }


        func completeSession() {
            guard let uid = userID else {
                errorMessage = "You're signed out. Sign in again to continue."
                return
            }

            guard currentSession.isValidForXP else {
                errorMessage = "Add at least one valid exercise (sets & reps > 0)"
                return
            }

            isLoading = true
            stopSavingIndicatorIfServerIsSlow()

            let earnedXP = GameEngine.workoutXP(for: currentSession)
            currentSession.totalXP = earnedXP


            if currentSession.date > Date() {

                currentSession.date = Date()
            }



            let batch = db.batch()


            let sessionRef = db.collection("users").document(uid).collection("workouts").document()
            do {
                try batch.setData(from: currentSession, forDocument: sessionRef)
            } catch {
                errorMessage = "This workout couldn't be prepared for saving. Check the exercise names and try again."
                isLoading = false
                return
            }


            let profileRef = db.collection("users").document(uid)
            batch.updateData([
                "xp": FieldValue.increment(Int64(earnedXP))
```

```
            ], forDocument: profileRef)


            batch.commit { [weak self] error in
                DispatchQueue.main.async {
                    if error != nil {
                        self?.errorMessage = "Couldn't save your workout. Check your connection and tap Complete again."
                    } else {
                        self?.errorMessage = nil

                        self?.currentSession = WorkoutSession()
                    }
                    self?.isLoading = false
                }
            }
        }




        func updateSession(_ edited: WorkoutSession, completion: @escaping (Error?) -> Void) {
            guard let uid = userID, let sessionID = edited.id else {
                completion(NSError(domain: "AGWorkout", code: 400,
                                   userInfo: [NSLocalizedDescriptionKey: "Missing session id."]))
                return
            }
            guard edited.isValidForXP else {
                completion(NSError(domain: "AGWorkout", code: 422,
                                   userInfo: [NSLocalizedDescriptionKey: "Need at least one valid exercise."]))
                return
            }

            var updated = edited
            let previousXP = edited.totalXP
            let newXP = GameEngine.workoutXP(for: updated)
            updated.totalXP = newXP
            let xpDelta = newXP - previousXP

            isLoading = true
            let batch = db.batch()
            let sessionRef = db.collection("users").document(uid).collection("workouts").document(sessionID)
            do {
                try batch.setData(from: updated, forDocument: sessionRef, merge: false)
            } catch {
                isLoading = false
                completion(error); return
            }
            if xpDelta != 0 {
                let profileRef = db.collection("users").document(uid)
                batch.updateData(["xp": FieldValue.increment(Int64(xpDelta))], forDocument: profileRef)
            }
            batch.commit { [weak self] error in
                DispatchQueue.main.async {
                    self?.isLoading = false
                    completion(error)
```

```
                }
            }
        }


        func deleteSession(_ session: WorkoutSession) {
            guard let uid = userID, let sessionID = session.id else { return }
            let batch = db.batch()
            let sessionRef = db.collection("users").document(uid).collection("workouts").document(sessionID)
            batch.deleteDocument(sessionRef)


            let profileRef = db.collection("users").document(uid)
            batch.updateData([
                "xp": FieldValue.increment(Int64(-session.totalXP))
            ], forDocument: profileRef)

            batch.commit { [weak self] error in
                DispatchQueue.main.async {
                    if error != nil {
                        self?.errorMessage = "Couldn't delete that session. Check your connection and try again."
                    }
                }
            }
        }


        private func loadHistory(for uid: String) {
            let collectionRef = db.collection("users").document(uid).collection("workouts")

            listener = collectionRef
                .order(by: "date", descending: true)
                .addSnapshotListener { [weak self] snapshot, error in
                    if error != nil {
                        self?.errorMessage = "Couldn't load your history. Pull down to refresh once you're back online."
                        return
                    }

                    guard let documents = snapshot?.documents else {
                        self?.history = []
                        return
                    }

                    self?.history = documents.compactMap { try? $0.data(as: WorkoutSession.self) }
                }
        }


        private func removeListener() {
            listener?.remove()
            listener = nil
        }

        private func stopSavingIndicatorIfServerIsSlow() {
            DispatchQueue.main.asyncAfter(deadline: .now() + 1.5) { [weak self] in
```

```
                guard let self, self.isLoading else { return }
                self.isLoading = false
            }
        }

        deinit {
            removeListener()
        }
    }
```

LoginView.swift

AetherGain/Views/Auth/LoginView.swift

```swift
import SwiftUI

struct LoginView: View {
    @StateObject private var authVM = AuthViewModel()

    @State private var email: String = ""
    @State private var password: String = ""
    @State private var isSignupMode: Bool = false
    @State private var errorMessage: String? = nil
    @State private var isBusy: Bool = false
    @FocusState private var focused: Field?

    private enum Field { case email, password }

    var body: some View {
        AGScreen {
            ScrollView {
                VStack(spacing: 22) {
                    hero
                    formCard
                    footer
                }
                .padding(20)
                .padding(.top, 24)
            }
            .scrollDismissesKeyboard(.interactively)
        }
        .alert("Session expired", isPresented: $authVM.showTimeoutAlert) {
            Button("OK") { }
        } message: {
            Text("You have been logged out due to 30 minutes of inactivity. Please log in again.")
        }
        .onTapGesture { authVM.userDidInteract() }
        .onAppear { authVM.userDidInteract() }
        .environmentObject(authVM)
    }



    private var hero: some View {
        VStack(spacing: 10) {
            ZStack {
                RoundedRectangle(cornerRadius: 28, style: .continuous)
```

```
                        .fill(LinearGradient(colors: [AGColor.coral, AGColor.peach],
                                             startPoint: .topLeading, endPoint: .bottomTrailing))
                        .frame(width: 92, height: 92)
                        .shadow(color: AGColor.coral.opacity(0.35), radius: 14, x: 0, y: 8)
                    Image(systemName: "bolt.heart.fill")
                        .font(.system(size: 44, weight: .bold))
                        .foregroundColor(.white)
                }
                Text("AetherGain").font(AGFont.display(34)).foregroundColor(AGColor.ink)
                Text(isSignupMode
                     ? "Create your account to start earning XP."
                     : "Welcome back ? let's keep the streak alive.")
                    .font(AGFont.body(15))
                    .foregroundColor(AGColor.inkMuted)
                    .multilineTextAlignment(.center)
                    .padding(.horizontal, 20)
            }
        }



        private var formCard: some View {
            VStack(spacing: 14) {
                fieldBox(icon: "envelope.fill", tint: AGColor.sky) {
                    TextField("Email", text: $email)
                        .keyboardType(.emailAddress)
                        .textInputAutocapitalization(.never)
                        .disableAutocorrection(true)
                        .focused($focused, equals: .email)
                        .submitLabel(.next)
                        .onSubmit { focused = .password }
                }

                fieldBox(icon: "lock.fill", tint: AGColor.lavender) {
                    SecureField("Password", text: $password)
                        .focused($focused, equals: .password)
                        .submitLabel(.go)
                        .onSubmit { submit() }
                }

                if !isSignupMode {
                    HStack {
                        Spacer()
                        Button {
                            sendReset()
                        } label: {
                            Text("Forgot password?")
                                .font(AGFont.caption(12))
                                .foregroundColor(AGColor.teal)
                        }
                        .buttonStyle(.plain)
                    }
                }

                if let error = errorMessage {
```

```
                    HStack(spacing: 6) {
                        Image(systemName: "exclamationmark.triangle.fill")
                        Text(error).font(AGFont.caption(12))
                    }
                    .foregroundColor(AGColor.danger)
                    .frame(maxWidth: .infinity, alignment: .leading)
                }

                AGPushButton(
                    title: isSignupMode ? "Create account" : "Log in",
                    icon: isSignupMode ? "sparkles" : "arrow.right",
                    fill: AGColor.coral,
                    isEnabled: !email.isEmpty && !password.isEmpty,
                    isLoading: isBusy
                ) {
                    submit()
                }

                Button(action: {
                    withAnimation(.spring(response: 0.35)) {
                        isSignupMode.toggle()
                        errorMessage = nil
                    }
                }) {
                    HStack(spacing: 4) {
                        Text(isSignupMode ? "Already have an account?" : "New here?")
                            .foregroundColor(AGColor.inkMuted)
                        Text(isSignupMode ? "Log in" : "Sign up")
                            .foregroundColor(AGColor.coral).bold()
                    }
                    .font(AGFont.body(14))
                }
                .buttonStyle(.plain)
                .padding(.top, 4)
            }
            .agCard()
        }

        private func fieldBox<Content: View>(icon: String, tint: Color,
                                             @ViewBuilder content: () -> Content) -> some View {
            HStack(spacing: 12) {
                ZStack {
                    RoundedRectangle(cornerRadius: 10, style: .continuous)
                        .fill(tint.opacity(0.20))
                        .frame(width: 36, height: 36)
                    Image(systemName: icon).foregroundColor(tint)
                }
                content()
                    .font(AGFont.body(15))
            }
            .padding(.horizontal, 12).padding(.vertical, 10)
            .background(
                RoundedRectangle(cornerRadius: 14, style: .continuous)
                    .fill(AGColor.canvas)
            )
```

```
            .overlay(
                RoundedRectangle(cornerRadius: 14, style: .continuous)
                    .stroke(AGColor.stroke, lineWidth: 1)
            )
        }

        private var footer: some View {
            VStack(spacing: 6) {
                HStack(spacing: 16) {
                    perk(icon: "flame.fill", label: "Streaks")
                    perk(icon: "trophy.fill", label: "Levels")
                    perk(icon: "chart.bar.fill", label: "Trends")
                }
                Text("Encrypted end-to-end ? Firebase Auth")
                    .font(AGFont.caption(11))
                    .foregroundColor(AGColor.inkMuted)
                    .padding(.top, 4)
            }
            .padding(.top, 8)
        }

        private func perk(icon: String, label: String) -> some View {
            HStack(spacing: 4) {
                Image(systemName: icon).foregroundColor(AGColor.coral)
                Text(label).foregroundColor(AGColor.inkMuted)
            }
            .font(AGFont.caption(12))
        }



        private func sendReset() {
            errorMessage = nil
            AGHaptics.tap()
            authVM.sendPasswordReset(email: email) { error in
                DispatchQueue.main.async {
                    if let err = error {
                        errorMessage = err.localizedDescription
                        AGToastCenter.shared.show(err.localizedDescription, kind: .error)
                    } else {
                        AGToastCenter.shared.show("Reset link sent to \(email).", kind: .success)
                    }
                }
            }
        }

        private func submit() {
            guard !isBusy else { return }
            errorMessage = nil
            isBusy = true
            let handler: (Error?) -> Void = { error in
                DispatchQueue.main.async {
                    isBusy = false
                    if let error = error as NSError? {
                        errorMessage = error.localizedDescription
```

```
                        AGHaptics.error()
                    } else {
                        AGToastCenter.shared.show(isSignupMode ? "Welcome to AetherGain!" : "Welcome back!",
                                                  kind: .success,
                                                  celebrate: isSignupMode)
                    }
                }
            }

            if isSignupMode {
                authVM.signUp(email: email, password: password, completion: handler)
            } else {
                authVM.signIn(email: email, password: password, completion: handler)
            }
            authVM.userDidInteract()
        }
    }

    #Preview {
        LoginView()
    }
```

ChartView.swift

AetherGain/Views/ChartView.swift

```swift


import SwiftUI

struct ChartView: View {
    var body: some View {
        AGProgressScreen()
    }
}

#Preview {
    ChartView().environmentObject(AuthViewModel())
}
```

HabitsView.swift

AetherGain/Views/HabitsView.swift

```swift


import SwiftUI

struct HabitsView: View {
    @EnvironmentObject var authVM: AuthViewModel
    @StateObject private var habitsVM = HabitsViewModel()

    @State private var reminderTime: Date = Calendar.current.date(bySettingHour: 20, minute: 0, second: 0, of: Date()) ?? Date()

    var body: some View {
        ScrollView {
            VStack(spacing: 14) {
                streakHeader
                waterCard
                sleepCard
                customHabitCard
                reminderCard
                saveButton
            }
            .padding(16)
            .padding(.bottom, 40)
        }
        .background(AGColor.canvas.ignoresSafeArea())
        .overlay(AGGrid().opacity(0.35).ignoresSafeArea().allowsHitTesting(false))
        .navigationBarTitleDisplayMode(.inline)
        .toolbar {
            ToolbarItem(placement: .principal) {
                Text("HABITS").kerning(2).font(AGFont.hud(12))
                    .foregroundColor(AGColor.amber)
            }
        }
        .onAppear {
            authVM.userDidInteract()
            reminderTime = Calendar.current.date(
                bySettingHour: habitsVM.profile.reminderHour,
                minute: habitsVM.profile.reminderMinute,
                second: 0, of: Date()
            ) ?? Date()
        }
    }

    private var streakProgress: Double {
        let done = [waterDone, sleepDone, habitsVM.profile.customHabitCompletedToday].filter { $0 }.count
        return Double(done) / 3.0
```

```
        }
        private var waterDone: Bool { habitsVM.profile.dailyWaterML >= habitsVM.profile.waterMLTarget * 0.8 }
        private var sleepDone: Bool { habitsVM.profile.sleepHours >= habitsVM.profile.sleepHoursTarget * 0.85 }



        private var streakHeader: some View {
            HStack(spacing: 16) {
                AGProgressRing(progress: streakProgress,
                               color: AGColor.amber, size: 88,
                               label: "\(habitsVM.profile.streakDays)",
                               sublabel: "DAY STREAK")
                VStack(alignment: .leading, spacing: 6) {
                    Text("KEEP THE CHAIN ALIVE").kerning(1.4).font(AGFont.hud(10))
                        .foregroundColor(AGColor.amber)
                    Text("Hit all three targets today to extend the streak by one day.")
                        .font(AGFont.body(12)).foregroundColor(AGColor.inkMuted)
                        .fixedSize(horizontal: false, vertical: true)
                }
            }
            .agCard()
        }



        private var waterCard: some View {
            VStack(alignment: .leading, spacing: 12) {
                HStack {
                    AGSectionHeader(title: "Water")
                    Spacer()
                    TextField("0", value: $habitsVM.profile.dailyWaterML, format: .number)
                        .keyboardType(.numberPad)
                        .multilineTextAlignment(.trailing)
                        .frame(width: 70)
                        .font(AGFont.mono(13, weight: .semibold))
                        .foregroundColor(AGColor.ink)
                        .padding(.horizontal, 8).padding(.vertical, 4)
                        .background(RoundedRectangle(cornerRadius: 6).fill(AGColor.panelHi))
                        .overlay(RoundedRectangle(cornerRadius: 6).stroke(AGColor.teal.opacity(0.35), lineWidth: 1))
                    Text("/ \(Int(habitsVM.profile.waterMLTarget)) ml")
                        .font(AGFont.mono(11)).foregroundColor(AGColor.inkMuted)
                }

                Slider(value: $habitsVM.profile.dailyWaterML,
                       in: 0...(max(4000, habitsVM.profile.waterMLTarget * 1.5)),
                       step: 100).tint(AGColor.teal)

                HStack(spacing: 8) {
                    ForEach([250, 500, 750, 1000], id: \.self) { amount in
                        Button {
                            habitsVM.profile.dailyWaterML = min(habitsVM.profile.waterMLTarget * 2,
                                                                habitsVM.profile.dailyWaterML + Double(amount))
                            AGHaptics.tap()
                        } label: {
                            Text("+\(amount)").font(AGFont.hud(10)).kerning(1)
```

```
                                .padding(.horizontal, 10).padding(.vertical, 5)
                                .background(RoundedRectangle(cornerRadius: 8).fill(AGColor.teal.opacity(0.10)))
                                .overlay(RoundedRectangle(cornerRadius: 8).stroke(AGColor.teal.opacity(0.35), lineWidth: 1))
                                .foregroundColor(AGColor.teal)
                        }
                        .buttonStyle(.plain)
                    }
                }

                targetSlider(title: "Daily target",
                             value: $habitsVM.profile.waterMLTarget,
                             range: 500...5000, step: 100,
                             format: { "\(Int($0)) ml" },
                             tint: AGColor.teal)
            }
            .agCard()
        }



        private var sleepCard: some View {
            VStack(alignment: .leading, spacing: 12) {
                HStack {
                    AGSectionHeader(title: "Sleep")
                    Spacer()
                    TextField("0", value: $habitsVM.profile.sleepHours, format: .number.precision(.fractionLength(1)))
                        .keyboardType(.decimalPad)
                        .multilineTextAlignment(.trailing)
                        .frame(width: 60)
                        .font(AGFont.mono(13, weight: .semibold))
                        .foregroundColor(AGColor.ink)
                        .padding(.horizontal, 8).padding(.vertical, 4)
                        .background(RoundedRectangle(cornerRadius: 6).fill(AGColor.panelHi))
                        .overlay(RoundedRectangle(cornerRadius: 6).stroke(AGColor.magenta.opacity(0.35), lineWidth: 1))
                    Text("/ \(String(format: "%.1f", habitsVM.profile.sleepHoursTarget)) h")
                        .font(AGFont.mono(11)).foregroundColor(AGColor.inkMuted)
                }
                Slider(value: $habitsVM.profile.sleepHours, in: 0...12, step: 0.25)
                    .tint(AGColor.magenta)
                targetSlider(title: "Sleep target",
                             value: $habitsVM.profile.sleepHoursTarget,
                             range: 4...12, step: 0.25,
                             format: { String(format: "%.1f h", $0) },
                             tint: AGColor.magenta)
            }
            .agCard()
        }



        private var customHabitCard: some View {
            VStack(alignment: .leading, spacing: 12) {
                HStack {
                    AGSectionHeader(title: "Custom habit")
                    Spacer()
```

```
                    Toggle("", isOn: $habitsVM.profile.customHabitCompletedToday)
                        .labelsHidden().tint(AGColor.amber)
                }
                TextField("Habit name (e.g. Meditation, Journaling)",
                          text: $habitsVM.profile.customHabitName)
                    .font(AGFont.body(14))
                    .foregroundColor(AGColor.ink)
                    .padding(.horizontal, 12).padding(.vertical, 10)
                    .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
                    .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.stroke, lineWidth: 1))
            }
            .agCard()
        }



        private var reminderCard: some View {
            VStack(alignment: .leading, spacing: 12) {
                Toggle(isOn: Binding(
                    get: { habitsVM.profile.remindersEnabled },
                    set: { newValue in
                        habitsVM.profile.remindersEnabled = newValue
                        habitsVM.toggleReminders(enabled: newValue)
                    }
                )) {
                    VStack(alignment: .leading, spacing: 2) {
                        Text("Daily reminder").font(AGFont.headline(15))
                            .foregroundColor(AGColor.ink)
                        Text("Notification to log water, sleep and your habit.")
                            .font(AGFont.body(12)).foregroundColor(AGColor.inkMuted)
                    }
                }
                .tint(AGColor.amber)

                if habitsVM.profile.remindersEnabled {
                    DatePicker("Time",
                               selection: $reminderTime,
                               displayedComponents: .hourAndMinute)
                        .datePickerStyle(.compact)
                        .foregroundColor(AGColor.ink)
                        .onChange(of: reminderTime) { _, new in
                            let comps = Calendar.current.dateComponents([.hour, .minute], from: new)
                            habitsVM.profile.reminderHour = comps.hour ?? 20
                            habitsVM.profile.reminderMinute = comps.minute ?? 0
                            habitsVM.saveHabits()
                            habitsVM.scheduleDailyReminder()
                        }
                }
            }
            .agCard()
        }



        private func targetSlider(title: String,
```

```
                                value: Binding<Double>,
                                range: ClosedRange<Double>,
                                step: Double,
                                format: @escaping (Double) -> String,
                                tint: Color) -> some View {
            VStack(alignment: .leading, spacing: 6) {
                HStack {
                    Text(title.uppercased()).kerning(1.2).font(AGFont.hud(10))
                        .foregroundColor(AGColor.inkMuted)
                    Spacer()
                    TextField("0", value: value, format: .number.precision(.fractionLength(step < 1 ? 1 : 0)))
                        .keyboardType(step < 1 ? .decimalPad : .numberPad)
                        .multilineTextAlignment(.trailing)
                        .frame(width: 60)
                        .font(AGFont.mono(11, weight: .semibold))
                        .foregroundColor(tint)
                        .padding(.horizontal, 6).padding(.vertical, 3)
                        .background(RoundedRectangle(cornerRadius: 6).fill(AGColor.panelHi))
                        .overlay(RoundedRectangle(cornerRadius: 6).stroke(tint.opacity(0.35), lineWidth: 1))
                        .onChange(of: value.wrappedValue) { _, newVal in

                            if newVal < range.lowerBound { value.wrappedValue = range.lowerBound }
                            if newVal > range.upperBound { value.wrappedValue = range.upperBound }
                        }
                }
                Slider(value: value, in: range, step: step).tint(tint)
            }
            .padding(.top, 4)
        }



        private var saveButton: some View {
            AGPushButton(title: "Save habits", icon: "checkmark",
                         fill: AGColor.amber, textColor: .black,
                         isLoading: habitsVM.isLoading) {
                let wasStreak = habitsVM.profile.streakDays
                habitsVM.saveHabits()
                authVM.userDidInteract()
                DispatchQueue.main.asyncAfter(deadline: .now() + 0.3) {
                    if habitsVM.profile.streakDays > wasStreak {
                        AGToastCenter.shared.show("Streak extended to \(habitsVM.profile.streakDays) days.",
                                                  kind: .success, celebrate: true)
                    } else if habitsVM.errorMessage == nil {
                        AGToastCenter.shared.show("Habits saved.", kind: .success)
                    } else if let msg = habitsVM.errorMessage {
                        AGToastCenter.shared.show(msg, kind: .error)
                    }
                }
            }
        }
    }

    #Preview {
        NavigationStack { HabitsView().environmentObject(AuthViewModel()) }
```

```
271     }
```

LeaderboardView.swift

AetherGain/Views/LeaderboardView.swift

```swift


import SwiftUI

struct LeaderboardView: View {
    @EnvironmentObject var authVM: AuthViewModel
    @StateObject private var leaderboardVM = LeaderboardViewModel()

    var body: some View {
        ScrollView {
            VStack(spacing: 14) {
                header
                if let error = leaderboardVM.errorMessage {
                    hintCard("Leaderboard could not load: " + error)
                }
                if let hint = leaderboardVM.rulesHint {
                    hintCard(hint)
                }
                if leaderboardVM.users.isEmpty {
                    emptyCard
                } else {
                    podiumCard
                    fullListCard
                }
                scoringCard
            }
            .padding(16)
            .padding(.bottom, 40)
        }
        .background(AGColor.canvas.ignoresSafeArea())
        .navigationBarTitleDisplayMode(.inline)
        .toolbar {
            ToolbarItem(placement: .principal) {
                Text("Leaderboard").font(AGFont.headline(17))
            }
        }
        .onAppear {
            leaderboardVM.loadLeaderboard()
            authVM.userDidInteract()
        }
    }

    private var header: some View {
        HStack(spacing: 12) {
            ZStack {
                RoundedRectangle(cornerRadius: 18, style: .continuous)
                    .fill(LinearGradient(colors: [AGColor.sunshine, AGColor.peach],
```

```
                                        startPoint: .topLeading, endPoint: .bottomTrailing))
                        .frame(width: 56, height: 56)
                    Image(systemName: "trophy.fill").foregroundColor(.white)
                        .font(.system(size: 24, weight: .bold))
                }
                VStack(alignment: .leading, spacing: 2) {
                    Text("Global XP ranking").font(AGFont.title(20))
                    Text("Only users who opted in appear here.")
                        .font(AGFont.caption(12)).foregroundColor(AGColor.inkMuted)
                }
                Spacer()
            }
            .agCard()
        }

        private func hintCard(_ hint: String) -> some View {
            HStack(alignment: .top, spacing: 10) {
                Image(systemName: "info.circle.fill").foregroundColor(AGColor.amber)
                Text(hint).font(AGFont.body(12)).foregroundColor(AGColor.inkMuted)
                    .fixedSize(horizontal: false, vertical: true)
                Spacer(minLength: 0)
            }
            .padding(12)
            .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.amber.opacity(0.08)))
            .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.amber.opacity(0.3), lineWidth: 1))
        }

        private var emptyCard: some View {
            VStack(spacing: 12) {
                Image(systemName: "person.3.fill").font(.system(size: 30))
                    .foregroundColor(AGColor.inkMuted)
                Text("No opt-ins yet").font(AGFont.headline(15))
                Text("Toggle 'Share on leaderboard' in Profile to appear here.")
                    .font(AGFont.caption(12)).foregroundColor(AGColor.inkMuted)
                    .multilineTextAlignment(.center)
            }
            .padding(30)
            .frame(maxWidth: .infinity)
            .agCard()
        }

        private var podiumCard: some View {
            let top = Array(leaderboardVM.users.prefix(3))
            return HStack(alignment: .bottom, spacing: 12) {
                if top.count >= 2 {
                    podiumTile(rank: 2, user: top[1], height: 92, tint: AGColor.sky)
                } else { Spacer().frame(maxWidth: .infinity) }
                if top.count >= 1 {
                    podiumTile(rank: 1, user: top[0], height: 116, tint: AGColor.sunshine)
                } else { Spacer().frame(maxWidth: .infinity) }
                if top.count >= 3 {
                    podiumTile(rank: 3, user: top[2], height: 76, tint: AGColor.peach)
                } else { Spacer().frame(maxWidth: .infinity) }
            }
            .padding(.vertical, 10)
```

```
            .agCard()
        }

        private func podiumTile(rank: Int, user: LeaderboardUser, height: CGFloat, tint: Color) -> some View {
            VStack(spacing: 6) {
                ZStack {
                    Circle().fill(tint.opacity(0.25)).frame(width: 46, height: 46)
                    Text(String(user.name.prefix(1)).uppercased())
                        .font(AGFont.title(20)).foregroundColor(tint)
                }
                Text(user.name.isEmpty ? "Anon" : user.name)
                    .font(AGFont.headline(12)).lineLimit(1)
                Text("\(user.xp) XP").font(AGFont.caption(11)).foregroundColor(AGColor.inkMuted)
                ZStack(alignment: .top) {
                    RoundedRectangle(cornerRadius: 12).fill(tint.opacity(0.25))
                        .frame(height: height)
                    Text("\(rank)")
                        .font(AGFont.number(28))
                        .foregroundColor(tint)
                        .padding(.top, 6)
                }
            }
            .frame(maxWidth: .infinity)
        }

        private var fullListCard: some View {
            VStack(spacing: 8) {
                ForEach(Array(leaderboardVM.users.enumerated()), id: \.element.id) { index, user in
                    HStack(spacing: 12) {
                        Text("\(index + 1)")
                            .font(AGFont.number(16))
                            .frame(width: 30, height: 30)
                            .background(Circle().fill(index < 3 ? AGColor.sunshine.opacity(0.25) : AGColor.canvas))
                            .overlay(Circle().stroke(AGColor.stroke, lineWidth: 1))
                        VStack(alignment: .leading, spacing: 2) {
                            Text(user.name.isEmpty ? "Unnamed" : user.name)
                                .font(AGFont.headline(14))
                            Text("Level \(user.level) · \(user.xp) XP · \(GameEngine.xpForNextLevel(from: user.xp)) to next")
                                .font(AGFont.caption(11)).foregroundColor(AGColor.inkMuted)
                            if !user.badges.isEmpty {
                                ScrollView(.horizontal, showsIndicators: false) {
                                    HStack(spacing: 6) {
                                        ForEach(user.badges) { badge in
                                            Text(badge.title)
                                                .font(AGFont.caption(10).weight(.semibold))
                                                .padding(.horizontal, 8).padding(.vertical, 3)
                                                .background(Capsule().fill(AGColor.coral.opacity(0.15)))
                                                .foregroundColor(AGColor.coral)
                                        }
                                    }
                                }
                            }
                        }
                        Spacer()
                    }
```

```
                    .padding(.vertical, 6)
                    if index < leaderboardVM.users.count - 1 {
                        Divider().background(AGColor.stroke)
                    }
                }
            }
            .agCard()
        }

        private var scoringCard: some View {
            VStack(alignment: .leading, spacing: 6) {
                AGSectionHeader(title: "How XP works")
                Text("Workouts award a base 50 XP plus bonuses: strength scales with volume, cardio with duration/distance/incline, mobility with time.
        Levels roll over every 500 XP. Habit streaks add 20 + 5 × streak days once all three habits are logged.")
                    .font(AGFont.body(13))
                    .foregroundColor(AGColor.inkMuted)
            }
            .agCard()
        }
    }

    #Preview {
        NavigationStack { LeaderboardView().environmentObject(AuthViewModel()) }
    }
```

HomeView.swift

AetherGain/Views/Main/HomeView.swift

```swift
import SwiftUI

struct HomeView: View {
    @EnvironmentObject var authVM: AuthViewModel
    @StateObject private var profileVM = ProfileViewModel()
    @StateObject private var habitsVM  = HabitsViewModel()
    @StateObject private var mealVM    = MealViewModel()
    @StateObject private var workoutVM = WorkoutViewModel()


    private var xp: Int { profileVM.profile.xp }
    private var level: Int { GameEngine.level(for: xp) }
    private var xpForLevel: Int { max(1, level * 500 - (level - 1) * 500) }
    private var progressToNext: Double {
        let base = (level - 1) * 500
        let toNext = level * 500
        let span = max(1, toNext - base)
        return Double(xp - base) / Double(span)
    }

    private var todaysMeals: [MealEntry] {
        let start = Calendar.current.startOfDay(for: Date())
        return mealVM.meals.filter { Calendar.current.isDate($0.date, inSameDayAs: start) }
    }
    private var todaysCalories: Int { Int(todaysMeals.reduce(0) { $0 + $1.calories }) }
    private var todaysProtein: Int { Int(todaysMeals.reduce(0) { $0 + $1.protein }) }
    private var calorieTarget: Int {
        profileVM.profile.calorieTarget > 0 ? profileVM.profile.calorieTarget : 2500
    }

    private var greeting: String {
        let h = Calendar.current.component(.hour, from: Date())
        switch h {
        case 5..<12: return "Rise, replicant"
        case 12..<17: return "Standby active"
        case 17..<22: return "Nightshift approach"
        default: return "3AM run detected"
        }
    }

    var body: some View {
        ScrollView {
            VStack(spacing: 14) {
                hudHeader
                targetsHUD
```

```
                    heroLevelCard
                    quickTiles
                    mealsPreviewCard
                    lastWorkoutCard
                    habitsMiniCard
                }
                .padding(16)
                .padding(.bottom, 60)
            }
            .background(AGColor.canvas.ignoresSafeArea())
            .overlay(AGGrid().opacity(0.35).ignoresSafeArea().allowsHitTesting(false))
            .navigationBarTitleDisplayMode(.inline)
            .toolbar {
                ToolbarItem(placement: .principal) {
                    Text("AETHERGAIN").kerning(3).font(AGFont.hud(13))
                        .foregroundColor(AGColor.amber)
                }
                ToolbarItem(placement: .topBarLeading) {
                    NavigationLink { LeaderboardView() } label: {
                        Image(systemName: "trophy.fill").foregroundColor(AGColor.amber)
                    }
                }
                ToolbarItem(placement: .topBarTrailing) {
                    NavigationLink { ProfileView() } label: {
                        Image(systemName: "person.crop.circle.fill").foregroundColor(AGColor.teal)
                    }
                }
            }
            .onAppear { authVM.userDidInteract() }
        }



        private var hudHeader: some View {
            HStack(alignment: .top) {
                VStack(alignment: .leading, spacing: 4) {
                    Text(greeting.uppercased()).kerning(1.8).font(AGFont.hud(10))
                        .foregroundColor(AGColor.inkMuted)
                    Text(profileVM.profile.name.isEmpty ? "Operator" : profileVM.profile.name)
                        .font(AGFont.display(28)).foregroundColor(AGColor.ink)
                }
                Spacer()
                AGStreakBadge(days: habitsVM.profile.streakDays)
            }
        }



        private var targetsHUD: some View {
            let remaining = calorieTarget - todaysCalories
            return VStack(alignment: .leading, spacing: 10) {
                AGSectionHeader(title: "Calories Remaining",
                                subtitle: "Goal ? Food + Exercise")
                HStack(alignment: .firstTextBaseline, spacing: 6) {
                    col("\(calorieTarget)", "GOAL")
```

```
                    sign("?")
                    col("\(todaysCalories)", "FOOD", tint: AGColor.amber)
                    sign("+")
                    col("0", "EXERCISE")
                    sign("=")
                    col("\(remaining)", "REMAIN",
                        tint: remaining >= 0 ? AGColor.teal : AGColor.danger)
                }
                ProgressView(value: min(1, Double(todaysCalories) / Double(max(1, calorieTarget))))
                    .tint(AGColor.amber)
                HStack(spacing: 8) {
                    AGHudChip(text: "P \(todaysProtein)/\(profileVM.profile.proteinTargetG)g", tint: AGColor.teal)
                    AGHudChip(text: "TDEE \(profileVM.profile.tdee) kcal", tint: AGColor.magenta)
                    AGHudChip(text: profileVM.profile.fitnessGoal.rawValue.uppercased(), tint: AGColor.amber)
                }
            }
            .agCard()
        }
        private func col(_ value: String, _ label: String, tint: Color = AGColor.ink) -> some View {
            VStack(spacing: 2) {
                Text(value).font(AGFont.number(18)).foregroundColor(tint).lineLimit(1)
                    .minimumScaleFactor(0.5)
                Text(label).kerning(1).font(AGFont.hud(9)).foregroundColor(AGColor.inkMuted)
            }
            .frame(maxWidth: .infinity)
        }
        private func sign(_ s: String) -> some View {
            Text(s).font(AGFont.mono(14)).foregroundColor(AGColor.inkMuted)
        }



        private var heroLevelCard: some View {
            HStack(spacing: 14) {
                AGProgressRing(progress: progressToNext, color: AGColor.amber, size: 88,
                               label: "L\(level)", sublabel: "\(xp) XP")
                VStack(alignment: .leading, spacing: 6) {
                    Text("LEVEL \(level)").kerning(1.4).font(AGFont.hud(11))
                        .foregroundColor(AGColor.amber)
                    Text("\(GameEngine.xpForNextLevel(from: xp)) XP to L\(level + 1)")
                        .font(AGFont.body(13)).foregroundColor(AGColor.inkMuted)
                    ProgressView(value: progressToNext).tint(AGColor.amber)
                }
                Spacer(minLength: 0)
            }
            .agCard()
        }



        private var quickTiles: some View {
            HStack(spacing: 10) {
                NavigationLink { WorkoutsHubView() } label: {
                    tile(icon: "dumbbell.fill", label: "LOG WORKOUT", tint: AGColor.amber)
                }.buttonStyle(.plain)
```

```
                NavigationLink { MealLogView() } label: {
                    tile(icon: "fork.knife", label: "LOG MEAL", tint: AGColor.teal)
                }.buttonStyle(.plain)
                NavigationLink { HabitsView() } label: {
                    tile(icon: "drop.fill", label: "HABITS", tint: AGColor.magenta)
                }.buttonStyle(.plain)
            }
        }
        private func tile(icon: String, label: String, tint: Color) -> some View {
            VStack(spacing: 6) {
                Image(systemName: icon).foregroundColor(tint)
                    .font(.system(size: 20, weight: .bold))
                Text(label).kerning(1.4).font(AGFont.hud(9)).foregroundColor(AGColor.ink)
            }
            .frame(maxWidth: .infinity)
            .padding(.vertical, 14)
            .background(RoundedRectangle(cornerRadius: 12).fill(AGColor.panel))
            .overlay(RoundedRectangle(cornerRadius: 12).stroke(tint.opacity(0.35), lineWidth: 1))
        }



        private var mealsPreviewCard: some View {
            VStack(alignment: .leading, spacing: 8) {
                AGSectionHeader(title: "Today's diary",
                                subtitle: todaysMeals.isEmpty ? "Nothing logged yet" : "\(todaysMeals.count) items · \(todaysCalories) kcal")
                if todaysMeals.isEmpty {
                    NavigationLink { MealLogView() } label: {
                        HStack {
                            Image(systemName: "plus.circle").foregroundColor(AGColor.teal)
                            Text("Log your first meal").font(AGFont.body(13))
                                .foregroundColor(AGColor.ink)
                            Spacer()
                            Image(systemName: "arrow.right").foregroundColor(AGColor.amber)
                        }
                    }
                    .buttonStyle(.plain)
                } else {
                    ForEach(todaysMeals.prefix(4)) { meal in
                        HStack(spacing: 10) {
                            Image(systemName: meal.mealType.icon).foregroundColor(AGColor.amber)
                                .frame(width: 20)
                            VStack(alignment: .leading, spacing: 2) {
                                Text(meal.foodName).font(AGFont.body(14))
                                    .foregroundColor(AGColor.ink).lineLimit(1)
                                Text("\(Int(meal.grams))g · \(meal.mealType.rawValue.uppercased())")
                                    .kerning(1).font(AGFont.hud(9)).foregroundColor(AGColor.inkMuted)
                            }
                            Spacer()
                            Text("\(Int(meal.calories))")
                                .font(AGFont.mono(13, weight: .semibold))
                                .foregroundColor(AGColor.ink)
                        }
                    }
                }
```

```
            }
            .agCard()
        }



        private var lastWorkoutCard: some View {
            VStack(alignment: .leading, spacing: 10) {
                AGSectionHeader(title: "Last workout")
                if let last = workoutVM.history.first {
                    HStack {
                        VStack(alignment: .leading, spacing: 2) {
                            Text(last.date, style: .date).font(AGFont.body(14))
                                .foregroundColor(AGColor.ink)
                            Text("\(last.exercises.count) EXERCISES").kerning(1.2)
                                .font(AGFont.hud(9)).foregroundColor(AGColor.inkMuted)
                        }
                        Spacer()
                        Text("+\(last.totalXP) XP").font(AGFont.mono(14, weight: .semibold))
                            .foregroundColor(AGColor.teal)
                            .padding(.horizontal, 10).padding(.vertical, 4)
                            .background(Capsule().fill(AGColor.teal.opacity(0.10)))
                            .overlay(Capsule().stroke(AGColor.teal.opacity(0.35), lineWidth: 1))
                    }
                } else {
                    NavigationLink { WorkoutsHubView() } label: {
                        HStack {
                            Text("Start your first workout").font(AGFont.body(13))
                                .foregroundColor(AGColor.ink)
                            Spacer()
                            Image(systemName: "arrow.right").foregroundColor(AGColor.amber)
                        }
                    }
                    .buttonStyle(.plain)
                }
            }
            .agCard()
        }



        private var habitsMiniCard: some View {
            VStack(alignment: .leading, spacing: 10) {
                AGSectionHeader(title: "Habits today")
                HStack(spacing: 10) {
                    AGProgressRing(progress: min(1, habitsVM.profile.dailyWaterML / max(1, habitsVM.profile.waterMLTarget)),
                                   color: AGColor.teal, size: 78,
                                   label: "\(Int(habitsVM.profile.dailyWaterML))",
                                   sublabel: "ML / \(Int(habitsVM.profile.waterMLTarget))")
                    AGProgressRing(progress: min(1, habitsVM.profile.sleepHours / max(1, habitsVM.profile.sleepHoursTarget)),
                                   color: AGColor.magenta, size: 78,
                                   label: String(format: "%.1f", habitsVM.profile.sleepHours),
                                   sublabel: "H / \(String(format: "%.0f", habitsVM.profile.sleepHoursTarget))")
                    AGProgressRing(progress: habitsVM.profile.customHabitCompletedToday ? 1 : 0,
                                   color: AGColor.amber, size: 78,
```

```
                                label: habitsVM.profile.customHabitCompletedToday ? "?" : "?",
                                sublabel: habitsVM.profile.customHabitName)
                }
            }
            .agCard()
        }
    }

    #Preview {
        NavigationStack { HomeView().environmentObject(AuthViewModel()) }
    }
```

MainTabView.swift

AetherGain/Views/Main/MainTabView.swift

```swift
import SwiftUI

struct MainTabView: View {
    @EnvironmentObject var authVM: AuthViewModel

    init() {
        let tab = UITabBarAppearance()
        tab.configureWithOpaqueBackground()
        tab.backgroundColor = UIColor(AGColor.card)
        tab.stackedLayoutAppearance.selected.iconColor = UIColor(AGColor.coral)
        tab.stackedLayoutAppearance.selected.titleTextAttributes = [
            .foregroundColor: UIColor(AGColor.coral),
            .font: UIFont.systemFont(ofSize: 11, weight: .semibold)
        ]
        tab.stackedLayoutAppearance.normal.iconColor = UIColor(AGColor.inkMuted)
        tab.stackedLayoutAppearance.normal.titleTextAttributes = [
            .foregroundColor: UIColor(AGColor.inkMuted),
            .font: UIFont.systemFont(ofSize: 11, weight: .medium)
        ]
        UITabBar.appearance().standardAppearance = tab
        UITabBar.appearance().scrollEdgeAppearance = tab

        let nav = UINavigationBarAppearance()
        nav.configureWithTransparentBackground()
        nav.backgroundColor = UIColor(AGColor.canvas)
        nav.titleTextAttributes = [.foregroundColor: UIColor(AGColor.ink)]
        nav.largeTitleTextAttributes = [.foregroundColor: UIColor(AGColor.ink)]
        UINavigationBar.appearance().standardAppearance = nav
        UINavigationBar.appearance().scrollEdgeAppearance = nav
        UINavigationBar.appearance().compactAppearance = nav
    }

    var body: some View {
        TabView {
            NavigationStack { HomeView() }
                .tabItem { Label("Home", systemImage: "house.fill") }

            NavigationStack { WorkoutsHubView() }
                .tabItem { Label("Workouts", systemImage: "dumbbell.fill") }

            NavigationStack { MealLogView() }
```

```
                    .tabItem { Label("Meals", systemImage: "fork.knife") }

                NavigationStack { HabitsView() }
                    .tabItem { Label("Habits", systemImage: "drop.fill") }

                NavigationStack { AGProgressScreen() }
                    .tabItem { Label("Progress", systemImage: "chart.bar.fill") }
            }
            .tint(AGColor.coral)
        }
    }



    struct WorkoutsHubView: View {
        @State private var selection: Int = 0

        var body: some View {
            AGScreen {
                VStack(spacing: 12) {
                    Picker("", selection: $selection) {
                        Text("Log").tag(0)
                        Text("History").tag(1)
                    }
                    .pickerStyle(.segmented)
                    .padding(.horizontal, 16)
                    .padding(.top, 8)

                    if selection == 0 {
                        LogWorkoutView()
                    } else {
                        WorkoutHistoryView()
                    }
                }
            }
            .navigationBarTitleDisplayMode(.inline)
            .toolbar {
                ToolbarItem(placement: .principal) {
                    Text("Workouts").font(AGFont.headline(17))
                }
            }
        }
    }

    #Preview {
        MainTabView().environmentObject(AuthViewModel())
    }
```

MealLogView.swift

AetherGain/Views/Meal Logging/MealLogView.swift

```swift


import SwiftUI
import PhotosUI

struct MealLogView: View {
    @EnvironmentObject var authVM: AuthViewModel
    @StateObject private var mealVM = MealViewModel()
    @StateObject private var profileVM = ProfileViewModel()

    @State private var selectedPhoto: PhotosPickerItem?
    @State private var previewImage: UIImage?
    @State private var showLogSheet = false
    @State private var focusedMealType: MealType = .breakfast


    @State private var selectedDate: Date = Calendar.current.startOfDay(for: Date())

    private let unitOptions = ["g", "kg", "ml", "cup", "tbsp", "tsp", "oz", "piece", "serving", "small", "medium", "large"]



    private var mealsForSelectedDay: [MealEntry] {
        mealVM.meals.filter { Calendar.current.isDate($0.date, inSameDayAs: selectedDate) }
    }
    private func meals(for type: MealType) -> [MealEntry] {
        mealsForSelectedDay.filter { $0.mealType == type }
    }
    private var totalCalories: Int { Int(mealsForSelectedDay.reduce(0) { $0 + $1.calories }) }
    private var totalProtein: Int  { Int(mealsForSelectedDay.reduce(0) { $0 + $1.protein }) }
    private var totalCarbs: Int    { Int(mealsForSelectedDay.reduce(0) { $0 + $1.carbs }) }
    private var totalFat: Int      { Int(mealsForSelectedDay.reduce(0) { $0 + $1.fat }) }

    private var calorieTarget: Int {
        profileVM.profile.calorieTarget > 0 ? profileVM.profile.calorieTarget : 2500
    }

    private var isToday: Bool { Calendar.current.isDateInToday(selectedDate) }



    private var dateForNewMeal: Date {
        if isToday { return Date() }

```

```
            let cal = Calendar.current
            let now = cal.dateComponents([.hour, .minute, .second], from: Date())
            return cal.date(bySettingHour: now.hour ?? 12,
                           minute: now.minute ?? 0,
                           second: now.second ?? 0,
                           of: selectedDate) ?? selectedDate
        }
        private var isTomorrowBlocked: Bool {

            Calendar.current.startOfDay(for: selectedDate) >= Calendar.current.startOfDay(for: Date())
        }

        var body: some View {
            ScrollView {
                VStack(alignment: .leading, spacing: 14) {
                    dayPickerBar
                    caloriesRemainingCard
                    macrosDailyCard
                    ForEach(MealType.allCases) { type in
                        mealSection(type: type)
                    }
                }
                .padding(16)
                .padding(.bottom, 40)
            }
            .background(AGColor.canvas.ignoresSafeArea())
            .overlay(AGGrid().opacity(0.35).ignoresSafeArea().allowsHitTesting(false))
            .navigationBarTitleDisplayMode(.inline)
            .toolbar {
                ToolbarItem(placement: .principal) {
                    Text("DIARY").kerning(2).font(AGFont.hud(12))
                        .foregroundColor(AGColor.amber)
                }
            }
            .sheet(isPresented: $showLogSheet) { logSheet.agToastLayer() }
            .onAppear { authVM.userDidInteract() }
        }



        private var caloriesRemainingCard: some View {
            let remaining = calorieTarget - totalCalories

            return VStack(alignment: .leading, spacing: 12) {
                AGSectionHeader(title: "Calories Remaining", subtitle: "Goal ? Food + Exercise")
                HStack(alignment: .firstTextBaseline, spacing: 6) {
                    formulaColumn(value: "\(calorieTarget)", label: "GOAL")
                    sign("?")
                    formulaColumn(value: "\(totalCalories)", label: "FOOD")
                    sign("+")
                    formulaColumn(value: "0", label: "EXERCISE")
                    sign("=")
                    formulaColumn(value: "\(remaining)",
                                  label: "REMAINING",
                                  tint: remaining >= 0 ? AGColor.teal : AGColor.danger)
```

```
                }
                ProgressView(value: min(1, max(0, Double(totalCalories) / Double(max(1, calorieTarget)))))
                    .tint(AGColor.amber)
            }
            .agCard()
        }

        private func formulaColumn(value: String, label: String, tint: Color = AGColor.ink) -> some View {
            VStack(alignment: .center, spacing: 2) {
                Text(value).font(AGFont.number(20)).foregroundColor(tint).lineLimit(1)
                    .minimumScaleFactor(0.5)
                Text(label).font(AGFont.hud(9)).kerning(1)
                    .foregroundColor(AGColor.inkMuted)
            }
            .frame(maxWidth: .infinity)
        }
        private func sign(_ s: String) -> some View {
            Text(s).font(AGFont.mono(16, weight: .semibold)).foregroundColor(AGColor.inkMuted)
        }



        private var macrosDailyCard: some View {
            HStack(spacing: 14) {
                AGProgressRing(progress: ratio(totalCalories, profileVM.profile.calorieTarget),
                               color: AGColor.amber, size: 88,
                               label: "\(totalCalories)", sublabel: "KCAL")
                VStack(alignment: .leading, spacing: 8) {
                    macroLine(label: "Protein", value: totalProtein,
                              target: profileVM.profile.proteinTargetG, tint: AGColor.teal)
                    macroLine(label: "Carbs", value: totalCarbs,
                              target: profileVM.profile.carbsTargetG, tint: AGColor.amberHi)
                    macroLine(label: "Fat",   value: totalFat,
                              target: profileVM.profile.fatTargetG, tint: AGColor.magenta)
                }
            }
            .agCard()
        }
        private func macroLine(label: String, value: Int, target: Int, tint: Color) -> some View {
            let t = max(1, target)
            return VStack(alignment: .leading, spacing: 2) {
                HStack {
                    Text(label.uppercased()).kerning(1.2).font(AGFont.hud(10))
                        .foregroundColor(AGColor.inkMuted)
                    Spacer()
                    Text("\(value) / \(target) g").font(AGFont.mono(11))
                        .foregroundColor(AGColor.ink)
                }
                ProgressView(value: min(1, Double(value) / Double(t))).tint(tint)
            }
        }
        private func ratio(_ value: Int, _ target: Int) -> Double {
            guard target > 0 else { return 0 }
            return min(1, Double(value) / Double(target))
        }
```

```

    private func mealSection(type: MealType) -> some View {
        let items = meals(for: type)
        let kcal = Int(items.reduce(0) { $0 + $1.calories })
        return VStack(alignment: .leading, spacing: 10) {
            HStack {
                Image(systemName: type.icon).foregroundColor(AGColor.amber)
                Text(type.rawValue.uppercased()).kerning(1.6).font(AGFont.hud(11))
                    .foregroundColor(AGColor.amber)
                Spacer()
                Text("\(kcal) KCAL").font(AGFont.mono(11))
                    .foregroundColor(AGColor.inkMuted)
            }
            if items.isEmpty {
                Button {
                    focusedMealType = type
                    mealVM.currentMeal.mealType = type
                    mealVM.currentMeal.date = dateForNewMeal
                    showLogSheet = true
                    AGHaptics.tap()
                } label: {
                    HStack(spacing: 6) {
                        Image(systemName: "plus.circle")
                        Text("Add food").font(AGFont.body(13))
                        Spacer()
                    }
                    .foregroundColor(AGColor.teal)
                    .padding(.vertical, 4)
                }
                .buttonStyle(.plain)
            } else {
                ForEach(items) { entry in
                    AGSwipeToDelete(onDelete: {
                        mealVM.deleteMeal(entry)
                        AGToastCenter.shared.show("Removed \(entry.foodName)", kind: .info)
                    }) {
                        mealRow(entry)
                    }
                    Divider().background(AGColor.stroke)
                }
                Button {
                    focusedMealType = type
                    mealVM.currentMeal.mealType = type
                    mealVM.currentMeal.date = dateForNewMeal
                    showLogSheet = true
                } label: {
                    HStack {
                        Image(systemName: "plus")
                        Text("Add another").font(AGFont.body(12))
                        Spacer()
                    }
                    .foregroundColor(AGColor.teal)
                }
```

```
                    .buttonStyle(.plain)
                }
            }
            .agCard()
        }



        private func mealRow(_ entry: MealEntry) -> some View {
            HStack(alignment: .center, spacing: 10) {
                Button {
                    mealVM.editMeal(entry)
                    focusedMealType = entry.mealType
                    showLogSheet = true
                    AGHaptics.tap()
                } label: {
                    HStack(alignment: .top, spacing: 10) {
                        Rectangle().fill(AGColor.amber.opacity(0.5)).frame(width: 2)
                        VStack(alignment: .leading, spacing: 3) {
                            Text(entry.foodName).font(AGFont.body(14))
                                .foregroundColor(AGColor.ink)
                            Text("\(formatted(entry.amount)) \(entry.unit) · \(Int(entry.grams)) g")
                                .font(AGFont.hud(10)).foregroundColor(AGColor.inkMuted)
                                .kerning(1)
                            sourceBadge(entry.macroSource)
                        }
                        Spacer()
                        VStack(alignment: .trailing, spacing: 2) {
                            Text("\(Int(entry.calories))").font(AGFont.mono(14, weight: .semibold))
                                .foregroundColor(AGColor.ink)
                            Text("KCAL").font(AGFont.hud(9)).kerning(1)
                                .foregroundColor(AGColor.inkMuted)
                        }
                    }
                }
                .buttonStyle(.plain)

                Button(role: .destructive) {
                    mealVM.deleteMeal(entry)
                    AGToastCenter.shared.show("Removed \(entry.foodName)", kind: .info)
                    AGHaptics.tap()
                } label: {
                    Image(systemName: "trash")
                        .font(.system(size: 13, weight: .semibold))
                        .foregroundColor(AGColor.danger)
                        .padding(6)
                        .background(RoundedRectangle(cornerRadius: 8).fill(AGColor.danger.opacity(0.10)))
                        .overlay(RoundedRectangle(cornerRadius: 8).stroke(AGColor.danger.opacity(0.35), lineWidth: 1))
                }
                .buttonStyle(.plain)
            }
        }

        private func sourceBadge(_ source: String) -> some View {
            let tint: Color = {
```

```
                switch source {
                case "Local DB":      return AGColor.teal
                case "OpenFoodFacts": return AGColor.magenta
                default:              return AGColor.inkMuted
                }
            }()
            return HStack(spacing: 4) {
                Image(systemName: source == "Manual" ? "hand.point.up.left.fill" :
                                  source == "Local DB" ? "cylinder.split.1x2.fill" :
                                  "globe")
                    .font(.system(size: 8, weight: .bold))
                Text("SRC · \(source.uppercased())")
                    .font(AGFont.hud(9)).kerning(1)
            }
            .foregroundColor(tint)
            .padding(.horizontal, 6).padding(.vertical, 2)
            .background(RoundedRectangle(cornerRadius: 5).fill(tint.opacity(0.10)))
            .overlay(RoundedRectangle(cornerRadius: 5).stroke(tint.opacity(0.35), lineWidth: 1))
        }

        private func formatted(_ v: Double) -> String {
            v == v.rounded() ? String(Int(v)) : String(format: "%.1f", v)
        }



        private var dayPickerBar: some View {
            HStack(spacing: 12) {
                Button {
                    selectedDate = Calendar.current.date(byAdding: .day, value: -1, to: selectedDate) ?? selectedDate
                    AGHaptics.tap()
                } label: {
                    Image(systemName: "chevron.left")
                        .font(.system(size: 14, weight: .bold))
                        .foregroundColor(AGColor.amber)
                        .padding(10)
                        .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panel))
                        .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.stroke, lineWidth: 1))
                }
                .buttonStyle(.plain)

                VStack(spacing: 2) {
                    Text(isToday ? "TODAY" : selectedDate.formatted(.dateTime.weekday(.wide)).uppercased())
                        .kerning(2).font(AGFont.hud(11))
                        .foregroundColor(AGColor.amber)
                    Text(selectedDate.formatted(.dateTime.day().month(.wide).year()))
                        .font(AGFont.mono(12)).foregroundColor(AGColor.ink)
                }
                .frame(maxWidth: .infinity)

                Button {
                    let next = Calendar.current.date(byAdding: .day, value: 1, to: selectedDate) ?? selectedDate
                    let today = Calendar.current.startOfDay(for: Date())
                    if next > today { return }
                    selectedDate = next
```

```
                    AGHaptics.tap()
                } label: {
                    Image(systemName: "chevron.right")
                        .font(.system(size: 14, weight: .bold))
                        .foregroundColor(isTomorrowBlocked ? AGColor.inkDim : AGColor.amber)
                        .padding(10)
                        .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panel))
                        .overlay(RoundedRectangle(cornerRadius: 10)
                            .stroke((isTomorrowBlocked ? AGColor.stroke : AGColor.amber.opacity(0.4)), lineWidth: 1))
                }
                .buttonStyle(.plain)
                .disabled(isTomorrowBlocked)
            }
        }



        private var logSheet: some View {
            NavigationStack {
                AGScreen {
                    ScrollView {
                        VStack(alignment: .leading, spacing: 14) {
                            Picker("Meal", selection: $mealVM.currentMeal.mealType) {
                                ForEach(MealType.allCases) { Text($0.rawValue).tag($0) }
                            }
                            .pickerStyle(.segmented)

                            photoCard
                            foodNameCard
                            amountUnitCard
                            macrosCard
                            notesCard
                            saveButton
                        }
                        .padding(16)
                        .padding(.bottom, 40)
                    }
                }
                .navigationBarTitleDisplayMode(.inline)
                .toolbar {
                    ToolbarItem(placement: .principal) {
                        Text(mealVM.editingMealID == nil ? "ADD FOOD" : "EDIT FOOD")
                            .kerning(2).font(AGFont.hud(12))
                            .foregroundColor(AGColor.amber)
                    }
                    ToolbarItem(placement: .cancellationAction) {
                        Button("Close") {
                            mealVM.resetCurrentMeal()
                            showLogSheet = false
                        }
                        .foregroundColor(AGColor.teal)
                    }
                }
            }
            .onChange(of: selectedPhoto) { _, newItem in
```

```
                handlePhotoChange(newItem)
            }
        }



        private var photoCard: some View {
            VStack(spacing: 10) {
                if let previewImage {
                    ZStack(alignment: .topTrailing) {
                        Image(uiImage: previewImage)
                            .resizable().scaledToFill()
                            .frame(height: 180)
                            .frame(maxWidth: .infinity)
                            .clipShape(RoundedRectangle(cornerRadius: 12))
                            .overlay(RoundedRectangle(cornerRadius: 12)
                                .stroke(AGColor.amber.opacity(0.4), lineWidth: 1))
                        Button {
                            self.previewImage = nil
                            self.selectedPhoto = nil
                            mealVM.clearPhotoAnalysis()
                            AGHaptics.tap()
                        } label: {
                            Image(systemName: "xmark")
                                .font(.system(size: 12, weight: .bold))
                                .foregroundColor(.white)
                                .padding(8)
                                .background(Circle().fill(Color.black.opacity(0.7)))
                                .overlay(Circle().stroke(AGColor.danger, lineWidth: 1))
                        }
                        .buttonStyle(.plain)
                        .padding(8)
                    }
                } else {
                    ZStack {
                        RoundedRectangle(cornerRadius: 12)
                            .fill(AGColor.panelHi)
                            .frame(height: 140)
                        VStack(spacing: 6) {
                            Image(systemName: "viewfinder").font(.system(size: 28, weight: .bold))
                                .foregroundColor(AGColor.amber)
                            Text("SNAP FOOD").kerning(2).font(AGFont.hud(10))
                                .foregroundColor(AGColor.amber)
                        }
                    }
                }

                HStack(spacing: 10) {
                    PhotosPicker(selection: $selectedPhoto, matching: .images) {
                        HStack(spacing: 6) {
                            Image(systemName: "photo.on.rectangle.angled")
                            Text(previewImage == nil ? "CHOOSE PHOTO" : "CHANGE").kerning(1.4)
                        }
                        .font(AGFont.hud(11))
                        .foregroundColor(.black)
```

```
                        .padding(.horizontal, 14).padding(.vertical, 8)
                        .background(RoundedRectangle(cornerRadius: 8).fill(AGColor.amber))
                }
                if mealVM.isAnalyzingPhoto {
                    ProgressView().tint(AGColor.amber)
                    Text("ANALYZING...").kerning(1.4).font(AGFont.hud(10))
                        .foregroundColor(AGColor.inkMuted)
                }
                Spacer()
            }

            if !mealVM.candidates.isEmpty {
                VStack(alignment: .leading, spacing: 6) {
                    Text("DETECTED · TAP TO USE").kerning(1.4).font(AGFont.hud(9))
                        .foregroundColor(AGColor.inkMuted)
                    ScrollView(.horizontal, showsIndicators: false) {
                        HStack(spacing: 6) {
                            ForEach(mealVM.candidates) { c in
                                Button {
                                    mealVM.selectCandidate(c)
                                    AGHaptics.tap()
                                } label: {
                                    HStack(spacing: 6) {
                                        Text(c.matched ?? c.rawLabel)
                                            .font(AGFont.hud(10)).kerning(1)
                                        Text("\(Int(c.confidence * 100))%")
                                            .font(AGFont.hud(9))
                                            .foregroundColor(AGColor.inkMuted)
                                    }
                                    .padding(.horizontal, 10).padding(.vertical, 5)
                                    .background(RoundedRectangle(cornerRadius: 8)
                                        .fill(c.matched != nil ? AGColor.teal.opacity(0.15) : AGColor.panelHi))
                                    .overlay(RoundedRectangle(cornerRadius: 8)
                                        .stroke((c.matched != nil ? AGColor.teal : AGColor.stroke), lineWidth: 1))
                                    .foregroundColor(c.matched != nil ? AGColor.teal : AGColor.ink)
                                }
                                .buttonStyle(.plain)
                            }
                        }
                    }
                }
            }
        }
        .agCard()
    }

    private var foodNameCard: some View {
        VStack(alignment: .leading, spacing: 8) {
            AGSectionHeader(title: "Food", subtitle: "One item OR a bowl of comma-separated items.")
            TextField("e.g. Oats · or · 100g oats, 2 cups milk, 30g protein",
                      text: $mealVM.currentMeal.foodName, axis: .vertical)
                .lineLimit(1...3)
                .font(AGFont.body(15))
                .foregroundColor(AGColor.ink)
                .padding(.horizontal, 12).padding(.vertical, 10)
```

```
                    .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
                    .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.stroke, lineWidth: 1))

                Text("TIP: separate multiple items with commas to log as one bowl.")
                    .kerning(1).font(AGFont.hud(9))
                    .foregroundColor(AGColor.inkMuted)

                HStack(spacing: 10) {
                    AGGhostButton(title: "Calculate macros", icon: "function", accent: AGColor.teal) {
                        mealVM.calculateNutritionForCurrentMeal()
                        authVM.userDidInteract()
                    }
                    Spacer()
                }
                if let hint = mealVM.referenceHint {
                    Text(hint).font(AGFont.mono(11)).foregroundColor(AGColor.teal)
                        .padding(.top, 2)
                        .fixedSize(horizontal: false, vertical: true)
                }
                if let err = mealVM.errorMessage {
                    Text(err).font(AGFont.body(12)).foregroundColor(AGColor.danger)
                        .fixedSize(horizontal: false, vertical: true)
                }
            }
            .agCard()
        }

        private var amountUnitCard: some View {
            VStack(alignment: .leading, spacing: 10) {
                AGSectionHeader(title: "Amount",
                                subtitle: "e.g. 30 g, 1 cup, 2 pieces ? macros scale automatically.")
                HStack(spacing: 10) {
                    TextField("Amount", value: $mealVM.currentMeal.amount, format: .number)
                        .keyboardType(.decimalPad)
                        .font(AGFont.number(22))
                        .foregroundColor(AGColor.ink)
                        .padding(.horizontal, 12).padding(.vertical, 10)
                        .frame(width: 110)
                        .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
                        .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.stroke, lineWidth: 1))
                        .onChange(of: mealVM.currentMeal.amount) { _, _ in mealVM.recalculateFromCurrent() }

                    Menu {
                        ForEach(unitOptions, id: \.self) { u in
                            Button(u) {
                                mealVM.currentMeal.unit = u
                                mealVM.recalculateFromCurrent()
                            }
                        }
                    } label: {
                        HStack(spacing: 4) {
                            Text(mealVM.currentMeal.unit.uppercased()).kerning(1.4)
                            Image(systemName: "chevron.down").font(.system(size: 10, weight: .bold))
                        }
                        .font(AGFont.hud(11))
```

```
                        .foregroundColor(AGColor.amber)
                        .padding(.horizontal, 12).padding(.vertical, 12)
                        .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
                        .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.amber.opacity(0.4), lineWidth: 1))
                }
                Spacer()
                Text("= \(Int(mealVM.currentMeal.grams)) g")
                    .font(AGFont.mono(13)).foregroundColor(AGColor.inkMuted)
            }
        }
        .agCard()
    }

    private var macrosCard: some View {
        VStack(alignment: .leading, spacing: 12) {
            AGSectionHeader(title: "Macros for this entry")
            HStack(spacing: 12) {
                AGProgressRing(progress: min(1, mealVM.currentMeal.calories / 1000),
                               color: AGColor.amber, size: 92,
                               label: "\(Int(mealVM.currentMeal.calories))",
                               sublabel: "KCAL")
                VStack(alignment: .leading, spacing: 10) {
                    editableMacroRow(name: "Protein", tint: AGColor.teal,
                                     value: Binding(get: { mealVM.currentMeal.protein },
                                                    set: { mealVM.currentMeal.protein = $0 }))
                    editableMacroRow(name: "Carbs", tint: AGColor.amberHi,
                                     value: Binding(get: { mealVM.currentMeal.carbs },
                                                    set: { mealVM.currentMeal.carbs = $0 }))
                    editableMacroRow(name: "Fat", tint: AGColor.magenta,
                                     value: Binding(get: { mealVM.currentMeal.fat },
                                                    set: { mealVM.currentMeal.fat = $0 }))
                }
            }
        }
        .agCard()
    }

    private func editableMacroRow(name: String, tint: Color, value: Binding<Double>) -> some View {
        HStack {
            Circle().fill(tint).frame(width: 8, height: 8)
            Text(name.uppercased()).kerning(1.2).font(AGFont.hud(10))
                .foregroundColor(AGColor.inkMuted)
            Spacer()
            TextField("g", value: value, format: .number)
                .keyboardType(.decimalPad)
                .multilineTextAlignment(.trailing)
                .frame(width: 60)
                .font(AGFont.mono(13))
                .foregroundColor(AGColor.ink)
                .padding(.horizontal, 8).padding(.vertical, 4)
                .background(RoundedRectangle(cornerRadius: 8).fill(AGColor.panelHi))
                .overlay(RoundedRectangle(cornerRadius: 8).stroke(AGColor.stroke, lineWidth: 1))
            Text("g").font(AGFont.hud(10)).foregroundColor(AGColor.inkMuted)
        }
    }
```

```
601
602         private var notesCard: some View {
603             VStack(alignment: .leading, spacing: 8) {
604                 AGSectionHeader(title: "Notes")
605                 TextField("Optional", text: $mealVM.currentMeal.notes)
606                     .font(AGFont.body(14))
607                     .foregroundColor(AGColor.ink)
608                     .padding(.horizontal, 12).padding(.vertical, 10)
609                     .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
610                     .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.stroke, lineWidth: 1))
611             }
612             .agCard()
613         }
614
615         private var saveButton: some View {
616             let editing = mealVM.editingMealID != nil
617             return AGPushButton(
618                 title: editing
619                     ? "Update \(mealVM.currentMeal.mealType.rawValue.uppercased())"
620                     : "Log to \(mealVM.currentMeal.mealType.rawValue.uppercased())",
621                 icon: "checkmark",
622                 fill: AGColor.amber,
623                 textColor: .black,
624                 isEnabled: !mealVM.currentMeal.foodName.isEmpty && mealVM.currentMeal.calories > 0,
625                 isLoading: mealVM.isLoading
626             ) {
627                 let foodName = mealVM.currentMeal.foodName
628                 let mealType = mealVM.currentMeal.mealType
629                 mealVM.saveMeal()
630                 authVM.userDidInteract()
631                 DispatchQueue.main.asyncAfter(deadline: .now() + 0.4) {
632                     if mealVM.errorMessage == nil {
633                         AGToastCenter.shared.show(
634                             editing ? "Updated \(foodName)" : "Logged \(foodName) ? \(mealType.rawValue)",
635                             kind: .success
636                         )
637                         previewImage = nil
638                         selectedPhoto = nil
639                         showLogSheet = false
640                     } else if let msg = mealVM.errorMessage {
641                         AGToastCenter.shared.show(msg, kind: .error)
642                     }
643                 }
644             }
645         }
646
647
648
649         private func handlePhotoChange(_ newItem: PhotosPickerItem?) {
650             Task {
651                 guard let newItem else { return }
652                 do {
653                     if let data = try await newItem.loadTransferable(type: Data.self),
654                        let uiImage = UIImage(data: data) {
655                         await MainActor.run {
```

```
                            self.previewImage = uiImage
                        }
                        mealVM.analyzeMealPhoto(uiImage)
                    } else {
                        await MainActor.run {
                            mealVM.errorMessage = "Couldn't read that image ? try another."
                        }
                    }
                } catch {
                    await MainActor.run {
                        mealVM.errorMessage = "Photo load failed: \(error.localizedDescription)"
                    }
                }
            }
        }
    }

    #Preview {
        NavigationStack { MealLogView().environmentObject(AuthViewModel()) }
    }
```

PrivacyNoticeView.swift

AetherGain/Views/Onboarding/PrivacyNoticeView.swift

```swift


import SwiftUI

struct PrivacyNoticeView: View {
    @AppStorage("ag.privacy.accepted") private var accepted: Bool = false
    @AppStorage("ag.privacy.shareLeaderboard") var optInLeaderboard: Bool = false
    var onDone: () -> Void

    var body: some View {
        AGScreen {
            ScrollView {
                VStack(alignment: .leading, spacing: 16) {
                    header
                    rightsCard(icon: "lock.shield.fill", tint: AGColor.teal,
                               title: "Encrypted",
                               body: "Every byte is encrypted in transit (HTTPS) and at rest on Google Cloud Firestore.")
                    rightsCard(icon: "person.crop.circle.badge.checkmark", tint: AGColor.amber,
                               title: "You control what's shared",
                               body: "Profile, workouts, meals, and habits stay private by default. Nothing appears on the leaderboard unless you opt in
     below.")
                    rightsCard(icon: "trash.circle.fill", tint: AGColor.magenta,
                               title: "Right to delete",
                               body: "Edit or delete any entry from the app. You may also request full account erasure.")
                    rightsCard(icon: "cloud.fill", tint: AGColor.amberHi,
                               title: "What we store",
                               body: "Email (login), profile info you enter, workout/meal/habit logs, XP and streaks. No location, contacts, or health
    data unless you connect it explicitly.")

                    Toggle(isOn: $optInLeaderboard) {
                        VStack(alignment: .leading, spacing: 2) {
                            Text("Show me on the leaderboard")
                                .font(AGFont.headline(14)).foregroundColor(AGColor.ink)
                            Text("Display name, XP, level, and badges become visible to other users. Toggle any time in Profile.")
                                .font(AGFont.body(11)).foregroundColor(AGColor.inkMuted)
                        }
                    }
                    .tint(AGColor.amber)
                    .agCard()

                    AGPushButton(title: "I agree ? continue", icon: "checkmark",
                                 fill: AGColor.amber, textColor: .black) {
                        accepted = true
                        onDone()
                    }
                    .padding(.top, 4)

```

```
                        Text("You are agreeing to the practices above. Revisit any time from Profile ? Privacy.")
                            .font(AGFont.body(11)).foregroundColor(AGColor.inkMuted)
                            .multilineTextAlignment(.center)
                            .frame(maxWidth: .infinity)
                            .padding(.bottom, 30)
                    }
                    .padding(16)
                }
            }
        }

        private var header: some View {
            VStack(alignment: .leading, spacing: 8) {
                Text("WELCOME").kerning(3).font(AGFont.hud(11))
                    .foregroundColor(AGColor.amber)
                Text("AetherGain").font(AGFont.display(32)).foregroundColor(AGColor.ink)
                Text("Before you jack in, a quick word on your data and your rights.")
                    .font(AGFont.body(13)).foregroundColor(AGColor.inkMuted)
            }
        }

        private func rightsCard(icon: String, tint: Color, title: String, body: String) -> some View {
            HStack(alignment: .top, spacing: 12) {
                ZStack {
                    RoundedRectangle(cornerRadius: 10, style: .continuous)
                        .fill(tint.opacity(0.15))
                        .frame(width: 44, height: 44)
                        .overlay(RoundedRectangle(cornerRadius: 10).stroke(tint.opacity(0.4), lineWidth: 1))
                    Image(systemName: icon).font(.system(size: 18, weight: .bold))
                        .foregroundColor(tint)
                }
                VStack(alignment: .leading, spacing: 4) {
                    Text(title.uppercased()).kerning(1.4).font(AGFont.hud(11))
                        .foregroundColor(tint)
                    Text(body).font(AGFont.body(12)).foregroundColor(AGColor.inkMuted)
                        .fixedSize(horizontal: false, vertical: true)
                }
                Spacer(minLength: 0)
            }
            .agCard()
        }
    }

    #Preview {
        PrivacyNoticeView(onDone: {})
    }
```

TargetsOnboardingView.swift

AetherGain/Views/Onboarding/TargetsOnboardingView.swift

```swift


import SwiftUI

struct TargetsOnboardingView: View {
    @StateObject private var profileVM = ProfileViewModel()
    @AppStorage("ag.onboarding.targetsDone") private var targetsDone: Bool = false
    var onDone: () -> Void


    @State private var draft = UserProfile()
    @State private var stepIndex: Int = 0
    private let stepCount = 4

    private var computed: DailyTargets {
        TargetsCalculator.compute(for: draft)
    }

    var body: some View {
        AGScreen {
            VStack(spacing: 0) {
                progressHeader
                ScrollView {
                    VStack(alignment: .leading, spacing: 16) {
                        switch stepIndex {
                        case 0: personalStep
                        case 1: anthropometricsStep
                        case 2: lifestyleStep
                        default: goalStep
                        }
                        previewCard
                    }
                    .padding(16)
                    .padding(.bottom, 30)
                }
                navBar
            }
        }
        .onAppear {

            draft = profileVM.profile
            if draft.age <= 0 { draft.age = 18 }
        }
```

```
        }



        private var progressHeader: some View {
            VStack(spacing: 8) {
                HStack {
                    Text("CALIBRATE TARGETS").kerning(2).font(AGFont.hud(11))
                        .foregroundColor(AGColor.amber)
                    Spacer()
                    Text("STEP \(stepIndex + 1) / \(stepCount)").kerning(1.4)
                        .font(AGFont.hud(10)).foregroundColor(AGColor.inkMuted)
                }
                GeometryReader { geo in
                    ZStack(alignment: .leading) {
                        Rectangle().fill(AGColor.panelHi).frame(height: 3)
                        Rectangle().fill(AGColor.amber)
                            .frame(width: geo.size.width * CGFloat(stepIndex + 1) / CGFloat(stepCount),
                                   height: 3)
                            .shadow(color: AGColor.amber, radius: 4)
                    }
                }
                .frame(height: 3)
            }
            .padding(.horizontal, 16).padding(.top, 12)
        }



        private var personalStep: some View {
            VStack(alignment: .leading, spacing: 12) {
                AGSectionHeader(title: "Who's calibrating?",
                                subtitle: "Basic identity for accurate metabolism.")
                fieldLabel("Name")
                TextField("Name", text: $draft.name)
                    .textFieldStyle(HUDTextFieldStyle())

                fieldLabel("Sex (affects BMR formula)")
                Picker("Sex", selection: $draft.sex) {
                    ForEach(Sex.allCases) { Text($0.rawValue).tag($0) }
                }
                .pickerStyle(.segmented)

                fieldLabel("Age")
                Stepper("Age: \(draft.age)", value: $draft.age, in: 12...100)
                    .font(AGFont.body(14))
                    .foregroundColor(AGColor.ink)
            }
            .agCard()
        }

        private var anthropometricsStep: some View {
            VStack(alignment: .leading, spacing: 12) {
                AGSectionHeader(title: "Body stats")
                HStack {
```

```
                    fieldLabel("Height").frame(width: 100, alignment: .leading)
                    TextField("cm", value: $draft.heightCM, format: .number)
                        .keyboardType(.decimalPad)
                        .textFieldStyle(HUDTextFieldStyle())
                    Text("cm").font(AGFont.hud(10)).foregroundColor(AGColor.inkMuted)
                }
                HStack {
                    fieldLabel("Weight").frame(width: 100, alignment: .leading)
                    TextField("kg", value: $draft.weightKG, format: .number)
                        .keyboardType(.decimalPad)
                        .textFieldStyle(HUDTextFieldStyle())
                    Text("kg").font(AGFont.hud(10)).foregroundColor(AGColor.inkMuted)
                }
                HStack {
                    fieldLabel("Body fat %").frame(width: 100, alignment: .leading)
                    TextField("optional", value: Binding(
                        get: { draft.bodyFatPercent ?? 0 },
                        set: { draft.bodyFatPercent = $0 == 0 ? nil : $0 }
                    ), format: .number)
                        .keyboardType(.decimalPad)
                        .textFieldStyle(HUDTextFieldStyle())
                    Text("%").font(AGFont.hud(10)).foregroundColor(AGColor.inkMuted)
                }
                Text("Body fat is optional ? if provided, we use Katch-McArdle (more accurate for lean users) instead of Mifflin-St Jeor.")
                    .font(AGFont.body(11)).foregroundColor(AGColor.inkMuted)
            }
            .agCard()
        }

        private var lifestyleStep: some View {
            VStack(alignment: .leading, spacing: 12) {
                AGSectionHeader(title: "Activity",
                                subtitle: "Used to convert BMR ? TDEE.")
                fieldLabel("Average steps per day")
                HStack {
                    Slider(value: Binding(
                        get: { Double(draft.avgStepsPerDay) },
                        set: { draft.avgStepsPerDay = Int($0) }
                    ), in: 0...20000, step: 500).tint(AGColor.amber)
                    Text("\(draft.avgStepsPerDay)").font(AGFont.mono(13))
                        .foregroundColor(AGColor.ink).frame(width: 60, alignment: .trailing)
                }

                fieldLabel("Workouts per week")
                Stepper("\(draft.workoutsPerWeek) sessions", value: $draft.workoutsPerWeek, in: 0...14)
                    .foregroundColor(AGColor.ink)

                HStack(spacing: 8) {
                    AGHudChip(text: "BMR = \(computed.bmr) kcal", tint: AGColor.teal)
                    AGHudChip(text: "Activity ×\(String(format: "%.2f", computed.activityMultiplier))",
                              tint: AGColor.magenta)
                }
            }
            .agCard()
        }
```

```

        private var goalStep: some View {
            VStack(alignment: .leading, spacing: 12) {
                AGSectionHeader(title: "Objective")
                fieldLabel("Goal")
                Picker("Goal", selection: $draft.fitnessGoal) {
                    ForEach(FitnessGoal.allCases) { Text($0.rawValue).tag($0) }
                }
                .pickerStyle(.segmented)

                if draft.fitnessGoal != .maintain {
                    fieldLabel("Rate (kg / week)")
                    HStack {
                        Slider(value: $draft.goalRateKgPerWeek, in: 0.1...1.0, step: 0.05)
                            .tint(AGColor.amber)
                        Text(String(format: "%.2f kg/wk", draft.goalRateKgPerWeek))
                            .font(AGFont.mono(12)).foregroundColor(AGColor.ink)
                            .frame(width: 90, alignment: .trailing)
                    }
                    Text("~\(Int(draft.goalRateKgPerWeek * 7700 / 7)) kcal/day \(draft.fitnessGoal == .cut ? "deficit" : "surplus")")
                        .font(AGFont.hud(10)).foregroundColor(AGColor.inkMuted).kerning(1)
                }

                fieldLabel("Protein preference")
                Picker("Protein", selection: $draft.proteinPreference) {
                    ForEach(ProteinPreference.allCases) { p in
                        Text("\(p.rawValue) · \(String(format: "%.1f", p.gramsPerKg))g/kg").tag(p)
                    }
                }
                .pickerStyle(.menu)
                .foregroundColor(AGColor.teal)
            }
            .agCard()
        }



        private var previewCard: some View {
            VStack(alignment: .leading, spacing: 12) {
                AGSectionHeader(title: "Computed daily targets",
                                subtitle: "Live preview ? finalises when you're done.")
                HStack {
                    statTile("BMR", "\(computed.bmr)", "kcal", tint: AGColor.teal)
                    statTile("TDEE", "\(computed.tdee)", "kcal", tint: AGColor.magenta)
                    statTile("Target", "\(computed.calories)", "kcal", tint: AGColor.amber)
                }
                HStack {
                    statTile("Protein", "\(computed.proteinG)", "g", tint: AGColor.teal)
                    statTile("Carbs",   "\(computed.carbsG)",   "g", tint: AGColor.amberHi)
                    statTile("Fat",     "\(computed.fatG)",     "g", tint: AGColor.magenta)
                }
            }
            .agCard()
        }

```

```
        private func statTile(_ label: String, _ value: String, _ unit: String, tint: Color) -> some View {
            VStack(spacing: 4) {
                Text(label.uppercased()).kerning(1.4).font(AGFont.hud(9))
                    .foregroundColor(AGColor.inkMuted)
                Text(value).font(AGFont.number(20)).foregroundColor(tint)
                Text(unit).font(AGFont.hud(9)).foregroundColor(AGColor.inkMuted)
            }
            .frame(maxWidth: .infinity)
            .padding(.vertical, 10)
            .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
            .overlay(RoundedRectangle(cornerRadius: 10).stroke(tint.opacity(0.35), lineWidth: 1))
        }



        private var navBar: some View {
            HStack(spacing: 12) {
                if stepIndex > 0 {
                    AGGhostButton(title: "Back", icon: "chevron.left", accent: AGColor.teal) {
                        withAnimation { stepIndex -= 1 }
                    }
                }
                Spacer()
                AGPushButton(
                    title: stepIndex == stepCount - 1 ? "Save targets" : "Next",
                    icon: stepIndex == stepCount - 1 ? "checkmark" : "chevron.right",
                    fill: AGColor.amber, textColor: .black
                ) {
                    if stepIndex == stepCount - 1 {
                        finish()
                    } else {
                        withAnimation { stepIndex += 1 }
                    }
                }
                .frame(width: 200)
            }
            .padding(16)
            .background(
                LinearGradient(colors: [AGColor.canvas.opacity(0), AGColor.canvas],
                               startPoint: .top, endPoint: .bottom)
            )
        }

        private func finish() {
            let targets = TargetsCalculator.compute(for: draft)
            TargetsCalculator.apply(targets, to: &draft)
            profileVM.profile = draft
            profileVM.saveProfile()
            targetsDone = true
            AGToastCenter.shared.show("Targets calibrated: \(targets.calories) kcal / day", kind: .success, celebrate: true)
            onDone()
        }

        private func fieldLabel(_ text: String) -> some View {
            Text(text.uppercased()).kerning(1.4).font(AGFont.hud(10))
```

```
                .foregroundColor(AGColor.inkMuted)
        }
    }

    private struct HUDTextFieldStyle: TextFieldStyle {
        func _body(configuration: TextField<Self._Label>) -> some View {
            configuration
                .font(AGFont.body(14))
                .foregroundColor(AGColor.ink)
                .padding(.horizontal, 10).padding(.vertical, 8)
                .background(RoundedRectangle(cornerRadius: 8).fill(AGColor.panelHi))
                .overlay(RoundedRectangle(cornerRadius: 8).stroke(AGColor.stroke, lineWidth: 1))
        }
    }

    #Preview {
        TargetsOnboardingView(onDone: {})
    }
```

ProfileView.swift

AetherGain/Views/Profile/ProfileView.swift

```swift
import SwiftUI
import PhotosUI

struct ProfileView: View {
    @EnvironmentObject var authVM: AuthViewModel
    @StateObject private var profileVM = ProfileViewModel()
    @AppStorage("ag.privacy.shareLeaderboard") private var appOptIn: Bool = false
    @AppStorage("ag.onboarding.targetsDone") private var targetsDone: Bool = false
    @State private var confirmSignOut = false
    @State private var showTargetsOnboarding = false
    @State private var pfpItem: PhotosPickerItem?
    @State private var isUploadingPfp = false

    var body: some View {
        ScrollView {
            VStack(spacing: 14) {
                identityCard
                targetsCard
                statsCard
                personalCard
                privacyCard
                actionsCard
            }
            .padding(16)
            .padding(.bottom, 40)
        }
        .background(AGColor.canvas.ignoresSafeArea())
        .overlay(AGGrid().opacity(0.35).ignoresSafeArea().allowsHitTesting(false))
        .navigationBarTitleDisplayMode(.inline)
        .toolbar {
            ToolbarItem(placement: .principal) {
                Text("PROFILE").kerning(2).font(AGFont.hud(12))
                    .foregroundColor(AGColor.amber)
            }
        }
        .sheet(isPresented: $showTargetsOnboarding) {
            TargetsOnboardingView(onDone: { showTargetsOnboarding = false })
                .agToastLayer()
        }
        .onAppear { authVM.userDidInteract() }
    }



    private var identityCard: some View {
```

```
            HStack(spacing: 14) {
                ZStack(alignment: .bottomTrailing) {
                    AGAvatar(urlString: profileVM.profile.profileImageURL,
                             initial: initial, size: 72, tint: AGColor.amber)
                    PhotosPicker(selection: $pfpItem, matching: .images) {
                        ZStack {
                            Circle().fill(AGColor.amber)
                                .frame(width: 24, height: 24)
                                .shadow(color: AGColor.amber.opacity(0.6), radius: 4)
                            if isUploadingPfp {
                                ProgressView().tint(.black).scaleEffect(0.5)
                            } else {
                                Image(systemName: "camera.fill").font(.system(size: 10, weight: .bold))
                                    .foregroundColor(.black)
                            }
                        }
                    }
                    .offset(x: 2, y: 2)
                }
                VStack(alignment: .leading, spacing: 4) {
                    Text(profileVM.profile.name.isEmpty ? "Operator" : profileVM.profile.name)
                        .font(AGFont.title(20)).foregroundColor(AGColor.ink)
                    Text("LVL \(GameEngine.level(for: profileVM.profile.xp)) · \(profileVM.profile.xp) XP")
                        .kerning(1.4).font(AGFont.hud(10))
                        .foregroundColor(AGColor.inkMuted)
                }
                Spacer()
            }
            .agCard()
            .onChange(of: pfpItem) { _, newItem in
                uploadPfp(newItem)
            }
        }

        private func uploadPfp(_ item: PhotosPickerItem?) {
            guard let item else { return }
            Task {
                isUploadingPfp = true
                defer { isUploadingPfp = false }
                do {
                    guard let data = try await item.loadTransferable(type: Data.self),
                          let uiImage = UIImage(data: data) else {
                        AGToastCenter.shared.show("Couldn't read that image.", kind: .error)
                        return
                    }
                    let url = try await ProfileImageService.upload(image: uiImage)
                    await MainActor.run {
                        profileVM.profile.profileImageURL = url.absoluteString
                        profileVM.saveProfile()
                        AGToastCenter.shared.show("Avatar updated.", kind: .success)
                    }
                } catch {
                    await MainActor.run {
                        AGToastCenter.shared.show("Upload failed: \(error.localizedDescription)", kind: .error)
                    }
```

```
                }
            }
        }

        private var initial: String {
            String(profileVM.profile.name.trimmingCharacters(in: .whitespaces).prefix(1)).uppercased()
                .ifEmpty("A")
        }



        private var targetsCard: some View {
            VStack(alignment: .leading, spacing: 10) {
                AGSectionHeader(title: "Daily targets",
                                subtitle: profileVM.profile.hasOnboardedTargets
                                            ? "From your calibration"
                                            : "Not calibrated yet")
                if profileVM.profile.hasOnboardedTargets {
                    HStack {
                        miniStat("BMR", "\(profileVM.profile.bmr)", "kcal", AGColor.teal)
                        miniStat("TDEE", "\(profileVM.profile.tdee)", "kcal", AGColor.magenta)
                        miniStat("Target", "\(profileVM.profile.calorieTarget)", "kcal", AGColor.amber)
                    }
                    HStack {
                        miniStat("Protein", "\(profileVM.profile.proteinTargetG)", "g", AGColor.teal)
                        miniStat("Carbs", "\(profileVM.profile.carbsTargetG)", "g", AGColor.amberHi)
                        miniStat("Fat", "\(profileVM.profile.fatTargetG)", "g", AGColor.magenta)
                    }
                    Text("GOAL: \(profileVM.profile.fitnessGoal.rawValue.uppercased()) · \(String(format: "%.2f", profileVM.profile.goalRateKgPerWeek))
        KG/WK · \(profileVM.profile.proteinPreference.rawValue.uppercased()) PROTEIN")
                        .kerning(1.2).font(AGFont.hud(10))
                        .foregroundColor(AGColor.inkMuted)
                        .padding(.top, 4)
                }
                AGGhostButton(title: profileVM.profile.hasOnboardedTargets ? "Recompute" : "Calibrate now",
                              icon: "arrow.triangle.2.circlepath",
                              accent: AGColor.amber) {
                    showTargetsOnboarding = true
                }
            }
            .agCard()
        }

        private func miniStat(_ label: String, _ value: String, _ unit: String, _ tint: Color) -> some View {
            VStack(spacing: 4) {
                Text(label.uppercased()).kerning(1.4).font(AGFont.hud(9))
                    .foregroundColor(AGColor.inkMuted)
                Text(value).font(AGFont.number(18)).foregroundColor(tint)
                Text(unit).font(AGFont.hud(9)).foregroundColor(AGColor.inkMuted)
            }
            .frame(maxWidth: .infinity)
            .padding(.vertical, 8)
            .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
            .overlay(RoundedRectangle(cornerRadius: 10).stroke(tint.opacity(0.3), lineWidth: 1))
        }
```

```
160
161
162
163         private var statsCard: some View {
164             HStack(spacing: 10) {
165                 statBox(icon: "star.fill", tint: AGColor.magenta, label: "XP", value: "\(profileVM.profile.xp)")
166                 statBox(icon: "flame.fill", tint: AGColor.amber, label: "Streak", value: "\(profileVM.profile.streakDays)")
167                 statBox(icon: "trophy.fill", tint: AGColor.teal, label: "Level", value: "\(GameEngine.level(for: profileVM.profile.xp))")
168             }
169         }
170
171         private func statBox(icon: String, tint: Color, label: String, value: String) -> some View {
172             VStack(spacing: 6) {
173                 Image(systemName: icon).foregroundColor(tint)
174                 Text(value).font(AGFont.number(20)).foregroundColor(AGColor.ink)
175                 Text(label.uppercased()).kerning(1.2).font(AGFont.hud(9))
176                     .foregroundColor(AGColor.inkMuted)
177             }
178             .frame(maxWidth: .infinity)
179             .padding(.vertical, 12)
180             .background(RoundedRectangle(cornerRadius: 12).fill(AGColor.panel))
181             .overlay(RoundedRectangle(cornerRadius: 12).stroke(tint.opacity(0.3), lineWidth: 1))
182         }
183
184
185
186         private var personalCard: some View {
187             VStack(alignment: .leading, spacing: 10) {
188                 AGSectionHeader(title: "Personal")
189                 hudField(icon: "person.fill", tint: AGColor.amber) {
190                     TextField("Name", text: $profileVM.profile.name)
191                 }
192                 HStack {
193                     Image(systemName: "calendar").foregroundColor(AGColor.teal).frame(width: 24)
194                     Stepper("Age: \(profileVM.profile.age)", value: $profileVM.profile.age, in: 10...100)
195                         .foregroundColor(AGColor.ink)
196                 }
197                 .padding(.horizontal, 12).padding(.vertical, 10)
198                 .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
199                 .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.stroke, lineWidth: 1))
200
201                 hudField(icon: "target", tint: AGColor.teal) {
202                     TextField("Goals (e.g. Build muscle)", text: $profileVM.profile.goals)
203                         .textInputAutocapitalization(.sentences)
204                 }
205                 hudField(icon: "leaf.fill", tint: AGColor.magenta) {
206                     TextField("Preferences (diet, workouts)", text: $profileVM.profile.preferences)
207                         .textInputAutocapitalization(.sentences)
208                 }
209             }
210             .agCard()
211         }
212
213         private func hudField<Content: View>(icon: String, tint: Color,
214                                              @ViewBuilder content: () -> Content) -> some View {
```

```
            HStack(spacing: 10) {
                Image(systemName: icon).foregroundColor(tint).frame(width: 24)
                content()
                    .font(AGFont.body(14))
                    .foregroundColor(AGColor.ink)
            }
            .padding(.horizontal, 12).padding(.vertical, 10)
            .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
            .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.stroke, lineWidth: 1))
        }



        private var privacyCard: some View {
            VStack(alignment: .leading, spacing: 8) {
                AGSectionHeader(title: "Privacy",
                                subtitle: "Encrypted at rest and in transit.")
                Toggle(isOn: leaderboardBinding) {
                    VStack(alignment: .leading, spacing: 2) {
                        Text("Share on leaderboard").font(AGFont.headline(14))
                            .foregroundColor(AGColor.ink)
                        Text("Name, XP, level, and badges are visible to others.")
                            .font(AGFont.body(11)).foregroundColor(AGColor.inkMuted)
                    }
                }
                .tint(AGColor.amber)
            }
            .agCard()
        }

        private var leaderboardBinding: Binding<Bool> {
            Binding(
                get: { profileVM.profile.sharesToLeaderboard ?? false },
                set: {
                    profileVM.profile.sharesToLeaderboard = $0
                    appOptIn = $0
                }
            )
        }



        private var actionsCard: some View {
            VStack(spacing: 10) {
                AGPushButton(title: "Save profile", icon: "checkmark",
                             fill: AGColor.amber, textColor: .black,
                             isLoading: profileVM.isLoading) {
                    profileVM.saveProfile()
                    authVM.userDidInteract()
                    DispatchQueue.main.asyncAfter(deadline: .now() + 0.4) {
                        if profileVM.errorMessage == nil {
                            AGToastCenter.shared.show("Profile saved.", kind: .success)
                        } else if let msg = profileVM.errorMessage {
                            AGToastCenter.shared.show(msg, kind: .error)
                        }
```

```
                    }
                }
                AGGhostButton(title: "Sign out", icon: "rectangle.portrait.and.arrow.right",
                              accent: AGColor.danger) {
                    confirmSignOut = true
                }
            }
            .alert("Sign out?", isPresented: $confirmSignOut) {
                Button("Cancel", role: .cancel) {}
                Button("Sign out", role: .destructive) {
                    authVM.signOut()
                    AGToastCenter.shared.show("Signed out.", kind: .info)
                }
            } message: {
                Text("Your data stays synced in the cloud.")
            }
        }
    }

    private extension String {
        func ifEmpty(_ fallback: String) -> String {
            isEmpty ? fallback : self
        }
    }

    #Preview {
        NavigationStack { ProfileView().environmentObject(AuthViewModel()) }
    }
```

ProgressView.swift

AetherGain/Views/Progress/ProgressView.swift

```swift


import SwiftUI
import Charts

struct AGProgressScreen: View {
    @EnvironmentObject var authVM: AuthViewModel
    @StateObject private var workoutVM = WorkoutViewModel()
    @StateObject private var mealVM = MealViewModel()
    @StateObject private var habitsVM = HabitsViewModel()
    @StateObject private var profileVM = ProfileViewModel()

    @State private var shareURL: URL?
    @State private var showShare = false

    private var stats: [DailyStat] {
        ProgressAnalytics.weeklyStats(
            workouts: workoutVM.history,
            meals: mealVM.meals,
            profile: habitsVM.profile
        )
    }
    private var heatmap: [ProgressAnalytics.HeatmapCell] {
        ProgressAnalytics.heatmap(workouts: workoutVM.history)
    }

    private var totalXPThisWeek: Int { stats.reduce(0) { $0 + $1.xp } }
    private var totalWorkouts: Int { stats.reduce(0) { $0 + $1.workouts } }
    private var avgCalories: Double {
        let logged = stats.filter { $0.calories > 0 }
        guard !logged.isEmpty else { return 0 }
        return logged.reduce(0) { $0 + $1.calories } / Double(logged.count)
    }

    var body: some View {
        ScrollView {
            VStack(alignment: .leading, spacing: 14) {
                summaryCard
                heatmapCard
                xpChartCard
                workoutsChartCard
                macrosChartCard
                habitsRingsCard
                targetsCard
                exportCard
```

```
                }
                .padding(16)
                .padding(.bottom, 40)
            }
            .background(AGColor.canvas.ignoresSafeArea())
            .overlay(AGGrid().opacity(0.35).ignoresSafeArea().allowsHitTesting(false))
            .navigationBarTitleDisplayMode(.inline)
            .toolbar {
                ToolbarItem(placement: .principal) {
                    Text("PROGRESS").kerning(2).font(AGFont.hud(12))
                        .foregroundColor(AGColor.amber)
                }
            }
            .sheet(isPresented: $showShare) {
                if let url = shareURL { ShareSheet(items: [url]) }
            }
            .onAppear { authVM.userDidInteract() }
        }



        private var summaryCard: some View {
            VStack(alignment: .leading, spacing: 12) {
                HStack {
                    VStack(alignment: .leading, spacing: 2) {
                        Text("LAST 7 DAYS").kerning(1.6).font(AGFont.hud(10))
                            .foregroundColor(AGColor.inkMuted)
                        Text("Your trend").font(AGFont.title(22))
                            .foregroundColor(AGColor.ink)
                    }
                    Spacer()
                    AGStreakBadge(days: habitsVM.profile.streakDays)
                }
                HStack(spacing: 8) {
                    AGStatPill(icon: "star.fill", title: "XP", value: "\(totalXPThisWeek)", tint: AGColor.magenta)
                    AGStatPill(icon: "dumbbell.fill", title: "Workouts", value: "\(totalWorkouts)", tint: AGColor.amber)
                    AGStatPill(icon: "flame.fill", title: "Avg kcal", value: "\(Int(avgCalories))", tint: AGColor.teal)
                }
            }
            .agCard()
        }

        private var heatmapCard: some View {
            VStack(alignment: .leading, spacing: 10) {
                AGSectionHeader(title: "Activity heatmap",
                                subtitle: "12 weeks · XP intensity per day")
                HeatmapGrid(cells: heatmap, columns: 12, rows: 7)
                    .frame(height: 130)
                HStack(spacing: 4) {
                    Text("LOW").font(AGFont.hud(9)).foregroundColor(AGColor.inkMuted)
                    ForEach(0..<5) { i in
                        RoundedRectangle(cornerRadius: 3)
                            .fill(AGColor.amber.opacity(0.1 + Double(i) * 0.2))
                            .frame(width: 14, height: 8)
                    }
```

```
                    Text("HIGH").font(AGFont.hud(9)).foregroundColor(AGColor.inkMuted)
                }
            }
            .agCard()
        }

        private var xpChartCard: some View {
            VStack(alignment: .leading, spacing: 12) {
                AGSectionHeader(title: "XP earned",
                                subtitle: "From workouts each day")
                Chart(stats) { s in
                    BarMark(x: .value("Day", s.dayLabel),
                            y: .value("XP", s.xp))
                        .foregroundStyle(LinearGradient(colors: [AGColor.amberHi, AGColor.amber],
                                                        startPoint: .top, endPoint: .bottom))
                        .cornerRadius(6)
                }
                .frame(height: 160)
            }
            .agCard()
        }

        private var workoutsChartCard: some View {
            VStack(alignment: .leading, spacing: 12) {
                AGSectionHeader(title: "Workouts",
                                subtitle: "Sessions per day")
                Chart(stats) { s in
                    BarMark(x: .value("Day", s.dayLabel),
                            y: .value("Count", s.workouts))
                        .foregroundStyle(AGColor.teal)
                        .cornerRadius(4)
                }
                .frame(height: 130)
            }
            .agCard()
        }

        private var macrosChartCard: some View {
            VStack(alignment: .leading, spacing: 12) {
                AGSectionHeader(title: "Macros trend")
                Chart {
                    ForEach(stats) { s in
                        LineMark(x: .value("Day", s.dayLabel), y: .value("Protein", s.protein))
                            .foregroundStyle(by: .value("Macro", "Protein"))
                            .symbol(by: .value("Macro", "Protein"))
                        LineMark(x: .value("Day", s.dayLabel), y: .value("Carbs", s.carbs))
                            .foregroundStyle(by: .value("Macro", "Carbs"))
                            .symbol(by: .value("Macro", "Carbs"))
                        LineMark(x: .value("Day", s.dayLabel), y: .value("Fat", s.fat))
                            .foregroundStyle(by: .value("Macro", "Fat"))
                            .symbol(by: .value("Macro", "Fat"))
                    }
                }
                .chartForegroundStyleScale([
                    "Protein": AGColor.teal,
```

```
                    "Carbs":   AGColor.amberHi,
                    "Fat":     AGColor.magenta
                ])
                .frame(height: 180)
            }
            .agCard()
        }

        private var habitsRingsCard: some View {
            VStack(alignment: .leading, spacing: 10) {
                AGSectionHeader(title: "Today's habits")
                HStack(spacing: 12) {
                    AGProgressRing(progress: min(1, habitsVM.profile.dailyWaterML / max(1, habitsVM.profile.waterMLTarget)),
                                   color: AGColor.teal, size: 82,
                                   label: "\(Int(habitsVM.profile.dailyWaterML))",
                                   sublabel: "ML")
                    AGProgressRing(progress: min(1, habitsVM.profile.sleepHours / max(1, habitsVM.profile.sleepHoursTarget)),
                                   color: AGColor.magenta, size: 82,
                                   label: String(format: "%.1f", habitsVM.profile.sleepHours),
                                   sublabel: "H")
                    AGProgressRing(progress: habitsVM.profile.customHabitCompletedToday ? 1 : 0,
                                   color: AGColor.amber, size: 82,
                                   label: habitsVM.profile.customHabitCompletedToday ? "?" : "?",
                                   sublabel: habitsVM.profile.customHabitName)
                }
            }
            .agCard()
        }

        private var targetsCard: some View {
            VStack(alignment: .leading, spacing: 8) {
                AGSectionHeader(title: "Daily targets",
                                subtitle: "From your BMR / TDEE calibration")
                HStack {
                    miniStat("BMR", "\(profileVM.profile.bmr)", "kcal", AGColor.teal)
                    miniStat("TDEE", "\(profileVM.profile.tdee)", "kcal", AGColor.magenta)
                    miniStat("Target", "\(profileVM.profile.calorieTarget)", "kcal", AGColor.amber)
                }
                HStack {
                    miniStat("Protein", "\(profileVM.profile.proteinTargetG)", "g", AGColor.teal)
                    miniStat("Carbs", "\(profileVM.profile.carbsTargetG)", "g", AGColor.amberHi)
                    miniStat("Fat", "\(profileVM.profile.fatTargetG)", "g", AGColor.magenta)
                }
            }
            .agCard()
        }

        private func miniStat(_ label: String, _ value: String, _ unit: String, _ tint: Color) -> some View {
            VStack(spacing: 4) {
                Text(label.uppercased()).kerning(1.4).font(AGFont.hud(9))
                    .foregroundColor(AGColor.inkMuted)
                Text(value).font(AGFont.number(18)).foregroundColor(tint)
                Text(unit).font(AGFont.hud(9)).foregroundColor(AGColor.inkMuted)
            }
            .frame(maxWidth: .infinity)
```

```
            .padding(.vertical, 8)
            .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
            .overlay(RoundedRectangle(cornerRadius: 10).stroke(tint.opacity(0.3), lineWidth: 1))
        }

        private var exportCard: some View {
            VStack(alignment: .leading, spacing: 12) {
                AGSectionHeader(title: "Export report",
                                subtitle: "Saved to Files ? On My iPhone ? AetherGain. Also opens Share sheet.")
                AGPushButton(title: "Export CSV", icon: "square.and.arrow.up",
                             fill: AGColor.teal, textColor: .black) {
                    let csv = ProgressAnalytics.csv(from: stats)
                    if let url = ProgressAnalytics.writeCSVToTemp(csv),
                       FileManager.default.fileExists(atPath: url.path) {


                        shareURL = url
                        DispatchQueue.main.asyncAfter(deadline: .now() + 0.05) {
                            showShare = true
                        }
                        AGToastCenter.shared.show("Report saved: \(url.lastPathComponent)", kind: .success)
                    } else {
                        AGToastCenter.shared.show("Couldn't create export file.", kind: .error)
                    }
                }
                if let url = shareURL {
                    Text("LAST EXPORT: \(url.lastPathComponent)").kerning(1)
                        .font(AGFont.hud(9)).foregroundColor(AGColor.inkMuted)
                }
            }
            .agCard()
        }
    }



    private struct HeatmapGrid: View {
        let cells: [ProgressAnalytics.HeatmapCell]
        let columns: Int
        let rows: Int

        var body: some View {
            GeometryReader { geo in
                let spacing: CGFloat = 3
                let w = (geo.size.width - CGFloat(columns - 1) * spacing) / CGFloat(columns)
                let h = (geo.size.height - CGFloat(rows - 1) * spacing) / CGFloat(rows)


                HStack(spacing: spacing) {
                    ForEach(0..<columns, id: \.self) { c in
                        VStack(spacing: spacing) {
                            ForEach(0..<rows, id: \.self) { r in
                                let idx = c * rows + r
                                if idx < cells.count {
                                    RoundedRectangle(cornerRadius: 3)
```

```
                                    .fill(color(for: cells[idx].intensity))
                                    .frame(width: w, height: h)
                            } else {
                                Color.clear.frame(width: w, height: h)
                            }
                        }
                    }
                }
            }
        }

        private func color(for intensity: Double) -> Color {
            if intensity <= 0 { return AGColor.panelHi }
            return AGColor.amber.opacity(0.15 + intensity * 0.85)
        }
    }



    struct ShareSheet: UIViewControllerRepresentable {
        let items: [Any]
        func makeUIViewController(context: Context) -> UIActivityViewController {
            UIActivityViewController(activityItems: items, applicationActivities: nil)
        }
        func updateUIViewController(_ vc: UIActivityViewController, context: Context) {}
    }

    #Preview {
        NavigationStack { AGProgressScreen().environmentObject(AuthViewModel()) }
    }
```

LogWorkoutView.swift

AetherGain/Views/Workouts/LogWorkoutView.swift

```swift


import SwiftUI

struct LogWorkoutView: View {
    @EnvironmentObject var authVM: AuthViewModel
    @StateObject private var workoutVM = WorkoutViewModel()
    @State private var showCustomExerciseSheet = false
    @State private var customExerciseName = ""

    private var estimatedXP: Int { GameEngine.workoutXP(for: workoutVM.currentSession) }

    var body: some View {
        ScrollView {
            VStack(spacing: 14) {
                xpHeaderCard
                dateCard

                ForEach(workoutVM.currentSession.exercises.indices, id: \.self) { index in
                    exerciseCard(index: index)
                }

                addExerciseCard
                completeCard
            }
            .padding(16)
            .padding(.bottom, 40)
        }
        .background(AGColor.canvas.ignoresSafeArea())
        .sheet(isPresented: $showCustomExerciseSheet) { customExerciseSheet }
        .onAppear {
            if workoutVM.currentSession.exercises.isEmpty {
                workoutVM.addExercise()
            }
            authVM.userDidInteract()
        }
    }



    private var xpHeaderCard: some View {
        HStack(spacing: 14) {
            ZStack {
                RoundedRectangle(cornerRadius: 16, style: .continuous)
                    .fill(LinearGradient(colors: [AGColor.coral, AGColor.peach],
                                         startPoint: .topLeading, endPoint: .bottomTrailing))
                    .frame(width: 62, height: 62)
```

```
                    Image(systemName: "bolt.fill").foregroundColor(.white).font(.system(size: 28, weight: .bold))
                }
                VStack(alignment: .leading, spacing: 4) {
                    Text("Estimated XP").font(AGFont.caption(12)).foregroundColor(AGColor.inkMuted)
                    Text("+\(estimatedXP)").font(AGFont.number(28)).foregroundColor(AGColor.ink)
                    Text("Base 50 + exercise bonuses")
                        .font(AGFont.caption(11)).foregroundColor(AGColor.inkMuted)
                }
                Spacer()
            }
            .agCard()
        }

        private var dateCard: some View {
            HStack(spacing: 12) {
                Image(systemName: "calendar")
                    .foregroundColor(AGColor.teal)
                    .font(.system(size: 16, weight: .bold))
                VStack(alignment: .leading, spacing: 2) {
                    Text("SESSION DATE").kerning(1.4).font(AGFont.hud(9))
                        .foregroundColor(AGColor.inkMuted)
                    Text("Backdate to log a past workout.")
                        .font(AGFont.body(11)).foregroundColor(AGColor.inkMuted)
                }
                Spacer()
                DatePicker(
                    "",
                    selection: Binding(
                        get: { workoutVM.currentSession.date },
                        set: { workoutVM.currentSession.date = $0 }
                    ),
                    in: ...Date(),
                    displayedComponents: [.date, .hourAndMinute]
                )
                .labelsHidden()
                .datePickerStyle(.compact)
                .tint(AGColor.amber)
            }
            .agCard()
        }

        private func exerciseCard(index: Int) -> some View {

            let category: ExerciseCategory = {
                guard workoutVM.currentSession.exercises.indices.contains(index) else { return .strength }
                return workoutVM.currentSession.exercises[index].category ?? .strength
            }()
            return VStack(alignment: .leading, spacing: 12) {
                HStack {
                    Text("Exercise \(index + 1)").font(AGFont.headline(15))
                    Spacer()
                    Button(role: .destructive) {
                        guard workoutVM.currentSession.exercises.indices.contains(index) else { return }
                        workoutVM.removeExercise(at: index)
                        AGHaptics.tap()
```

```
                    } label: {
                        Image(systemName: "xmark.circle.fill")
                            .foregroundColor(AGColor.inkMuted)
                    }
                    .buttonStyle(.plain)
                }

                HStack(spacing: 8) {
                    TextField("Exercise name", text: safeText(index: index))
                        .textInputAutocapitalization(.words)
                        .autocorrectionDisabled()
                        .font(AGFont.body(15))
                        .foregroundColor(AGColor.ink)
                        .padding(.horizontal, 12).padding(.vertical, 10)
                        .background(RoundedRectangle(cornerRadius: 12).fill(AGColor.canvas))
                        .overlay(RoundedRectangle(cornerRadius: 12).stroke(AGColor.stroke, lineWidth: 1))
                        .onChange(of: workoutVM.currentSession.exercises.indices.contains(index)
                                  ? workoutVM.currentSession.exercises[index].type : "") { _, _ in
                            authVM.userDidInteract()
                        }

                    Menu {
                        ForEach(ExerciseTemplate.catalog) { template in
                            Button(template.name) {
                                workoutVM.applyTemplate(template, at: index)
                            }
                        }
                    } label: {
                        HStack(spacing: 4) {
                            Image(systemName: "list.bullet")
                            Text("TEMPLATES").kerning(1.4)
                        }
                        .font(AGFont.hud(10))
                        .foregroundColor(AGColor.teal)
                        .padding(.horizontal, 10).padding(.vertical, 10)
                        .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.teal.opacity(0.10)))
                        .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.teal.opacity(0.35), lineWidth: 1))
                    }
                }


                categoryPicker(index: index, currentCategory: category)

                HStack(spacing: 8) {
                    Spacer()
                    Text("+\(safeExerciseXP(index: index)) XP")
                        .font(AGFont.caption(12))
                        .foregroundColor(AGColor.success)
                        .padding(.horizontal, 10).padding(.vertical, 4)
                        .background(Capsule().fill(AGColor.success.opacity(0.15)))
                }

                switch category {
                case .strength: strengthFields(for: index)
                case .cardio:   cardioFields(for: index)
```

```
                case .mobility: mobilityFields(for: index)
                }

                TextField("Notes (optional)", text: safeNotes(index: index))
                    .textInputAutocapitalization(.sentences)
                    .padding(.horizontal, 12).padding(.vertical, 10)
                    .background(RoundedRectangle(cornerRadius: 12).fill(AGColor.canvas))
                    .overlay(RoundedRectangle(cornerRadius: 12).stroke(AGColor.stroke, lineWidth: 1))
            }
            .agCard()
        }



        private func categoryPicker(index: Int, currentCategory: ExerciseCategory) -> some View {
            HStack(spacing: 6) {
                ForEach(ExerciseCategory.allCases) { c in
                    Button {
                        guard workoutVM.currentSession.exercises.indices.contains(index) else { return }
                        workoutVM.currentSession.exercises[index].category = c
                            AGHaptics.tap()
                    } label: {
                        HStack(spacing: 4) {
                            Image(systemName: iconFor(c))
                            Text(c.displayName.uppercased()).kerning(1.2)
                        }
                        .font(AGFont.hud(10))
                        .foregroundColor(c == currentCategory ? .black : colorFor(c))
                        .padding(.horizontal, 10).padding(.vertical, 6)
                        .background(
                            RoundedRectangle(cornerRadius: 8)
                                .fill(c == currentCategory ? colorFor(c) : colorFor(c).opacity(0.08))
                        )
                        .overlay(
                            RoundedRectangle(cornerRadius: 8)
                                .stroke(colorFor(c).opacity(0.5), lineWidth: 1)
                        )
                    }
                    .buttonStyle(.plain)
                }
                Spacer()
            }
        }

        private func colorFor(_ c: ExerciseCategory) -> Color {
            switch c {
            case .strength: return AGColor.amber
            case .cardio:   return AGColor.teal
            case .mobility: return AGColor.magenta
            }
        }

        private func iconFor(_ c: ExerciseCategory) -> String {
            switch c {
            case .strength: return "dumbbell.fill"
```

```
            case .cardio:   return "figure.run"
            case .mobility: return "figure.mind.and.body"
            }
        }



        private func safeText(index: Int) -> Binding<String> {
            Binding(
                get: {
                    guard workoutVM.currentSession.exercises.indices.contains(index) else { return "" }
                    return workoutVM.currentSession.exercises[index].type
                },
                set: {
                    guard workoutVM.currentSession.exercises.indices.contains(index) else { return }
                    workoutVM.currentSession.exercises[index].type = $0
                }
            )
        }

        private func safeNotes(index: Int) -> Binding<String> {
            Binding(
                get: {
                    guard workoutVM.currentSession.exercises.indices.contains(index) else { return "" }
                    return workoutVM.currentSession.exercises[index].notes
                },
                set: {
                    guard workoutVM.currentSession.exercises.indices.contains(index) else { return }
                    workoutVM.currentSession.exercises[index].notes = $0
                }
            )
        }

        private func safeInt(index: Int, keyPath: WritableKeyPath<Exercise, Int>) -> Binding<Int> {
            Binding(
                get: {
                    guard workoutVM.currentSession.exercises.indices.contains(index) else { return 0 }
                    return workoutVM.currentSession.exercises[index][keyPath: keyPath]
                },
                set: {
                    guard workoutVM.currentSession.exercises.indices.contains(index) else { return }
                    workoutVM.currentSession.exercises[index][keyPath: keyPath] = $0
                }
            )
        }

        private func safeDouble(index: Int, keyPath: WritableKeyPath<Exercise, Double>) -> Binding<Double> {
            Binding(
                get: {
                    guard workoutVM.currentSession.exercises.indices.contains(index) else { return 0 }
                    return workoutVM.currentSession.exercises[index][keyPath: keyPath]
                },
                set: {
                    guard workoutVM.currentSession.exercises.indices.contains(index) else { return }
                    workoutVM.currentSession.exercises[index][keyPath: keyPath] = $0
```

```
                }
            )
        }

        private func safeOptionalDouble(index: Int, keyPath: WritableKeyPath<Exercise, Double?>) -> Binding<Double> {
            Binding(
                get: {
                    guard workoutVM.currentSession.exercises.indices.contains(index) else { return 0 }
                    return workoutVM.currentSession.exercises[index][keyPath: keyPath] ?? 0
                },
                set: {
                    guard workoutVM.currentSession.exercises.indices.contains(index) else { return }
                    workoutVM.currentSession.exercises[index][keyPath: keyPath] = $0
                }
            )
        }

        private func safeExerciseXP(index: Int) -> Int {
            guard workoutVM.currentSession.exercises.indices.contains(index) else { return 0 }
            return GameEngine.exerciseXP(for: workoutVM.currentSession.exercises[index])
        }

        private func safeSets(index: Int) -> Int {
            guard workoutVM.currentSession.exercises.indices.contains(index) else { return 0 }
            return workoutVM.currentSession.exercises[index].sets
        }

        private func safeReps(index: Int) -> Int {
            guard workoutVM.currentSession.exercises.indices.contains(index) else { return 0 }
            return workoutVM.currentSession.exercises[index].reps
        }



        private func strengthFields(for index: Int) -> some View {
            VStack(spacing: 10) {
                Stepper("Sets: \(safeSets(index: index))",
                        value: safeInt(index: index, keyPath: \.sets), in: 1...20)
                Stepper("Reps: \(safeReps(index: index))",
                        value: safeInt(index: index, keyPath: \.reps), in: 1...50)
                numberRow(label: "Weight", unit: "kg",
                          value: safeDouble(index: index, keyPath: \.weightKG))
            }
        }

        private func cardioFields(for index: Int) -> some View {
            VStack(spacing: 10) {
                numberRow(label: "Distance", unit: "km",
                          value: safeOptionalDouble(index: index, keyPath: \.distanceKM))
                numberRow(label: "Time", unit: "min",
                          value: safeOptionalDouble(index: index, keyPath: \.durationMinutes))
                numberRow(label: "Incline", unit: "%",
                          value: safeOptionalDouble(index: index, keyPath: \.inclinePercent))
            }
        }
```

```

        private func mobilityFields(for index: Int) -> some View {
            numberRow(label: "Time", unit: "min",
                      value: safeOptionalDouble(index: index, keyPath: \.durationMinutes))
        }

        private func numberRow(label: String, unit: String, value: Binding<Double>) -> some View {
            HStack {
                Text(label).font(AGFont.body(14)).foregroundColor(AGColor.ink)
                Spacer()
                TextField(unit, value: value, format: .number)
                    .keyboardType(.decimalPad)
                    .multilineTextAlignment(.trailing)
                    .frame(width: 90)
                    .padding(.horizontal, 10).padding(.vertical, 6)
                    .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.canvas))
                    .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.stroke, lineWidth: 1))
                Text(unit).font(AGFont.caption(12)).foregroundColor(AGColor.inkMuted)
            }
        }

        private var addExerciseCard: some View {
            HStack(spacing: 12) {
                AGGhostButton(title: "Add exercise", icon: "plus", accent: AGColor.coral) {
                    workoutVM.addExercise()
                    authVM.userDidInteract()
                }
                AGGhostButton(title: "Custom", icon: "square.and.pencil", accent: AGColor.lavender) {
                    showCustomExerciseSheet = true
                }
                Spacer()
            }
        }

        private var completeCard: some View {
            VStack(spacing: 10) {
                if let error = workoutVM.errorMessage {
                    HStack(spacing: 6) {
                        Image(systemName: "exclamationmark.triangle.fill")
                        Text(error).font(AGFont.caption(12))
                    }
                    .foregroundColor(AGColor.danger)
                    .frame(maxWidth: .infinity, alignment: .leading)
                }
                AGPushButton(
                    title: "Complete workout · +\(estimatedXP) XP",
                    icon: "checkmark",
                    fill: AGColor.success,
                    isEnabled: workoutVM.currentSession.isValidForXP,
                    isLoading: workoutVM.isLoading
                ) {
                    let xpForToast = estimatedXP
                    workoutVM.completeSession()
                    authVM.userDidInteract()

```

```
381
382
383                 if let msg = workoutVM.errorMessage {
384                     AGToastCenter.shared.show(msg, kind: .error)
385                 } else {
386                     AGToastCenter.shared.show("Workout logged! +\(xpForToast) XP earned",
387                                               kind: .success, celebrate: true)
388                 }
389             }
390         }
391     }
392
393
394
395     private var customExerciseSheet: some View {
396         NavigationStack {
397             AGScreen {
398                 VStack(alignment: .leading, spacing: 14) {
399                     VStack(alignment: .leading, spacing: 8) {
400                         Text("Exercise name").font(AGFont.headline(15))
401                         TextField("e.g. Kettlebell swing", text: $customExerciseName)
402                             .textInputAutocapitalization(.words)
403                             .padding(.horizontal, 12).padding(.vertical, 10)
404                             .background(RoundedRectangle(cornerRadius: 12).fill(AGColor.canvas))
405                             .overlay(RoundedRectangle(cornerRadius: 12).stroke(AGColor.stroke, lineWidth: 1))
406                     }
407                     .agCard()
408
409                     Text("Names like Running, Cycling, Swimming, Yoga, or Stretching auto-classify. Others default to strength.")
410                         .font(AGFont.caption(11)).foregroundColor(AGColor.inkMuted)
411
412                     Spacer()
413                 }
414                 .padding(16)
415             }
416             .navigationTitle("New exercise")
417             .navigationBarTitleDisplayMode(.inline)
418             .toolbar {
419                 ToolbarItem(placement: .cancellationAction) {
420                     Button("Cancel") {
421                         customExerciseName = ""
422                         showCustomExerciseSheet = false
423                     }
424                 }
425                 ToolbarItem(placement: .confirmationAction) {
426                     Button("Add") {
427                         workoutVM.addCustomExercise(named: customExerciseName)
428                         customExerciseName = ""
429                         showCustomExerciseSheet = false
430                     }
431                     .disabled(customExerciseName.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty)
432                 }
433             }
434         }
435     }
```

```
436     }
437
438     #Preview {
439         LogWorkoutView().environmentObject(AuthViewModel())
440     }
```

WorkoutEditSheet.swift

AetherGain/Views/Workouts/WorkoutEditSheet.swift

```swift
import SwiftUI

struct WorkoutEditSheet: View {
    @Environment(\.dismiss) private var dismiss
    @ObservedObject var workoutVM: WorkoutViewModel


    @State private var draft: WorkoutSession
    @State private var isSaving = false
    @State private var errorMessage: String?

    init(session: WorkoutSession, workoutVM: WorkoutViewModel) {
        _draft = State(initialValue: session)
        self.workoutVM = workoutVM
    }

    private var estimatedXP: Int { GameEngine.workoutXP(for: draft) }
    private var xpDelta: Int { estimatedXP - draft.totalXP }

    var body: some View {
        NavigationStack {
            ScrollView {
                VStack(spacing: 14) {
                    headerCard
                    ForEach(draft.exercises.indices, id: \.self) { index in
                        if draft.exercises.indices.contains(index) {
                            exerciseCard(index: index)
                        }
                    }
                    addExerciseRow
                    if let msg = errorMessage {
                        HStack(spacing: 6) {
                            Image(systemName: "exclamationmark.triangle.fill")
                            Text(msg).font(AGFont.body(12))
                        }
                        .foregroundColor(AGColor.danger)
                    }
                    AGPushButton(
                        title: "Save changes",
```

```
                            icon: "checkmark",
                            fill: AGColor.amber, textColor: .black,
                            isEnabled: draft.isValidForXP,
                            isLoading: isSaving
                        ) {
                            save()
                        }
                    }
                    .padding(16)
                    .padding(.bottom, 40)
                }
                .background(AGColor.canvas.ignoresSafeArea())
                .navigationBarTitleDisplayMode(.inline)
                .toolbar {
                    ToolbarItem(placement: .principal) {
                        Text("EDIT WORKOUT").kerning(2).font(AGFont.hud(12))
                            .foregroundColor(AGColor.amber)
                    }
                    ToolbarItem(placement: .cancellationAction) {
                        Button("Close") { dismiss() }
                            .foregroundColor(AGColor.teal)
                    }
                }
            }
        }



        private var headerCard: some View {
            VStack(alignment: .leading, spacing: 8) {
                AGSectionHeader(title: "Session summary",
                                subtitle: "XP will adjust by the delta below.")
                HStack {
                    VStack(alignment: .leading, spacing: 2) {
                        Text("ORIGINAL XP").kerning(1.2).font(AGFont.hud(9))
                            .foregroundColor(AGColor.inkMuted)
                        Text("\(draft.totalXP)").font(AGFont.number(20))
                            .foregroundColor(AGColor.inkMuted)
                    }
                    Spacer()
                    VStack(alignment: .center, spacing: 2) {
                        Text("NEW XP").kerning(1.2).font(AGFont.hud(9))
                            .foregroundColor(AGColor.amber)
                        Text("\(estimatedXP)").font(AGFont.number(24))
                            .foregroundColor(AGColor.amber)
                    }
                    Spacer()
                    VStack(alignment: .trailing, spacing: 2) {
                        Text("DELTA").kerning(1.2).font(AGFont.hud(9))
                            .foregroundColor(AGColor.inkMuted)
                        Text(xpDelta >= 0 ? "+\(xpDelta)" : "\(xpDelta)")
                            .font(AGFont.number(20))
                            .foregroundColor(xpDelta >= 0 ? AGColor.teal : AGColor.danger)
                    }
                }
```

```
            }
            .agCard()
        }



        private func exerciseCard(index: Int) -> some View {
            let cat = draft.exercises[index].category ?? .strength
            return VStack(alignment: .leading, spacing: 12) {
                HStack {
                    Text("Exercise \(index + 1)").font(AGFont.headline(15))
                        .foregroundColor(AGColor.ink)
                    Spacer()
                    Button(role: .destructive) {
                        guard draft.exercises.indices.contains(index) else { return }
                        draft.exercises.remove(at: index)
                        AGHaptics.tap()
                    } label: {
                        Image(systemName: "xmark.circle.fill")
                            .foregroundColor(AGColor.inkMuted)
                    }
                    .buttonStyle(.plain)
                }

                TextField("Exercise name", text: safeText(index: index))
                    .textInputAutocapitalization(.words)
                    .autocorrectionDisabled()
                    .font(AGFont.body(15))
                    .foregroundColor(AGColor.ink)
                    .padding(.horizontal, 12).padding(.vertical, 10)
                    .background(RoundedRectangle(cornerRadius: 12).fill(AGColor.canvas))
                    .overlay(RoundedRectangle(cornerRadius: 12).stroke(AGColor.stroke, lineWidth: 1))

                categoryPicker(index: index, currentCategory: cat)

                switch cat {
                case .strength: strengthFields(index: index)
                case .cardio:   cardioFields(index: index)
                case .mobility: mobilityFields(index: index)
                }

                TextField("Notes (optional)", text: safeNotes(index: index))
                    .padding(.horizontal, 12).padding(.vertical, 10)
                    .background(RoundedRectangle(cornerRadius: 12).fill(AGColor.canvas))
                    .overlay(RoundedRectangle(cornerRadius: 12).stroke(AGColor.stroke, lineWidth: 1))
            }
            .agCard()
        }



        private func categoryPicker(index: Int, currentCategory: ExerciseCategory) -> some View {
            HStack(spacing: 6) {
                ForEach(ExerciseCategory.allCases) { c in
                    Button {
```

```
                        guard draft.exercises.indices.contains(index) else { return }
                        draft.exercises[index].category = c
                        AGHaptics.tap()
                    } label: {
                        HStack(spacing: 4) {
                            Image(systemName: iconFor(c))
                            Text(c.displayName.uppercased()).kerning(1.2)
                        }
                        .font(AGFont.hud(10))
                        .foregroundColor(c == currentCategory ? .black : colorFor(c))
                        .padding(.horizontal, 10).padding(.vertical, 6)
                        .background(
                            RoundedRectangle(cornerRadius: 8)
                                .fill(c == currentCategory ? colorFor(c) : colorFor(c).opacity(0.08))
                        )
                        .overlay(
                            RoundedRectangle(cornerRadius: 8)
                                .stroke(colorFor(c).opacity(0.5), lineWidth: 1)
                        )
                    }
                    .buttonStyle(.plain)
                }
                Spacer()
            }
        }



        private func strengthFields(index: Int) -> some View {
            VStack(spacing: 10) {
                Stepper("Sets: \(safeInt(index: index, keyPath: \.sets).wrappedValue)",
                        value: safeInt(index: index, keyPath: \.sets), in: 1...20)
                Stepper("Reps: \(safeInt(index: index, keyPath: \.reps).wrappedValue)",
                        value: safeInt(index: index, keyPath: \.reps), in: 1...50)
                numberRow(label: "Weight", unit: "kg",
                          value: safeDouble(index: index, keyPath: \.weightKG))
            }
        }

        private func cardioFields(index: Int) -> some View {
            VStack(spacing: 10) {
                numberRow(label: "Distance", unit: "km",
                          value: safeOptionalDouble(index: index, keyPath: \.distanceKM))
                numberRow(label: "Time", unit: "min",
                          value: safeOptionalDouble(index: index, keyPath: \.durationMinutes))
                numberRow(label: "Incline", unit: "%",
                          value: safeOptionalDouble(index: index, keyPath: \.inclinePercent))
            }
        }

        private func mobilityFields(index: Int) -> some View {
            numberRow(label: "Time", unit: "min",
                      value: safeOptionalDouble(index: index, keyPath: \.durationMinutes))
        }

```

```
        private func numberRow(label: String, unit: String, value: Binding<Double>) -> some View {
            HStack {
                Text(label).font(AGFont.body(14)).foregroundColor(AGColor.ink)
                Spacer()
                TextField(unit, value: value, format: .number)
                    .keyboardType(.decimalPad)
                    .multilineTextAlignment(.trailing)
                    .frame(width: 90)
                    .padding(.horizontal, 10).padding(.vertical, 6)
                    .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.canvas))
                    .overlay(RoundedRectangle(cornerRadius: 10).stroke(AGColor.stroke, lineWidth: 1))
                Text(unit).font(AGFont.caption(12)).foregroundColor(AGColor.inkMuted)
            }
        }

        private var addExerciseRow: some View {
            HStack(spacing: 12) {
                AGGhostButton(title: "Add exercise", icon: "plus", accent: AGColor.amber) {
                    draft.exercises.append(ExerciseTemplate.configuredExercise(named: "Bench Press"))
                }
                Spacer()
            }
        }



        private func safeText(index: Int) -> Binding<String> {
            Binding(
                get: { draft.exercises.indices.contains(index) ? draft.exercises[index].type : "" },
                set: {
                    guard draft.exercises.indices.contains(index) else { return }
                    draft.exercises[index].type = $0
                }
            )
        }

        private func safeNotes(index: Int) -> Binding<String> {
            Binding(
                get: { draft.exercises.indices.contains(index) ? draft.exercises[index].notes : "" },
                set: {
                    guard draft.exercises.indices.contains(index) else { return }
                    draft.exercises[index].notes = $0
                }
            )
        }

        private func safeInt(index: Int, keyPath: WritableKeyPath<Exercise, Int>) -> Binding<Int> {
            Binding(
                get: { draft.exercises.indices.contains(index) ? draft.exercises[index][keyPath: keyPath] : 0 },
                set: {
                    guard draft.exercises.indices.contains(index) else { return }
                    draft.exercises[index][keyPath: keyPath] = $0
                }
            )
        }
```

```
271
272         private func safeDouble(index: Int, keyPath: WritableKeyPath<Exercise, Double>) -> Binding<Double> {
273             Binding(
274                 get: { draft.exercises.indices.contains(index) ? draft.exercises[index][keyPath: keyPath] : 0 },
275                 set: {
276                     guard draft.exercises.indices.contains(index) else { return }
277                     draft.exercises[index][keyPath: keyPath] = $0
278                 }
279             )
280         }
281
282         private func safeOptionalDouble(index: Int, keyPath: WritableKeyPath<Exercise, Double?>) -> Binding<Double> {
283             Binding(
284                 get: { draft.exercises.indices.contains(index) ? (draft.exercises[index][keyPath: keyPath] ?? 0) : 0 },
285                 set: {
286                     guard draft.exercises.indices.contains(index) else { return }
287                     draft.exercises[index][keyPath: keyPath] = $0
288                 }
289             )
290         }
291
292         private func iconFor(_ c: ExerciseCategory) -> String {
293             switch c {
294             case .strength: return "dumbbell.fill"
295             case .cardio:   return "figure.run"
296             case .mobility: return "figure.mind.and.body"
297             }
298         }
299         private func colorFor(_ c: ExerciseCategory) -> Color {
300             switch c {
301             case .strength: return AGColor.amber
302             case .cardio:   return AGColor.teal
303             case .mobility: return AGColor.magenta
304             }
305         }
306
307
308
309         private func save() {
310             errorMessage = nil
311             isSaving = true
312             workoutVM.updateSession(draft) { error in
313                 isSaving = false
314                 if let err = error {
315                     errorMessage = err.localizedDescription
316                     AGToastCenter.shared.show(err.localizedDescription, kind: .error)
317                 } else {
318                     AGToastCenter.shared.show("Workout updated (delta \(xpDelta) XP).",
319                                               kind: .success)
320                     dismiss()
321                 }
322             }
323         }
324     }
```

WorkoutHistoryView.swift

AetherGain/Views/Workouts/WorkoutHistoryView.swift

```swift


import SwiftUI

struct WorkoutHistoryView: View {
    @EnvironmentObject var authVM: AuthViewModel
    @StateObject private var workoutVM = WorkoutViewModel()

    @State private var searchText = ""
    @State private var sortByDate = true
    @State private var sessionToDelete: WorkoutSession?
    @State private var sessionToEdit: WorkoutSession?



    var filteredAndSortedHistory: [WorkoutSession] {
        var filtered = workoutVM.history
        if !searchText.isEmpty {
            filtered = filtered.filter { session in
                session.exercises.contains { $0.type.lowercased().contains(searchText.lowercased()) }
            }
        }
        return filtered.sorted { a, b in
            if sortByDate { return a.date > b.date }
            let ta = a.exercises.first?.type ?? ""
            let tb = b.exercises.first?.type ?? ""
            return ta < tb
        }
    }

    private var totalXP: Int { workoutVM.history.reduce(0) { $0 + $1.totalXP } }
    private var totalSessions: Int { workoutVM.history.count }
    private var thisWeekSessions: Int {
        let cutoff = Calendar.current.date(byAdding: .day, value: -7, to: Date()) ?? Date()
        return workoutVM.history.filter { $0.date >= cutoff }.count
    }
    private var categoryBreakdown: [ExerciseCategory: Int] {
        var counts: [ExerciseCategory: Int] = [.strength: 0, .cardio: 0, .mobility: 0]
        for session in workoutVM.history {
            for ex in session.exercises {
                counts[ex.category ?? .strength, default: 0] += 1
            }
        }
        return counts
```

```
        }

        var body: some View {
            VStack(spacing: 12) {
                aggregatesCard
                searchBar
                sortRow
                ScrollView {
                    LazyVStack(spacing: 10) {
                        if filteredAndSortedHistory.isEmpty {
                            emptyState
                        } else {
                            ForEach(filteredAndSortedHistory) { session in
                                AGSwipeToDelete(onDelete: {
                                    let xp = session.totalXP
                                    workoutVM.deleteSession(session)
                                    AGToastCenter.shared.show("Deleted · refunded \(xp) XP", kind: .info)
                                }) {
                                    sessionCard(session)
                                }
                            }
                        }
                    }
                    .padding(.horizontal, 16)
                    .padding(.bottom, 40)
                }
            }
            .padding(.top, 8)
            .background(AGColor.canvas.ignoresSafeArea())
            .sheet(item: $sessionToEdit) { session in
                WorkoutEditSheet(session: session, workoutVM: workoutVM)
                    .agToastLayer()
            }
            .alert("Delete workout?", isPresented: Binding(
                get: { sessionToDelete != nil },
                set: { if !$0 { sessionToDelete = nil } }
            )) {
                Button("Cancel", role: .cancel) { sessionToDelete = nil }
                Button("Delete", role: .destructive) {
                    if let s = sessionToDelete {
                        let xp = s.totalXP
                        workoutVM.deleteSession(s)
                        AGToastCenter.shared.show("Deleted · refunded \(xp) XP", kind: .info)
                    }
                    sessionToDelete = nil
                }
            } message: {
                Text("This will refund the XP earned from this session.")
            }
            .onAppear { authVM.userDidInteract() }
        }


        private var aggregatesCard: some View {
```

```
            VStack(alignment: .leading, spacing: 10) {
                HStack {
                    Text("HISTORY").kerning(1.6).font(AGFont.hud(11))
                        .foregroundColor(AGColor.amber)
                    Spacer()
                    Text("\(totalSessions) SESSIONS")
                        .kerning(1.2).font(AGFont.hud(10))
                        .foregroundColor(AGColor.inkMuted)
                }
                HStack(spacing: 8) {
                    aggregateStat(icon: "star.fill", label: "TOTAL XP",
                                  value: "\(totalXP)", tint: AGColor.magenta)
                    aggregateStat(icon: "calendar", label: "7 DAYS",
                                  value: "\(thisWeekSessions)", tint: AGColor.teal)
                    aggregateStat(icon: "flame.fill", label: "AVG XP",
                                  value: totalSessions > 0 ? "\(totalXP / totalSessions)" : "0",
                                  tint: AGColor.amber)
                }
                HStack(spacing: 8) {
                    categoryChip(.strength, count: categoryBreakdown[.strength] ?? 0)
                    categoryChip(.cardio,   count: categoryBreakdown[.cardio]   ?? 0)
                    categoryChip(.mobility, count: categoryBreakdown[.mobility] ?? 0)
                }
            }
            .padding(16)
            .background(RoundedRectangle(cornerRadius: 14).fill(AGColor.panel))
            .overlay(RoundedRectangle(cornerRadius: 14).stroke(AGColor.stroke, lineWidth: 1))
            .padding(.horizontal, 16)
        }

        private func aggregateStat(icon: String, label: String, value: String, tint: Color) -> some View {
            VStack(spacing: 4) {
                HStack(spacing: 4) {
                    Image(systemName: icon).foregroundColor(tint)
                    Text(label).kerning(1.2).font(AGFont.hud(9))
                        .foregroundColor(AGColor.inkMuted)
                }
                Text(value).font(AGFont.number(18)).foregroundColor(AGColor.ink)
            }
            .frame(maxWidth: .infinity)
            .padding(.vertical, 8)
            .background(RoundedRectangle(cornerRadius: 10).fill(AGColor.panelHi))
            .overlay(RoundedRectangle(cornerRadius: 10).stroke(tint.opacity(0.3), lineWidth: 1))
        }

        private func categoryChip(_ category: ExerciseCategory, count: Int) -> some View {
            let color = colorFor(category)
            return HStack(spacing: 4) {
                Circle().fill(color).frame(width: 6, height: 6)
                Text(category.displayName.uppercased()).kerning(1.2).font(AGFont.hud(9))
                    .foregroundColor(color)
                Text("\(count)").font(AGFont.mono(11, weight: .semibold))
                    .foregroundColor(AGColor.ink)
            }
            .padding(.horizontal, 8).padding(.vertical, 4)
```

```
            .background(Capsule().fill(color.opacity(0.1)))
            .overlay(Capsule().stroke(color.opacity(0.3), lineWidth: 1))
        }



        private var searchBar: some View {
            HStack(spacing: 10) {
                Image(systemName: "magnifyingglass").foregroundColor(AGColor.inkMuted)
                TextField("Search by exercise", text: $searchText)
                    .textInputAutocapitalization(.never)
                    .disableAutocorrection(true)
                    .font(AGFont.body(14))
                    .foregroundColor(AGColor.ink)
                if !searchText.isEmpty {
                    Button { searchText = "" } label: {
                        Image(systemName: "xmark.circle.fill").foregroundColor(AGColor.inkMuted)
                    }
                    .buttonStyle(.plain)
                }
            }
            .padding(.horizontal, 14).padding(.vertical, 10)
            .background(RoundedRectangle(cornerRadius: 12).fill(AGColor.panel))
            .overlay(RoundedRectangle(cornerRadius: 12).stroke(AGColor.stroke, lineWidth: 1))
            .padding(.horizontal, 16)
        }

        private var sortRow: some View {
            HStack {
                Text("\(filteredAndSortedHistory.count) shown")
                    .font(AGFont.hud(10)).kerning(1).foregroundColor(AGColor.inkMuted)
                Spacer()
                Button {
                    sortByDate.toggle()
                    AGHaptics.tap()
                } label: {
                    HStack(spacing: 4) {
                        Image(systemName: sortByDate ? "calendar" : "text.alignleft")
                        Text(sortByDate ? "BY DATE" : "BY TYPE").kerning(1.2)
                    }
                    .font(AGFont.hud(10))
                    .padding(.horizontal, 10).padding(.vertical, 5)
                    .background(Capsule().fill(AGColor.amber.opacity(0.15)))
                    .overlay(Capsule().stroke(AGColor.amber.opacity(0.4), lineWidth: 1))
                    .foregroundColor(AGColor.amber)
                }
                .buttonStyle(.plain)
            }
            .padding(.horizontal, 16)
        }



        private func sessionCard(_ session: WorkoutSession) -> some View {
            VStack(alignment: .leading, spacing: 10) {
```

```
                HStack {
                    VStack(alignment: .leading, spacing: 2) {
                        Text(session.date, style: .date).font(AGFont.headline(14))
                            .foregroundColor(AGColor.ink)
                        Text(session.date, style: .time)
                            .font(AGFont.hud(9)).kerning(1)
                            .foregroundColor(AGColor.inkMuted)
                    }
                    Spacer()
                    Text("+\(session.totalXP) XP")
                        .font(AGFont.mono(13, weight: .semibold))
                        .foregroundColor(AGColor.teal)
                        .padding(.horizontal, 10).padding(.vertical, 4)
                        .background(Capsule().fill(AGColor.teal.opacity(0.1)))
                        .overlay(Capsule().stroke(AGColor.teal.opacity(0.35), lineWidth: 1))
                    Button {
                        sessionToEdit = session
                        AGHaptics.tap()
                    } label: {
                        Image(systemName: "pencil")
                            .font(.system(size: 12, weight: .semibold))
                            .foregroundColor(AGColor.amber)
                            .padding(6)
                            .background(RoundedRectangle(cornerRadius: 8).fill(AGColor.amber.opacity(0.10)))
                            .overlay(RoundedRectangle(cornerRadius: 8).stroke(AGColor.amber.opacity(0.35), lineWidth: 1))
                    }
                    .buttonStyle(.plain)
                }


                HStack(spacing: 6) {
                    AGHudChip(text: "\(session.exercises.count) EXERCISES", tint: AGColor.amber)
                    if session.exercises.contains(where: { $0.category == .strength }) {
                        let vol = strengthVolume(session)
                        AGHudChip(text: "VOL \(Int(vol)) kg", tint: AGColor.amberHi)
                    }
                    if session.exercises.contains(where: { $0.category == .cardio }) {
                        let mins = cardioMinutes(session)
                        AGHudChip(text: "\(Int(mins)) MIN CARDIO", tint: AGColor.teal)
                    }
                }

                Divider().background(AGColor.stroke)




                ForEach(Array(session.exercises.enumerated()), id: \.offset) { pair in
                    let exercise = pair.element
                    HStack(spacing: 10) {
                        Circle().fill(colorFor(exercise.category ?? .strength))
                            .frame(width: 8, height: 8)
                        Text(exerciseSummary(exercise))
                            .font(AGFont.body(13))
                            .foregroundColor(AGColor.ink)
```

```
                        Spacer()
                    }
                }
                if !session.notes.isEmpty {
                    Text(session.notes)
                        .font(AGFont.body(11)).foregroundColor(AGColor.inkMuted)
                        .padding(.top, 2)
                }
            }
            .padding(14)
            .background(RoundedRectangle(cornerRadius: 12).fill(AGColor.panel))
            .overlay(RoundedRectangle(cornerRadius: 12).stroke(AGColor.stroke, lineWidth: 1))
            .contentShape(Rectangle())
            .onTapGesture {
                sessionToEdit = session
                AGHaptics.tap()
            }
            .contextMenu {
                Button {
                    sessionToEdit = session
                } label: { Label("Edit", systemImage: "pencil") }
                Button(role: .destructive) {
                    sessionToDelete = session
                } label: { Label("Delete", systemImage: "trash") }
            }
        }



        private var emptyState: some View {
            VStack(spacing: 10) {
                Image(systemName: "dumbbell.fill").font(.system(size: 32))
                    .foregroundColor(AGColor.amber.opacity(0.5))
                Text("NO WORKOUTS YET").kerning(1.6).font(AGFont.hud(11))
                    .foregroundColor(AGColor.amber)
                Text("Log your first session in the Log tab.")
                    .font(AGFont.body(12)).foregroundColor(AGColor.inkMuted)
            }
            .padding(40)
            .frame(maxWidth: .infinity)
            .background(RoundedRectangle(cornerRadius: 12).fill(AGColor.panel))
            .overlay(RoundedRectangle(cornerRadius: 12).stroke(AGColor.stroke, lineWidth: 1))
        }



        private func colorFor(_ c: ExerciseCategory) -> Color {
            switch c {
            case .strength: return AGColor.amber
            case .cardio:   return AGColor.teal
            case .mobility: return AGColor.magenta
            }
        }

        private func strengthVolume(_ session: WorkoutSession) -> Double {
```

```
            session.exercises
                .filter { $0.category == .strength }
                .reduce(0) { $0 + Double($1.sets * $1.reps) * max($1.weightKG, 1) }
        }

        private func cardioMinutes(_ session: WorkoutSession) -> Double {
            session.exercises
                .filter { $0.category == .cardio }
                .reduce(0) { $0 + ($1.durationMinutes ?? 0) }
        }

        private func exerciseSummary(_ exercise: Exercise) -> String {
            switch exercise.category ?? .strength {
            case .strength:
                return String(format: "%@ · %dx%d @ %.1f kg",
                              exercise.type, exercise.sets, exercise.reps, exercise.weightKG)
            case .cardio:
                let d = exercise.distanceKM ?? 0
                let t = exercise.durationMinutes ?? 0
                let i = exercise.inclinePercent ?? 0
                return String(format: "%@ · %.1f km · %.0f min · %.0f%% incline",
                              exercise.type, d, t, i)
            case .mobility:
                let t = exercise.durationMinutes ?? 0
                return String(format: "%@ · %.0f min", exercise.type, t)
            }
        }
    }

    #Preview {
        WorkoutHistoryView().environmentObject(AuthViewModel())
    }
```

AetherGainTests.swift

AetherGainTests/AetherGainTests.swift

```
1
2
3
4
5
6
7
8   import Testing
9   @testable import AetherGain
10
11  struct AetherGainTests {
12
13      @Test func example() async throws {
14
15      }
16
17  }
```

AetherGainUITests.swift

AetherGainUITests/AetherGainUITests.swift

```
1
2
3
4
5
6
7
8   import XCTest
9
10  final class AetherGainUITests: XCTestCase {
11
12      override func setUpWithError() throws {
13
14
15
16          continueAfterFailure = false
17
18
19      }
20
21      override func tearDownWithError() throws {
22
23      }
24
25      @MainActor
26      func testExample() throws {
27
28          let app = XCUIApplication()
29          app.launch()
30
31
32      }
33
34      @MainActor
35      func testLaunchPerformance() throws {
36
37          measure(metrics: [XCTApplicationLaunchMetric()]) {
38              XCUIApplication().launch()
39          }
40      }
41  }
```

AetherGainUITestsLaunchTests.swift

AetherGainUITests/AetherGainUITestsLaunchTests.swift

```
1
2
3
4
5
6
7
8     import XCTest
9
10    final class AetherGainUITestsLaunchTests: XCTestCase {
11
12        override class var runsForEachTargetApplicationUIConfiguration: Bool {
13            true
14        }
15
16        override func setUpWithError() throws {
17            continueAfterFailure = false
18        }
19
20        @MainActor
21        func testLaunch() throws {
22            let app = XCUIApplication()
23            app.launch()
24
25
26
27
28            let attachment = XCTAttachment(screenshot: app.screenshot())
29            attachment.name = "Launch Screen"
30            attachment.lifetime = .keepAlways
31            add(attachment)
32        }
33    }
```

check_dataset.py

MLTraining/check_dataset.py

```
#!/usr/bin/env python3
"""
Checks the training set before train_model.swift runs.

Layout
------
Training/<FoodName>/                the class label
Training/<FoodName>/<variation>/    optional, counted towards the same label

Every image below Training/<FoodName>/ trains as the single label <FoodName>.
The class name must match a `name` in FoodDatabase.swift exactly, otherwise the
model emits a label that bestMatch() cannot resolve to any macros.
"""

import os
import re
import sys

EXTENSIONS = {".jpg", ".jpeg", ".png", ".heic"}
MIN_PER_CLASS = 25          # below this Create ML's validation split gets noisy
MIN_PER_VARIATION = 5
IMBALANCE_FACTOR = 3        # largest class vs smallest

HERE = os.path.dirname(os.path.abspath(__file__))
TRAINING = os.path.join(HERE, "Training")
FOOD_DB = os.path.join(HERE, os.pardir, "AetherGain", "Services", "FoodDatabase.swift")


def database_names():
    """Names defined in FoodDatabase.swift, or None if the file is missing."""
    try:
        with open(FOOD_DB, encoding="utf-8") as handle:
            source = handle.read()
    except OSError:
        return None
    return {m.lower() for m in re.findall(r'FoodMacros\(name:\s*"([^"]+)"', source)}


def count_images(directory):
    """(total, {variation: count}) for everything below `directory`."""
    per_variation = {}
    total = 0
    for root, dirs, files in os.walk(directory):
        dirs[:] = [d for d in dirs if not d.startswith(".")]
        images = [f for f in files
                  if os.path.splitext(f)[1].lower() in EXTENSIONS
                  and not f.startswith(".")]
        if not images:
            continue
        relative = os.path.relpath(root, directory)
```

```
            label = "(top level)" if relative == "." else relative
            per_variation[label] = len(images)
            total += len(images)
        return total, per_variation


    def main():
        if not os.path.isdir(TRAINING):
            sys.exit("Training/ is missing.")

        known = database_names()
        classes = sorted(d for d in os.listdir(TRAINING)
                         if os.path.isdir(os.path.join(TRAINING, d)) and not d.startswith("."))
        if not classes:
            sys.exit("No class folders under Training/.")

        problems, notes, counts = [], [], {}
        width = max(len(c) for c in classes)

        for name in classes:
            total, variations = count_images(os.path.join(TRAINING, name))
            counts[name] = total

            print(f"{name.ljust(width)}  {total:>4} images")
            if len(variations) > 1 or "(top level)" not in variations:
                for variation, n in sorted(variations.items()):
                    print(f"{' ' * width}    {variation}: {n}")
                    if n < MIN_PER_VARIATION:
                        notes.append(f"{name}/{variation}: only {n} images ? "
                                     f"too few to teach the variation anything")

            if total == 0:
                problems.append(f"{name}: empty ? Create ML fails on an empty class, "
                                f"delete the folder or fill it")
            elif total < MIN_PER_CLASS:
                notes.append(f"{name}: {total} images, {MIN_PER_CLASS} recommended")

            if known is not None and name.lower() not in known:
                problems.append(f"{name}: no entry with this name in FoodDatabase.swift ? "
                                f"the model would emit a label nothing can resolve")

        print("-" * (width + 14))
        print(f"{'total'.ljust(width)}  {sum(counts.values()):>4} images in {len(classes)} classes")

        populated = {k: v for k, v in counts.items() if v}
        if len(populated) >= 2:
            smallest, largest = min(populated.values()), max(populated.values())
            if largest > smallest * IMBALANCE_FACTOR:
                weakest = min(populated, key=populated.get)
                notes.append(f"imbalance: largest class has {largest} images, "
                             f"{weakest} has {smallest} ? the model will favour the big ones")
        if len(populated) < 2:
            problems.append("fewer than two classes have images ? nothing to train")

        if problems:
```

```
            print("\nFix first:")
            for p in problems:
                print("  -", p)
        if notes:
            print("\nWorth improving:")
            for n in notes:
                print("  -", n)

        if problems:
            sys.exit(1)
        print("\nGood to go:  swift train_model.swift")


    if __name__ == "__main__":
        main()
```

prepare_dataset.py

MLTraining/prepare_dataset.py

```
#!/usr/bin/env python3
"""
prepare_dataset.py ? fold an external image set into MLTraining/Training.

It does NOT download anything. Point it at a folder you already have: a
dataset you downloaded yourself under its own licence, or a folder of your
own photos. It cleans that folder up and files the images under the class
names this project uses.

    python3 prepare_dataset.py ~/Downloads/indian-food --make-map
    # edit mapping.json, then
    python3 prepare_dataset.py ~/Downloads/indian-food --map mapping.json

Expected source layout ? one subfolder per class, any names:

    indian-food/
      masala_dosa/  *.jpg
      plain_rice/   *.jpg
      ...

What it does per image
    - opens and verifies it (corrupt files are skipped, not copied)
    - drops anything smaller than --min-size on the short side
    - drops near-duplicates using a difference hash, within and across classes
    - converts to RGB JPEG, longest side --max-size
    - writes to Training/<FoodDatabase name>/<class>_ext_NNN.jpg

It never overwrites a file that is already there, so your own photos are safe.

Class names in `mapping.json` must match a `name` in FoodDatabase.swift, since
that is what bestMatch() compares the model's label against.
"""

from __future__ import annotations

import argparse
import json
import os
import re
import sys
from collections import defaultdict

try:
    from PIL import Image
except ImportError:
    sys.exit("Pillow is missing.  pip3 install Pillow")

EXTENSIONS = {".jpg", ".jpeg", ".png", ".webp", ".bmp", ".heic"}
HERE = os.path.dirname(os.path.abspath(__file__))
DEFAULT_OUT = os.path.join(HERE, "Training")
```

```
    FOOD_DB = os.path.join(HERE, os.pardir, "AetherGain", "Services", "FoodDatabase.swift")


    # ---------------------------------------------------------------- FoodDatabase

    def database_names() -> list[str]:
        """The `name` of every FoodMacros entry, in file order."""
        try:
            with open(FOOD_DB, encoding="utf-8") as handle:
                source = handle.read()
        except OSError:
            return []
        return re.findall(r'FoodMacros\(name:\s*"([^"]+)"', source)


    def normalise(text: str) -> str:
        return re.sub(r"[^a-z0-9]+", " ", text.lower()).strip()


    def guess_mapping(source_classes: list[str], known: list[str]) -> dict[str, str]:
        """Best-effort source folder -> FoodDatabase name, for --make-map.

        Only offers a guess when the normalised names share a whole word, so a
        wrong guess is obvious rather than plausible. Everything else is left
        empty for you to fill in or delete.
        """
        by_word: dict[str, str] = {}
        for name in known:
            for word in normalise(name).split():
                by_word.setdefault(word, name)

        mapping: dict[str, str] = {}
        for folder in source_classes:
            words = normalise(folder).split()
            hit = ""
            for word in words:
                if word in by_word:
                    hit = by_word[word]
                    break
            mapping[folder] = hit
        return mapping


    # ---------------------------------------------------------------- image tools

    def dhash(image: Image.Image, size: int = 8) -> int:
        """Difference hash: resize to (size+1, size), compare adjacent pixels.

        Two images that differ only in compression, small crops or resizing land
        on the same hash, which is what we want for de-duplicating a scraped set.
        """
        grey = image.convert("L").resize((size + 1, size), Image.LANCZOS)
        pixels = grey.tobytes()
        bits = 0
        for row in range(size):
```

```
            offset = row * (size + 1)
            for col in range(size):
                bits = (bits << 1) | int(pixels[offset + col] > pixels[offset + col + 1])
        return bits


    def hamming(a: int, b: int) -> int:
        return bin(a ^ b).count("1")


    # ---------------------------------------------------------------- main passes

    def collect(source_root: str) -> dict[str, list[str]]:
        """{source class folder: [image paths]} for immediate subfolders."""
        found: dict[str, list[str]] = {}
        for entry in sorted(os.listdir(source_root)):
            folder = os.path.join(source_root, entry)
            if not os.path.isdir(folder) or entry.startswith("."):
                continue
            images = []
            for root, dirs, files in os.walk(folder):
                dirs[:] = [d for d in dirs if not d.startswith(".")]
                for name in sorted(files):
                    if os.path.splitext(name)[1].lower() in EXTENSIONS and not name.startswith("."):
                        images.append(os.path.join(root, name))
            if images:
                found[entry] = images
        return found


    def existing_hashes(out_root: str, max_side: int) -> tuple[set[int], list[int]]:
        """Hashes of what is already in Training/, so we never re-add a photo."""
        seen: list[int] = []
        for root, dirs, files in os.walk(out_root):
            dirs[:] = [d for d in dirs if not d.startswith(".")]
            for name in files:
                if os.path.splitext(name)[1].lower() not in EXTENSIONS:
                    continue
                try:
                    with Image.open(os.path.join(root, name)) as image:
                        seen.append(dhash(image))
                except Exception:
                    continue
        return set(seen), seen


    def main() -> None:
        parser = argparse.ArgumentParser(description=__doc__,
                                         formatter_class=argparse.RawDescriptionHelpFormatter)
        parser.add_argument("source", help="folder with one subfolder per class")
        parser.add_argument("--map", dest="mapping", default=None,
                            help="JSON of {source folder: FoodDatabase name}")
        parser.add_argument("--make-map", action="store_true",
                            help="write a starter mapping.json and stop")
        parser.add_argument("--out", default=DEFAULT_OUT, help="Training/ directory")
```

```
        parser.add_argument("--cap", type=int, default=200,
                            help="maximum images to add per class (default 200)")
        parser.add_argument("--min-size", type=int, default=200,
                            help="skip images whose short side is under this (default 200)")
        parser.add_argument("--max-size", type=int, default=512,
                            help="longest side after resizing (default 512)")
        parser.add_argument("--hash-distance", type=int, default=4,
                            help="0 = byte-identical only, higher = more aggressive (default 4)")
        parser.add_argument("--dry-run", action="store_true", help="report, write nothing")
        args = parser.parse_args()

        if not os.path.isdir(args.source):
            sys.exit(f"{args.source} is not a folder.")

        sources = collect(args.source)
        if not sources:
            sys.exit("No class subfolders with images under that path.")

        known = database_names()

        if args.make_map:
            mapping = guess_mapping(list(sources), known)
            path = os.path.join(HERE, "mapping.json")
            with open(path, "w", encoding="utf-8") as handle:
                json.dump(mapping, handle, indent=2, ensure_ascii=False, sort_keys=True)
            filled = sum(1 for v in mapping.values() if v)
            print(f"Wrote {path}")
            print(f"{len(mapping)} source classes, {filled} guessed, "
                  f"{len(mapping) - filled} left blank.")
            print("Edit it: put a FoodDatabase name on the right, or leave it blank to skip")
            print("that class. Names must match FoodDatabase.swift exactly.")
            if known:
                print(f"\nAvailable names ({len(known)}):")
                print("  " + ", ".join(known))
            return

        if not args.mapping:
            sys.exit("Need --map mapping.json (run --make-map first).")
        with open(args.mapping, encoding="utf-8") as handle:
            mapping = json.load(handle)

        unknown = sorted({v for v in mapping.values() if v and known and v not in known})
        if unknown:
            print("These targets have no entry in FoodDatabase.swift:")
            for name in unknown:
                print("  -", name)
            print("Add them to FoodDatabase.all or fix the spelling, then run again.\n")
            sys.exit(1)

        seen_hashes, hash_list = ([], []) if args.dry_run else existing_hashes(args.out, args.max_size)
        hash_list = list(hash_list)

        kept = defaultdict(int)
        skipped = defaultdict(lambda: defaultdict(int))

```

```
        for source_class, images in sources.items():
            target = mapping.get(source_class, "")
            if not target:
                skipped[source_class]["unmapped"] = len(images)
                continue

            destination = os.path.join(args.out, target)
            if not args.dry_run:
                os.makedirs(destination, exist_ok=True)

            index = 0
            for path in images:
                if kept[target] >= args.cap:
                    skipped[source_class]["over cap"] += 1
                    continue
                try:
                    with Image.open(path) as image:
                        image.load()
                        if min(image.size) < args.min_size:
                            skipped[source_class]["too small"] += 1
                            continue
                        digest = dhash(image)
                        if any(hamming(digest, other) <= args.hash_distance for other in hash_list):
                            skipped[source_class]["duplicate"] += 1
                            continue

                        image = image.convert("RGB")
                        longest = max(image.size)
                        if longest > args.max_size:
                            scale = args.max_size / longest
                            image = image.resize((max(1, round(image.width * scale)),
                                                  max(1, round(image.height * scale))),
                                                 Image.LANCZOS)

                        if not args.dry_run:
                            while True:
                                index += 1
                                name = f"{target.lower().replace(' ', '_')}_ext_{index:03d}.jpg"
                                candidate = os.path.join(destination, name)
                                if not os.path.exists(candidate):
                                    break
                            image.save(candidate, "JPEG", quality=90)

                        hash_list.append(digest)
                        kept[target] += 1
                except Exception as error:
                    skipped[source_class][f"unreadable ({type(error).__name__})"] += 1

        print("Added" + (" (dry run, nothing written)" if args.dry_run else ""))
        width = max((len(k) for k in kept), default=10)
        for name in sorted(kept):
            print(f"  {name.ljust(width)}  {kept[name]:>4}")
        print(f"  {'total'.ljust(width)}  {sum(kept.values()):>4}")

        if skipped:
```

```
            print("\nSkipped")
            for source_class in sorted(skipped):
                reasons = ", ".join(f"{n} {why}" for why, n in sorted(skipped[source_class].items()))
                print(f"  {source_class}: {reasons}")

        if known:
            covered = {v for v in mapping.values() if v}
            print(f"\nCoverage: {len(covered)} of {len(known)} FoodDatabase entries have a class.")

        print("\nNext:  python3 check_dataset.py   then   swift train_model.swift")


    if __name__ == "__main__":
        main()
```

train_model.swift

MLTraining/train_model.swift

```swift
#!/usr/bin/env swift
//
//  train_model.swift
//  AetherGain
//
//  Trains an image classifier from MLTraining/Training and writes
//  FoodClassifier.mlmodel next to the MLTraining folder.
//
//  Run from inside MLTraining:
//      swift train_model.swift
//
//  Folder layout
//  -------------
//  Training/<FoodName>/                    <- the class label
//  Training/<FoodName>/<variation>/        <- optional, any depth
//
//  Every image found anywhere below Training/<FoodName>/ trains as the single
//  label <FoodName>. Variation subfolders exist so you can keep "plain" and
//  "masala" dosa apart while you are collecting, without splitting the class.
//  The script copies everything into a flat staging directory before training,
//  because Create ML's labeledDirectories reads one level only.
//
//  IMPORTANT: <FoodName> must match a `name` in FoodDatabase.swift exactly.
//  The model emits the folder name as its label, and FoodDatabase.bestMatch
//  compares that label against those names.
//
//  If a variation has materially different macros (masala dosa is not plain
//  dosa), it needs its own FoodDatabase entry and its own top-level folder.
//  Keep it as a subfolder only when the macros are the same.
//

import CreateML
import Foundation

let fm = FileManager.default
let imageExtensions: Set<String> = ["jpg", "jpeg", "png", "heic"]

let cwd = URL(fileURLWithPath: fm.currentDirectoryPath)
let trainingDir = cwd.appendingPathComponent("Training")
let outputURL = cwd.deletingLastPathComponent()
    .appendingPathComponent("AetherGain")
    .appendingPathComponent("AetherGainFoodModel.mlmodel")

guard fm.fileExists(atPath: trainingDir.path) else {
    print("ERROR: Training/ not found. Run this script from inside MLTraining.")
    exit(1)
}

// MARK: - Flatten variation subfolders into one directory per class

```

```
    let stagingDir = fm.temporaryDirectory
        .appendingPathComponent("AetherGainDataset-\(UUID().uuidString)")

    func imageURLs(under directory: URL) -> [URL] {
        guard let walker = fm.enumerator(at: directory,
                                         includingPropertiesForKeys: [.isRegularFileKey],
                                         options: [.skipsHiddenFiles]) else { return [] }
        return walker.compactMap { $0 as? URL }
            .filter { imageExtensions.contains($0.pathExtension.lowercased()) }
    }

    var classCounts: [String: Int] = [:]

    do {
        try fm.createDirectory(at: stagingDir, withIntermediateDirectories: true)

        let classDirs = try fm.contentsOfDirectory(at: trainingDir,
                                                   includingPropertiesForKeys: [.isDirectoryKey],
                                                   options: [.skipsHiddenFiles])
            .filter { (try? $0.resourceValues(forKeys: [.isDirectoryKey]).isDirectory) == true }
            .sorted { $0.lastPathComponent < $1.lastPathComponent }

        for classDir in classDirs {
            let label = classDir.lastPathComponent
            let images = imageURLs(under: classDir)
            guard !images.isEmpty else {
                print("SKIPPED \(label): no images ? an empty folder makes Create ML fail.")
                continue
            }

            let destination = stagingDir.appendingPathComponent(label)
            try fm.createDirectory(at: destination, withIntermediateDirectories: true)

            for (index, image) in images.enumerated() {
                // Keep the variation in the file name so a misfiled photo is easy
                // to trace back after training.
                let variation = image.deletingLastPathComponent().lastPathComponent
                let suffix = (variation == label) ? "" : "_\(variation)"
                let name = "\(label)\(suffix)_\(String(format: "%03d", index + 1)).\(image.pathExtension)"
                try fm.copyItem(at: image, to: destination.appendingPathComponent(name))
            }
            classCounts[label] = images.count
        }
    } catch {
        print("ERROR while preparing the dataset: \(error)")
        exit(1)
    }

    guard classCounts.count >= 2 else {
        print("ERROR: need at least two classes with images, found \(classCounts.count).")
        try? fm.removeItem(at: stagingDir)
        exit(1)
    }

    print("Dataset")
```

```
    for (label, count) in classCounts.sorted(by: { $0.key < $1.key }) {
        print("  \(label.padding(toLength: 14, withPad: " ", startingAt: 0)) \(count) images")
    }
    print("  total: \(classCounts.values.reduce(0, +)) images in \(classCounts.count) classes")
    print("")
    print("Output: \(outputURL.path)")
    print("Training, this takes a few minutes...")
    print("")

    // MARK: - Train

    do {
        let data = MLImageClassifier.DataSource.labeledDirectories(at: stagingDir)
        let classifier = try MLImageClassifier(trainingData: data)

        let trainingAccuracy = (1.0 - classifier.trainingMetrics.classificationError) * 100
        let validationAccuracy = (1.0 - classifier.validationMetrics.classificationError) * 100
        print(String(format: "Training accuracy:   %.1f %%", trainingAccuracy))
        if validationAccuracy.isNaN {
            // With very few images per class the automatic split can leave the
            // validation set empty, and Create ML reports NaN. Train on more
            // photos before trusting the model.
            print("Validation accuracy: not measured ? too few images for a split.")
        } else {
            print(String(format: "Validation accuracy: %.1f %%", validationAccuracy))
        }

        let metadata = MLModelMetadata(
            author: "Dhriti",
            shortDescription: "AetherGain food classifier trained on the client's own meals.",
            version: "1.0"
        )
        if fm.fileExists(atPath: outputURL.path) {
            try fm.removeItem(at: outputURL)
        }
        try classifier.write(to: outputURL, metadata: metadata)

        print("")
        print("Done. AetherGainFoodModel.mlmodel is in the AetherGain/ source folder.")
        print("That folder is a synchronized group, so Xcode picks the model up on")
        print("the next build ? nothing to drag, nothing to tick.")

        if validationAccuracy.isNaN || validationAccuracy < 70 {
            print("")
            print("WARNING: below 70 % validation accuracy this model is not worth bundling.")
            print("A confidently wrong model is worse than the Vision fallback.")
            print("Add more photos per class, or drop the classes with the fewest.")
        }
    } catch {
        print("ERROR during training: \(error)")
        try? fm.removeItem(at: stagingDir)
        exit(1)
    }

    try? fm.removeItem(at: stagingDir)
```

generate_report.py

../analysis/generate_report.py

```
#!/usr/bin/env python3
"""
AetherGain weekly analysis (Criterion 8).

Reads a CSV exported from the app (Progress tab -> Export CSV) and produces:
  1. A CSV summary of weekly totals and averages.
  2. A PDF report with weekly trend charts (XP, workouts, macros, water, sleep).

Usage:
    python3 generate_report.py aethergain_weekly.csv --out report.pdf

CSV schema (11 columns):
    date,day,xp,workouts,calories,protein_g,carbs_g,fat_g,water_ml,sleep_hours,habit_done

Dependencies:
    pip install pandas matplotlib
"""

import argparse
import os
import sys
from datetime import datetime

import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages


PALETTE = {
    "coral":    "#FF8C7A",
    "lavender": "#B7AEFA",
    "mint":     "#8CDAB5",
    "sky":      "#8CCCF9",
    "peach":    "#FFC6A5",
    "sun":      "#FFD966",
}


def load(csv_path: str) -> pd.DataFrame:
    df = pd.read_csv(csv_path)
    df["date"] = pd.to_datetime(df["date"])
    df = df.sort_values("date").reset_index(drop=True)
    return df


def summarise(df: pd.DataFrame) -> dict:
    logged_meals = df[df["calories"] > 0]
    logged_sleep = df[df["sleep_hours"] > 0]
```

```
        return {
            "range_start": df["date"].min().strftime("%Y-%m-%d"),
            "range_end":   df["date"].max().strftime("%Y-%m-%d"),
            "total_xp":       int(df["xp"].sum()),
            "total_workouts": int(df["workouts"].sum()),
            "avg_calories":   round(logged_meals["calories"].mean(), 1) if not logged_meals.empty else 0.0,
            "avg_protein":    round(logged_meals["protein_g"].mean(), 1) if not logged_meals.empty else 0.0,
            "avg_carbs":      round(logged_meals["carbs_g"].mean(), 1) if not logged_meals.empty else 0.0,
            "avg_fat":        round(logged_meals["fat_g"].mean(), 1) if not logged_meals.empty else 0.0,
            "avg_water_ml":   round(df["water_ml"].mean(), 0),
            "avg_sleep":      round(logged_sleep["sleep_hours"].mean(), 1) if not logged_sleep.empty else 0.0,
            "habit_hit_days": int(df["habit_done"].sum()),
        }


    def write_summary_csv(summary: dict, out_path: str) -> None:
        pd.DataFrame([summary]).to_csv(out_path, index=False)


    def build_pdf(df: pd.DataFrame, summary: dict, out_path: str) -> None:
        with PdfPages(out_path) as pdf:
            # --- Cover / summary page ---
            fig = plt.figure(figsize=(8.5, 11))
            ax = fig.add_subplot(111)
            ax.axis("off")
            title = f"AetherGain Weekly Report\n{summary['range_start']} ? {summary['range_end']}"
            ax.text(0.5, 0.92, title, ha="center", va="top", fontsize=20, fontweight="bold")

            stats_lines = [
                f"Total XP earned:         {summary['total_xp']}",
                f"Workouts completed:      {summary['total_workouts']}",
                f"Days habits completed:   {summary['habit_hit_days']}",
                "",
                f"Avg calories / logged day:  {summary['avg_calories']:.0f} kcal",
                f"Avg protein:                {summary['avg_protein']:.0f} g",
                f"Avg carbs:                  {summary['avg_carbs']:.0f} g",
                f"Avg fat:                    {summary['avg_fat']:.0f} g",
                "",
                f"Avg water (today snapshot): {summary['avg_water_ml']:.0f} ml",
                f"Avg sleep / logged day:     {summary['avg_sleep']:.1f} h",
            ]
            ax.text(0.08, 0.78, "\n".join(stats_lines), va="top", fontsize=13,
                    family="monospace")
            ax.text(0.5, 0.04,
                    f"Generated {datetime.now().strftime('%Y-%m-%d %H:%M')}",
                    ha="center", va="bottom", fontsize=9, color="gray")
            pdf.savefig(fig)
            plt.close(fig)

            # --- XP + workouts page ---
            fig, axes = plt.subplots(2, 1, figsize=(8.5, 11))
            axes[0].bar(df["day"], df["xp"], color=PALETTE["lavender"])
            axes[0].set_title("Daily XP")
            axes[0].set_ylabel("XP")
            axes[1].bar(df["day"], df["workouts"], color=PALETTE["mint"])
```

```
            axes[1].set_title("Workouts per day")
            axes[1].set_ylabel("Sessions")
            for ax in axes:
                ax.spines["top"].set_visible(False)
                ax.spines["right"].set_visible(False)
            fig.tight_layout()
            pdf.savefig(fig)
            plt.close(fig)

            # --- Macros page ---
            fig, ax = plt.subplots(figsize=(8.5, 6))
            ax.plot(df["day"], df["protein_g"], marker="o", color=PALETTE["lavender"], label="Protein (g)")
            ax.plot(df["day"], df["carbs_g"],   marker="o", color=PALETTE["peach"],    label="Carbs (g)")
            ax.plot(df["day"], df["fat_g"],     marker="o", color=PALETTE["sky"],      label="Fat (g)")
            ax.set_title("Macros trend")
            ax.set_ylabel("Grams")
            ax.legend()
            ax.spines["top"].set_visible(False)
            ax.spines["right"].set_visible(False)
            fig.tight_layout()
            pdf.savefig(fig)
            plt.close(fig)


    def main(argv=None) -> int:
        parser = argparse.ArgumentParser(description="AetherGain weekly analysis report")
        parser.add_argument("csv", help="Path to CSV exported from the app")
        parser.add_argument("--out", default="report.pdf", help="Output PDF path")
        parser.add_argument("--summary", default="summary.csv",
                            help="Output CSV summary path")
        args = parser.parse_args(argv)

        if not os.path.isfile(args.csv):
            print(f"Input CSV not found: {args.csv}", file=sys.stderr)
            return 1

        df = load(args.csv)
        if df.empty:
            print("No rows in input CSV.", file=sys.stderr)
            return 1

        summary = summarise(df)
        write_summary_csv(summary, args.summary)
        build_pdf(df, summary, args.out)
        print(f"Summary CSV: {args.summary}")
        print(f"PDF report:  {args.out}")
        return 0


    if __name__ == "__main__":
        sys.exit(main())
```